PRÁCTICAS DE ARQUITECTURA DE COMPUTADORES II

Sistemas embebidos con STM32

Prácticas de Arquitectura de Computadores II

VOLUMEN 1 · DEL ALUMNADO

Universidade de Vigo · ESEI · Ourense

Créditos

Libro de prácticas de **Arquitectura de Computadores II**, del Grado en Ingeniería Informática de la Universidade de Vigo, impartida en la Escola Superior de Enxeñaría Informática de Ourense.

El material está centrado en el microcontrolador **STM32F401RE**, sobre la placa NUCLEO-F401RE y el módulo LED&KEY con controlador TM1638.

Autor	Eduardo Balvís Outeiriño
Departamento	Ingeniería de Sistemas y Automática

Volumen	1 · Del alumnado
Documentación de referencia	STMicroelectronics: RM0368 (manual de referencia del F401) y UM1724 (manual de la placa NUCLEO)

Contenido

Créditos 2

Prólogo 10

1. El microcontrolador 12

1.1 En qué se diferencia de un procesador de ordenador 12

1.2 El núcleo Cortex-M4 13

1.3 El mapa de memoria 14

1.4 Qué ocurre al encender 15

1.5 Lo que trae el F401 y lo que se usa 16

2. La placa y el módulo 17

2.1 Qué es una placa Nucleo 17

2.2 El ST-LINK: tres cosas por un solo cable 18

2.3 Lo que la placa ya trae cableado 18

2.4 Los conectores 19

2.5 El módulo LED&KEY 20

2.6 El conexionado de las prácticas 21

2.7 Cómo se graba un programa 22

3. El entorno 23

3.1 Las tres piezas 24

3.2 El fichero de configuración 24

3.3 Qué genera CubeMX 25

3.4 La HAL: qué te ahorra y qué te cuesta 26

3.5 El contrato USER CODE 27

3.6 Debug y Release 27

3.7 Depurar 28

4. El reloj del sistema 29

4.1 Por qué no hay un solo reloj 29

4.2 De dónde parte la señal 29

4.3 El PLL: de 16 MHz a 84 30

4.4 El reparto por los buses 31
4.5 La espera de la memoria Flash 32
4.6 Qué depende del reloj y qué no 32
4.7 Cómo saber a qué frecuencia va un proyecto 33
5. Entradas y salidas digitales 34
5.1 Un pin no es solo un cable 34
5.2 Por qué una entrada al aire no vale cero 35
5.3 El rebote 36
5.4 Nivel y flanco 37
5.5 La velocidad de conmutación 38
6. El tiempo 38
6.1 El SysTick 39
6.2 Lo que hace la HAL con él 39
6.3 El límite de 24 bits 40
6.4 Una base de tiempo que no derive 41
6.5 Por qué la resta funciona aunque el contador desborde 42
6.6 Deriva y temblor 42
6.7 Cuándo esto no basta 43
7. Interrupciones 43
7.1 Preguntar o que te avisen 43
7.2 Qué ocurre exactamente 44
7.3 El controlador de interrupciones 45
7.4 Las líneas externas 45
7.5 Qué se puede hacer dentro y qué no 46
7.6 La frontera: volatile 47
7.7 El rebote también interrumpe 48
8. Temporizadores 48
8.1 Qué hay dentro 48
8.2 Cómo se calcula un periodo 49
8.3 El reloj que recibe no es el del bus 50

8.4 Cuántos bits 51
8.5 El aviso llega a una función compartida 51
8.6 Otros modos, en una línea 52
8.7 Temporizador o SysTick 52
9. Conversión analógica-digital 53
9.1 Cómo se convierte una tensión en un número 53
9.2 La resolución, y el 4095 que no es 4096 54
9.3 El tiempo de muestreo 55
9.4 El reloj del conversor 55
9.5 Las tres formas de leerlo 55
9.6 Los canales de dentro del chip 57
9.7 Por qué el valor no para quieto 58
10. El TM1638 por dentro 58
10.1 Por qué tres hilos y no sesenta 58
10.2 Cómo se envía un byte 59
10.3 Las tres familias de orden 60
10.4 La memoria del display 61
10.5 Un byte por dígito 62
10.6 Por qué faltan letras 62
10.7 Las teclas 63
10.8 El brillo 63
11. Depurar 64
11.1 Por qué probar sale caro 64
11.2 Qué te da el depurador 65
11.3 El método 66
11.4 Puntos de ruptura que valen más que uno normal 67
11.5 Lo que cambia al depurar un microcontrolador 67
11.6 Cuando el depurador no llega 68
11.7 Del síntoma a la comprobación 69
11.8 Cuando el depurador es el que engaña 70

11.9 Depurar sin la placa 70
11.10 Lo que conviene llevarse 83
12. Cómo se construye una biblioteca 84
12.1 El problema que resuelve 84
12.2 La idea central: capas 85
12.3 Qué se enseña y qué se esconde 86
12.4 Por qué una estructura y no unas constantes 86
12.5 Cuando la biblioteca tiene que recordar 87
12.6 La parte que no se puede tocar 88
12.7 Qué hacer con lo imposible 89
12.8 La tabla de caracteres, y sus límites 89
12.9 Si tuvieras que escribir una 89
Práctica 1 · Primer proyecto: el .ioc, CubeMX y el reloj 92
1. Ficha de la práctica 92
2. Qué se construye 92
3. De dónde sale el código 93
4. El reloj: el dato que no está en el código 95
5. Desarrollo paso a paso 97
6. Criterios de aceptación 102
7. Criterios de evaluación 103
8. Entregables 104
9. Uso de asistentes de inteligencia artificial 104
Anexo A. Las claves del .ioc que conviene reconocer 106
Anexo B. Qué hay en la carpeta del proyecto 107
Anexo C. Conexionado 107
Práctica 2 · Control de una puerta de garaje 109
1. Ficha de la práctica 109
2. Objetivos de aprendizaje 109
3. El sistema que hay que construir 110
4. Modelo del sistema 110

5. Comportamiento exigido 112
6. Desarrollo paso a paso 114
7. Criterios de aceptación 118
8. Criterios de evaluación 119
9. Entregables 120
10. Uso de asistentes de inteligencia artificial 121
Anexo A. La biblioteca TM1638 122
Anexo B. Conexionado 123
Anexo C. El protocolo del TM1638 125
Práctica 3 · Cronómetro con precisión de centésimas 128
1. Ficha de la práctica 128
2. Objetivos de aprendizaje 128
3. El problema 129
4. Modelo del sistema 129
5. Comportamiento exigido 130
6. Desarrollo paso a paso 131
7. Criterios de aceptación 134
8. Criterios de evaluación 134
9. Entregables 135
10. Uso de asistentes de inteligencia artificial 136
Anexo A. La función HAL_GetTick() 137
Anexo B. Por qué el error es exactamente del 9,1 % 137
Anexo C. Conexionado 138
Práctica 4 · Interrupciones: el pulsador, SysTick y TIM2 139
1. Ficha de la práctica 139
2. Qué se construye 139
3. Los tres mecanismos, y por qué son tres 140
4. Desarrollo paso a paso 142
5. Criterios de aceptación 145
6. Criterios de evaluación 146

7. Entregables 147
8. Uso de asistentes de inteligencia artificial 147
Anexo A. Lo mínimo dentro de una interrupción 149
Anexo B. Versión para PC 149
Anexo C. Conexionado 149
Práctica 5 · Conversión analógica-digital 151
1. Ficha de la práctica 151
2. Objetivos de aprendizaje 151
3. Del mundo físico al número 152
4. El ADC del STM32F401RE 152
5. Las tres formas de leer el ADC 156
6. Desarrollo paso a paso 158
7. Criterios de aceptación 160
8. Criterios de evaluación 161
9. Entregables 162
10. Uso de asistentes de inteligencia artificial 163
Anexo A. Configuración del ADC en CubeMX 164
Anexo B. Conexionado 164
Anexo C. El sensor de temperatura interno 166
Práctica 6 · Ordenador de a bordo 168
1. Ficha de la práctica 168
2. Qué se construye 168
3. El modelo del vehículo 169
4. Por qué la interrupción es imprescindible aquí 171
5. Desarrollo paso a paso 172
6. Criterios de aceptación 173
7. Criterios de evaluación 174
8. Entregables 175
9. Uso de asistentes de inteligencia artificial 175
Anexo A. Versión para PC 177

Anexo B. Ampliaciones 177
Anexo C. Conexionado 177
Práctica 7 · Semáforo de salida: tiempo de reacción 179
1. Ficha de la práctica 179
2. De qué va 179
3. Cómo es una partida 180
4. El problema interesante: ¿de dónde sale el azar? 181
5. Medir bien el tiempo de reacción 182
6. Desarrollo paso a paso 183
7. Criterios de aceptación 185
8. Criterios de evaluación 186
9. Entregables 187
10. Uso de asistentes de inteligencia artificial 187
Anexo A. Ampliaciones, para quien acabe pronto 189
Anexo B. Conexionado 189
Anexo A. El juego de caracteres del TM1638 191
Lo que no se puede escribir 192
Anexo B. Conexionado completo 192
Alimentación 193
Anexo C. Glosario 194

Prólogo

Este libro reúne las siete prácticas de la asignatura y, por delante, los fundamentos que hacen falta para entenderlas. Las prácticas se pueden hacer siguiendo solo su enunciado; los capítulos de la primera parte están para cuando quieras saber **por qué** las cosas son como son.

La primera parte se puede leer de corrido o consultar por capítulos: cada uno se sostiene solo. La segunda son las siete prácticas, tal como se entregan en cada sesión.

PARTE I

Fundamentos

1. El microcontrolador

Un microcontrolador no es un ordenador pequeño. Se parece más a un ordenador entero metido en una sola pastilla, con la diferencia de que renuncia a casi todo lo que un ordenador tiene de sobra y añade cosas que un ordenador no tiene. Este capítulo cuenta qué hay dentro del que usa la asignatura y por qué está hecho así.

1.1 En qué se diferencia de un procesador de ordenador

	ORDENADOR	MICROCONTROLADOR
Memoria	Gigabytes, en pastillas aparte	Kilobytes, dentro del mismo chip
Sistema operativo	Siempre	Casi nunca: tu programa **es** todo el software
Arranque	Segundos	Milisegundos
Periféricos	Se conectan por buses estándar	Van dentro: temporizadores, conversores, puertos serie
Qué optimiza	Ir rápido	Consumir poco y responder a tiempo

La consecuencia práctica más importante es la del sistema operativo: **no hay ninguno**. Nadie reparte el tiempo entre programas, nadie protege una zona de memoria de otra y nadie avisa si te sales de un array. Tu programa arranca, se queda con la máquina entera y no la suelta hasta que se apague. Por eso el bucle principal nunca termina.

> Esto también explica por qué un fallo en un microcontrolador se manifiesta de formas tan raras. Escribir fuera de un array no da un aviso: sobrescribe otra variable, y el síntoma aparece en una parte del programa que no tiene nada que ver.

1.2 El núcleo Cortex-M4

El chip lo fabrica STMicroelectronics, pero el núcleo que lleva dentro lo diseña ARM. Es un reparto habitual: ARM diseña procesadores y los licencia, y cada fabricante les añade los periféricos que quiere.

Eso tiene una ventaja concreta para quien aprende: **lo que sepas del núcleo sirve para chips de otros fabricantes**. El SysTick, el controlador de interrupciones y el juego de instrucciones son los mismos en un Cortex-M4 de cualquier marca. Lo que cambia son los periféricos.

CARACTERÍSTICA	QUÉ SIGNIFICA
32 bits	Registros y direcciones de 32 bits. Un `int` son cuatro bytes
Arquitectura de carga y almacenamiento	Para operar con un dato hay que traerlo a un registro. No se opera directamente sobre la memoria
Coma flotante en hardware	Sabe multiplicar y dividir números reales sin emularlo
Juego de instrucciones compacto	Casi todas ocupan dos bytes, para que quepa más programa en poca Flash

La coma flotante no sale gratis

La unidad de coma flotante trabaja con **precisión simple**: los `float`. Un `double` no la usa, se emula por software, y eso es muchísimo más lento.

> Escribir `3.3` en una expresión hace que el cálculo entero sea de doble precisión, porque en C una constante decimal es un `double` salvo que se diga otra cosa. Con `3.3f` se queda en precisión simple. Es una `f` que puede multiplicar por diez el tiempo de un cálculo, y no da ningún aviso.

1.3 El mapa de memoria

Todo lo que el procesador puede tocar vive en **un solo espacio de direcciones** de 4 GB. No solo la memoria: también los registros de los periféricos. Encender un LED es escribir en una dirección, igual que guardar una variable.



DÓNDE CAE CADA COSA EN EL ESPACIO DE DIRECCIONES

ZONA	QUÉ GUARDA	AL APAGAR
Flash	El programa y las constantes. 512 KB	Se conserva
SRAM	Las variables, la pila y el montón. 96 KB	**Se pierde**
Periféricos	Los registros con los que se gobierna cada periférico	No aplica
Núcleo	El SysTick y el controlador de interrupciones	No aplica

Que los periféricos estén en el mapa de memoria es lo que hace posible la HAL: `HAL_GPIO_WritePin` acaba siendo una escritura en una dirección concreta. La biblioteca no hace magia, solo te ahorra saberte la dirección y el bit.

96 KB no son pocos hasta que lo son

Parece poco comparado con un ordenador, y para lo que hacen las prácticas sobra. Empieza a no sobrar en cuanto aparece algo que crece: un array grande, una cadena por cada elemento de una lista, o una biblioteca que reserve memoria por su cuenta.

> Aquí no hay memoria virtual. Si la pila crece hasta chocar con las variables, **no hay ningún aviso**: simplemente se corrompen datos. Es la razón de que en sistemas empotrados se evite reservar memoria dinámicamente y se prefiera saber, al compilar, cuánto va a ocupar todo.

1.4 Qué ocurre al encender

Entre que llega la alimentación y que se ejecuta la primera línea de `main` pasan varias cosas, y ninguna la has escrito tú:

PASO	QUÉ OCURRE
1	El procesador lee de las dos primeras posiciones de memoria el valor inicial de la pila y la dirección donde empezar
2	Salta ahí, a una rutina de arranque escrita en ensamblador que viene con el proyecto
3	Esa rutina copia a la SRAM las variables con valor inicial, y pone a cero las que no lo tienen
4	Ajusta el reloj del sistema
5	Y por fin llama a `main`

El paso 3 explica algo que se da por hecho sin pensarlo: que una variable global sin valor inicial valga cero. No vale cero por arte de magia, vale cero **porque alguien la ha puesto a cero** durante el arranque. Y por eso ese proceso lleva su tiempo, poco pero no nulo, proporcional a cuántas variables haya.

> La tabla de direcciones del paso 1 es la misma que se usa para las interrupciones, y está al principio de la Flash. Ahí está la explicación de que las rutinas de atención tengan nombres fijos, como se cuenta en el capítulo 7: la tabla apunta a esos nombres.

1.5 Lo que trae el F401 y lo que se usa

El chip lleva bastantes más periféricos de los que la asignatura toca. Conviene saber que están, aunque solo sea para no reinventarlos:

PERIFÉRICO	¿SE USA?	EN QUÉ PRÁCTICA
Puertos de entrada y salida	Sí	Todas
Temporizadores	Sí, TIM2	P4 y P6
Conversor analógico-digital	Sí	P5 y P6
DMA	Sí, de pasada	P5, en una de las tres variantes
Puerto serie (USART)	No, pero está cableado	—
I^2C, SPI, USB, temporizadores avanzados	No	—

La familia STM32 es muy amplia y todos sus miembros comparten la misma HAL. Un programa escrito para este chip **se mueve a otro de la familia con poco esfuerzo**: lo que cambia son los pines y qué periféricos hay, no la forma de usarlos. Es una de las razones de aprender sobre esta familia y no sobre un chip aislado.

2. La placa y el módulo

Todo el material de la asignatura corre sobre dos piezas: una placa **NUCLEO-F401RE** y un módulo **LED&KEY** con controlador TM1638. Este capítulo cuenta qué hay dentro de cada una y por qué el conexionado es el que es. Los enunciados dan por sabido lo que aquí se explica.

2.1 Qué es una placa Nucleo

Una placa Nucleo no es un producto terminado: es una **placa de evaluación**. Lleva el microcontrolador, lo mínimo para que arranque, y saca todas sus patillas a conectores para que puedas engancharle lo que quieras. No tiene pantalla, ni teclado, ni caja.

Lo que la hace cómoda para aprender es que trae el programador **incorporado**. En un montaje profesional harían falta dos cosas: la placa con el microcontrolador y una sonda de programación aparte, que cuesta más que la placa entera. Aquí van juntas y se conectan con un solo cable USB.

DATO	VALOR
Microcontrolador	`STM32F401RET6`
Núcleo	ARM Cortex-M4 con unidad de coma flotante
Frecuencia máxima	**84 MHz**
Memoria de programa	512 KB de Flash
Memoria de datos	96 KB de SRAM
Alimentación de los pines	**3,3 V**
Programador	ST-LINK integrado en la propia placa

> **Los pines trabajan a 3,3 V, no a 5.** Es el error más caro que se puede cometer con esta placa: conectar a una entrada algo que entregue 5 V puede estropear el pin, y a veces el chip. Antes de enchufar nada que no sea el módulo de la asignatura, comprueba a qué tensión trabaja.

2.2 El ST-LINK: tres cosas por un solo cable

La mitad superior de la placa, separada por una línea de puntos, es el programador. Al conectar el USB, esa mitad se presenta ante el ordenador como **tres dispositivos a la vez**:

SE VE COMO	PARA QUÉ SIRVE
Una unidad de disco llamada `NODE_F401RE`	Copiar ahí un fichero `.bin` graba el programa. Es la vía más rápida para probar algo ya compilado, sin abrir el entorno
Un puerto serie virtual	Recibe lo que el programa escriba por la USART2. Ninguna práctica lo usa, pero está disponible
Una sonda de depuración	Es lo que permite parar el programa, mirar variables y avanzar línea a línea desde STM32CubeIDE

> Que la unidad de disco aparezca **no significa que el programa funcione**. Al copiar el `.bin`, la unidad desaparece y vuelve a montarse: eso solo indica que la grabación terminó. Si el programa se cuelga nada más arrancar, la unidad sigue apareciendo igual.

2.3 Lo que la placa ya trae cableado

Tres elementos están conectados de fábrica, y las prácticas los usan sin montar nada:

ELEMENTO	PIN	DETALLE
LD2, LED verde de usuario	PA5	Escribir un 1 lo enciende
B1, pulsador azul	PC13	**Activo a cero**: pulsado se lee `GPIO_PIN_RESET`
USART2	PA2 y PA3	Cableada al puerto serie del ST-LINK

El pulsador negro que hay al lado del azul es el de **reinicio**. No se puede leer desde el programa: reinicia el microcontrolador, y es lo que se usa para volver a empezar una medida desde cero.

El pulsador está al revés de lo que parece

B1 no conecta el pin a la alimentación cuando se pulsa, sino a masa. En reposo una resistencia lo mantiene a nivel alto, y al pulsar cae a nivel bajo. Por eso la lectura se compara con `GPIO_PIN_RESET` y no con `GPIO_PIN_SET`:

```
/* Verdadero cuando el boton esta pulsado */
if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET) {
    ...
}
```

> **Esta inversión se cuela en las interrupciones.** Si configuras la línea para que salte en flanco de **subida**, saltará cuando **sueltes** el botón, no cuando lo pulses. El programa parece funcionar pero va siempre un paso por detrás, y leyendo el código no se ve. Para detectar la pulsación, flanco de bajada.

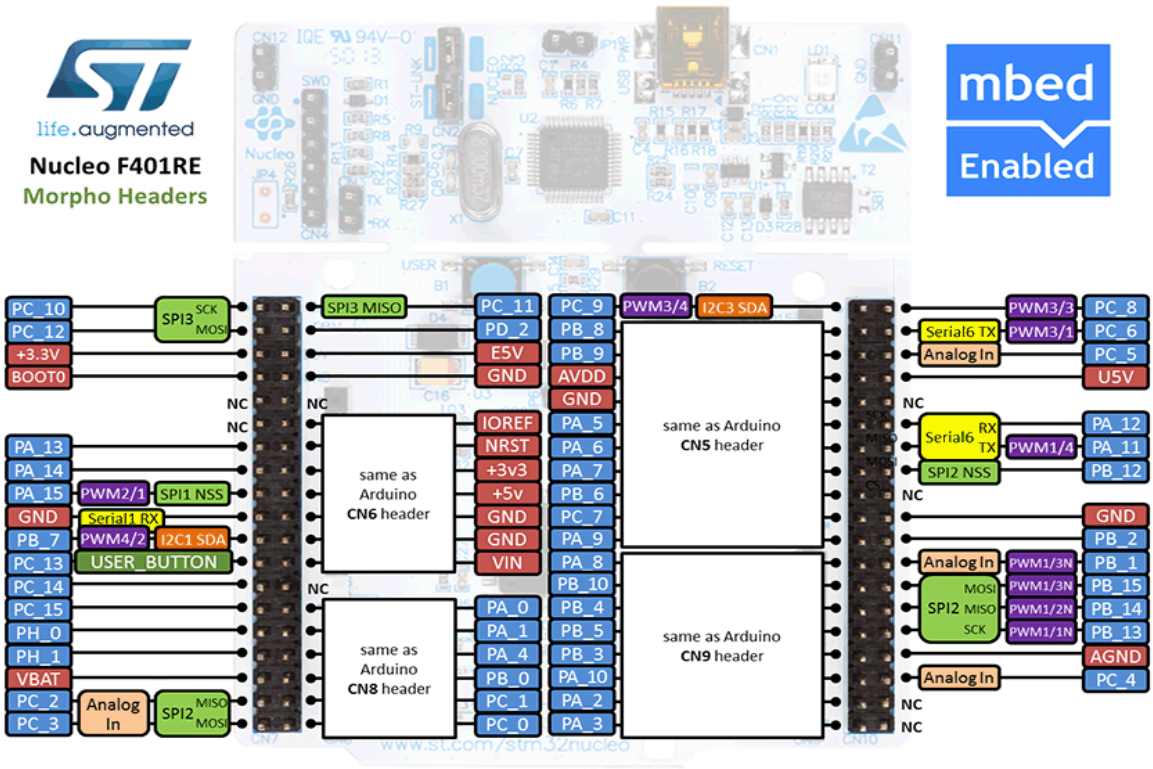
2.4 Los conectores

La placa saca sus patillas por dos juegos de conectores, uno dentro del otro:

- **Los de tipo Arduino**, los cortos de los bordes. Repiten la disposición de una placa Arduino Uno, así que admiten las tarjetas de expansión hechas para ella.

- **Los ST morpho**, las dos tiras largas de dos filas. Sacan **todas** las patillas del microcontrolador, incluidas las que el conector Arduino no alcanza.

Los pines que usan las prácticas están en los dos sitios, así que puedes montar el módulo donde te resulte más cómodo. Lo que no puedes es cambiar **qué pin** usas sin cambiar también el fichero de configuración del proyecto.



LAS PATILLAS DE LA NUCLEO-F401RE

2.5 El módulo LED&KEY

El módulo lleva un controlador **TM1638** que gobierna, él solo, veinticuatro elementos:

RECURSO	CANTIDAD	NOTA
Dígitos de siete segmentos	8	Cada uno con su punto decimal
LEDs rojos	8	Uno encima de cada dígito

RECURSO	CANTIDAD	NOTA
Pulsadores	**S1** a **S8**	Se leen los ocho de una vez

Lo interesante es que todo eso se maneja con **tres hilos**, más alimentación y masa. Sin el controlador harían falta muchos más: solo los ocho dígitos, con sus ocho segmentos, ya serían sesenta y cuatro señales.

HILO	QUÉ HACE	QUIÉN LO MUEVE
CLK	El reloj. Marca cuándo vale el dato	Siempre el microcontrolador
DIO	El dato. **Es bidireccional**	El microcontrolador al escribir; el TM1638 al leer las teclas
STB	Delimita cada orden. En reposo, a nivel alto	Siempre el microcontrolador

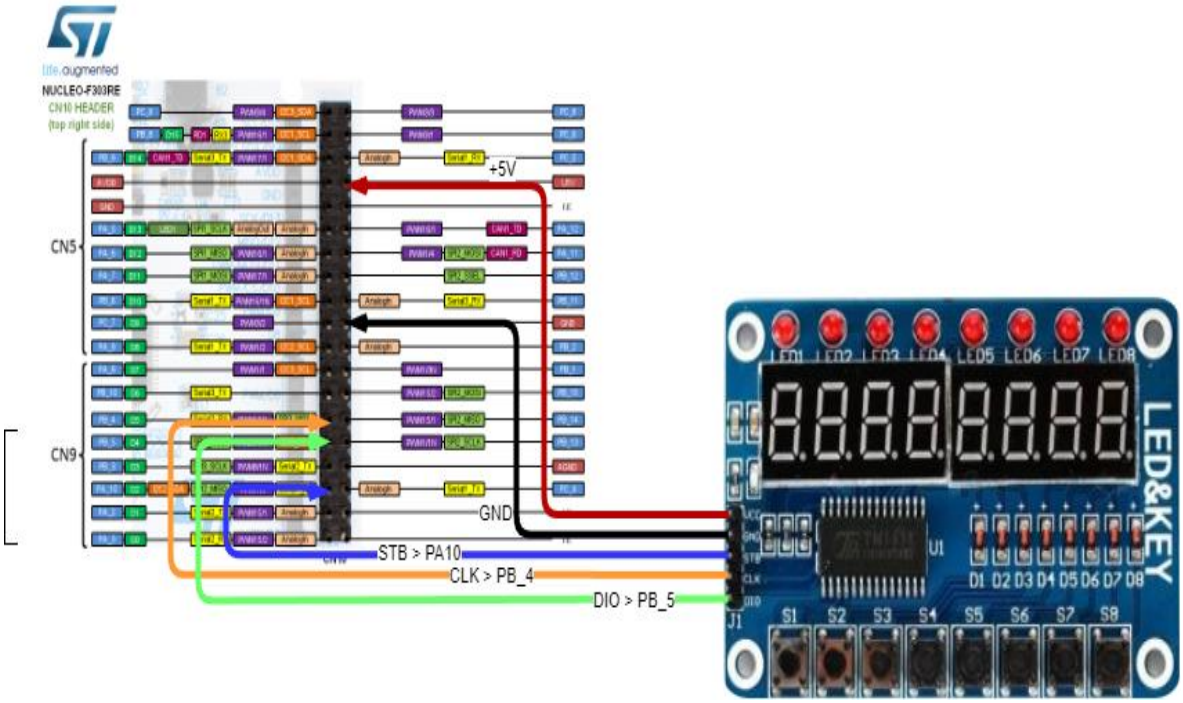
> **DIO cambia de sentido.** Es el único pin de la asignatura que unas veces es salida y otras entrada: el programa lo suelta durante el instante en que lee los pulsadores, y lo recupera después. La biblioteca lo hace por ti, pero explica por qué ese pin se configura como salida en el proyecto y aun así funciona para leer.

2.6 El conexionado de las prácticas

Todas las prácticas usan la misma asignación. Respetándola, cualquier programa del bloque funciona sin tocar nada:

PIN DE LA PLACA	VA A	DESDE QUÉ PRÁCTICA
PB4	CLK del módulo	P2

PIN DE LA PLACA	VA A	DESDE QUÉ PRÁCTICA
PB5	DIO del módulo	P2
PA10	STB del módulo	P2
3V3 y **GND**	Alimentación del módulo	P2
PA1	Cursor del potenciómetro	P5



EL MÓDULO LED&KEY CONECTADO A LA PLACA

> El potenciómetro se conecta como **divisor de tensión**: los extremos a 3V3 y a masa, y el cursor a PA1. Si se conecta solo por dos patillas, la entrada queda flotando y el valor leído salta sin sentido aunque no toques el mando.

2.7 Cómo se graba un programa

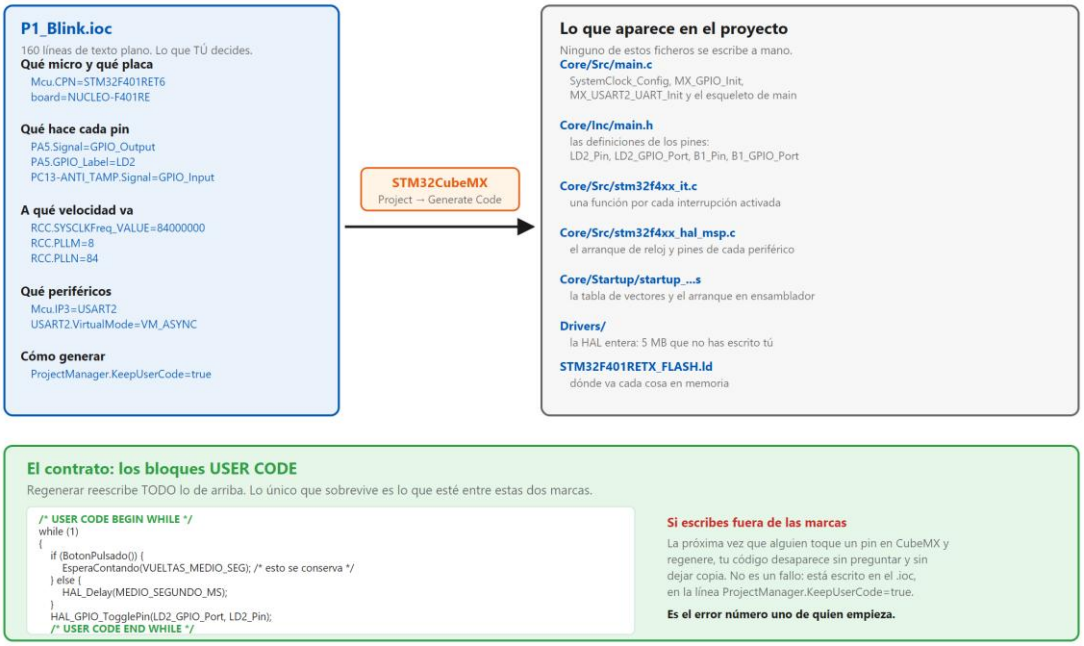
Hay dos vías, y conviene conocer las dos:

VÍA	CUÁNDO	CÓMO
Desde el entorno	Mientras desarrollas	STM32CubeIDE compila y graba en un paso. Es la única que permite depurar
Arrastrando el `.bin`	Para probar algo ya compilado	Se copia el fichero a la unidad `NODE_F401RE`. No hace falta abrir nada

El `.bin` aparece dentro de la carpeta `Debug` del proyecto después de compilar. Es la imagen de memoria tal cual, sin ninguna cabecera: por eso se puede copiar sin más.

3. El entorno

Un proyecto de STM32 no se escribe entero a mano. Una parte la genera una herramienta a partir de un fichero de configuración, y otra la escribes tú. Saber **dónde está la frontera** es lo que separa un proyecto que sobrevive a los cambios de uno que se rompe al primer retoque.



DE DÓNDE SALE CADA PARTE DEL CÓDIGO

3.1 Las tres piezas

PIEZA	QUÉ ES
STM32CubeIDE	El entorno: editor, compilador y depurador. Es donde se trabaja
STM32CubeMX	El configurador. Traduce «quiero este pin como salida» a código de inicialización. **Es un programa aparte**: desde CubeIDE 2.0 ya no va integrado, y un proyecto creado desde CubeIDE sale sin `.ioc`. El proyecto se crea en CubeMX, con **STM32CubeIDE** como Toolchain/IDE, y se importa
La HAL	La biblioteca de ST que da funciones como `HAL_GPIO_WritePin` en lugar de obligarte a escribir en los registros del chip

3.2 El fichero de configuración

El `.ioc` es el fichero que guarda **todo lo que has decidido** sobre el chip: qué pines usas, cómo está el reloj, qué periféricos están encendidos. A pesar de la extensión, no es binario: es texto plano de líneas `clave=valor`.

CINCO LÍNEAS DE UN .IOC

```
Mcu.CPN=STM32F401RET6
PA5.Signal=GPIO_Output
PA5.GPIO_Label=LD2
PC13-ANTI_TAMP.Signal=GPIO_Input
RCC.SYSCLKFreq_VALUE=84000000
```

Que sea texto tiene una consecuencia práctica: se puede **leer** para saber cómo está configurado un proyecto, sin abrir la herramienta gráfica. Y se puede comparar la configuración de dos proyectos con cualquier herramienta de comparación de ficheros.

> **El `.ioc` es la fuente, no el resultado.** Si abres el proyecto y cambias a mano una línea de `MX_GPIO_Init`, tu cambio dura hasta la próxima vez que alguien toque la configuración: al regenerar, CubeMX reescribe esa función a partir del `.ioc` y tu línea desaparece. Lo que hay que cambiar es el `.ioc`.

Cómo se lee un .ioc

PREFIJO	DE QUÉ HABLA
`Mcu.`	El chip: modelo, encapsulado, cuántos pines y periféricos hay declarados
`PA5.`, `PC13-...`	Un pin concreto: su función y la etiqueta que le has puesto
`RCC.`	El reloj: fuentes, multiplicadores, divisores y las frecuencias resultantes
`ProjectManager.`	Cómo se genera el proyecto, incluido si se conserva tu código

3.3 Qué genera CubeMX

Al generar, aparece una estructura de carpetas siempre igual:

```
MiProyecto/
    MiProyecto.ioc          <- la configuracion: LA FUENTE
    Core/
        Inc/                <- las cabeceras, entre ellas main.h
        Src/                <- main.c y las rutinas de interrupcion
    Drivers/                <- la HAL de ST. NO se toca
    Debug/                  <- lo que produce el compilador
```

CARPETA	LA TOCAS	SE PUEDE BORRAR
`Core/Src` y `Core/Inc`	**Sí**, es donde escribes	No
`Drivers`	No	No: sin ella el proyecto no compila

CARPETA	LA TOCAS	SE PUEDE BORRAR
Debug	No	**Sí**, se regenera al compilar. Quítala antes de entregar

> `Drivers` ocupa decenas de megas porque lleva la HAL entera, no solo lo que usas. Es lo que hace que el proyecto compile en cualquier ordenador sin instalar nada, así que no la quites; pero **no la incluyas al entregar** si el enunciado no lo pide.

3.4 La HAL: qué te ahorra y qué te cuesta

Sin la HAL, encender un LED significa escribir en un registro del chip, con el bit correcto y después de haber habilitado el reloj de ese puerto. Con la HAL es una llamada. La diferencia no es solo comodidad: el código con la HAL **se entiende sin el manual delante**.

A cambio se paga algo:

GANAS	PAGAS
Código legible y portable entre chips de la familia	Más instrucciones por operación: una llamada a función donde había una escritura
Comprobaciones de parámetros	Menos control sobre el instante exacto en que ocurre algo
No tener que memorizar registros	Una capa más donde puede estar el fallo

> Ese coste importa poco casi siempre, y mucho de vez en cuando. En la Práctica 2 el módulo se maneja moviendo pines a mano, uno a uno, y ahí **el número de instrucciones que hay entre dos operaciones es el que fija la velocidad de la señal**.

> En sitios así, quitar o añadir código cambia el comportamiento aunque la lógica sea la misma.

3.5 El contrato USER CODE

Es la regla más importante del entorno. CubeMX **reescribe todo lo que hay fuera** de unas marcas cada vez que se regenera el proyecto. Lo que está dentro se conserva.

```
/* USER CODE BEGIN 2 */
    tm1638_DisplayClear(&TM);   /* esto sobrevive */
/* USER CODE END 2 */
```

Las marcas están repartidas por todo `main.c` y por los ficheros de interrupciones, y cada una tiene su sitio:

MARCA	QUÉ VA AHÍ
`Includes`	Tus `#include`
`PV`	Variables globales tuyas
`0`	Funciones auxiliares y constantes
`2`	Inicialización propia, antes del bucle
`WHILE` y `3`	El cuerpo del bucle principal

> **Si algo tiene que ir fuera de las marcas por necesidad, apúntalo.** Es una deuda: alguien la pagará la próxima vez que abra la configuración, y si no está escrito en ninguna parte, el código desaparecerá sin que nadie sepa por qué.

3.6 Debug y Release

El proyecto se puede compilar de dos maneras, y no dan el mismo binario:

	DEBUG	RELEASE
Optimización	Ninguna	Activada
Tamaño	Mayor	Menor
Se puede depurar	Sí, cómodamente	A duras penas: el código no se corresponde línea a línea con el fuente
Para qué	Desarrollar	Entregar o medir de verdad

> **Hay errores que solo aparecen en Release.** El optimizador puede eliminar código que a él le parece inútil pero que tú necesitas: un bucle de espera vacío, o una variable que solo modifica una interrupción. La palabra `volatile` es la forma de decirle que no lo haga, y su efecto **no se puede comprobar en Debug**, porque allí no optimiza nada.

3.7 Depurar

El entorno trae depurador, y es la diferencia entre suponer y saber: permite **parar el programa donde tú digas** y mirar qué está pasando. En un microcontrolador importa más que en un ordenador, porque no hay consola donde escribir ni forma de observar el interior de otra manera.

> **Es lo que evita que la asignatura se haga por ensayo y error.** Merece un capítulo entero, y lo tiene: el **capítulo 11**. Ahí está tanto lo que ofrece la herramienta como el método para acorralar un fallo en vez de ir probando.
>
> **No guardes el proyecto en una ruta con tildes ni eñes.** El depurador no encontrará los ficheros fuente y, al parar en un punto de ruptura, mostrará ensamblador en lugar de tu código en C. No da ningún error ni aviso: simplemente no funciona como esperas. Una carpeta del tipo `C:\ACII` lo evita. Tampoco uses rutas muy largas: pasados los 260

> caracteres que admite Windows, el compilador puede fallar **diciendo que todo ha ido bien**.

4. El reloj del sistema

Un microcontrolador no hace nada sin un reloj. Cada instrucción, cada temporizador y cada conversión del ADC ocurren al ritmo de una señal cuadrada que alguien tiene que generar y repartir. Este capítulo cuenta de dónde sale esa señal en el F401, cómo llega a cada sitio, y por qué conviene saberlo antes de fiarse de cualquier medida de tiempo.

4.1 Por qué no hay un solo reloj

Sería más sencillo que todo el chip funcionase a la misma frecuencia, pero no interesa. Hay dos razones:

- **No todo aguanta la misma velocidad.** El núcleo llega a 84 MHz, pero algunos periféricos no. El conversor analógico-digital del F401, por ejemplo, admite **36 MHz** como máximo.
- **Ir rápido cuesta energía.** Un periférico que solo tiene que refrescar una pantalla no gana nada corriendo al máximo, y consume más.

Por eso el chip tiene un **árbol de reloj**: una señal de partida que se multiplica, se divide y se reparte por ramas, cada una con su frecuencia.

4.2 De dónde parte la señal

El F401 puede arrancar de cuatro fuentes. Las prácticas usan la primera:

FUENTE	QUÉ ES	PRECISIÓN
HSI	Oscilador interno de 16 MHz. Va dentro del chip	Regular: depende de la temperatura
HSE	Cristal externo. Hay que montarlo fuera	Buena

FUENTE	QUÉ ES	PRECISIÓN
LSI	Oscilador interno lento, unos 32 kHz	Mala. Para el perro guardián
LSE	Cristal externo lento, 32.768 Hz	Muy buena. Para el reloj de calendario

> **Las prácticas usan el HSI**, el interno, porque no exige montar nada. A cambio, su precisión es la que es: si construyes un cronómetro y lo comparas con el reloj del móvil durante una hora, verás derivar el resultado. Para medir minutos no importa; para un reloj de pared, sí. Ahí es donde entra el cristal externo.

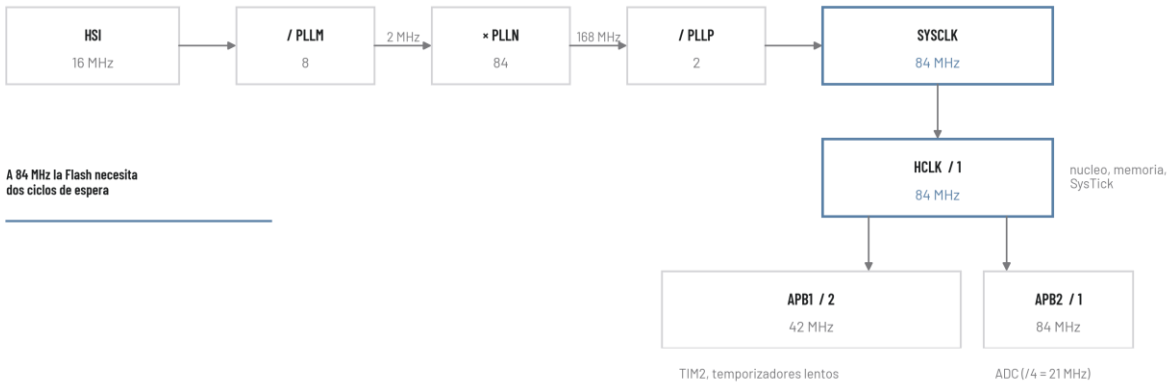
4.3 El PLL: de 16 MHz a 84

Los 16 MHz del oscilador interno no dan para el máximo del chip. Para subir de ahí está el **PLL**, un circuito que multiplica frecuencias. En el F401 se configura con tres números, y se aplican en este orden:

LA CADENA, CON LOS VALORES DE LAS PRÁCTICAS

```
16 MHz  /  PLLM 8   =    2 MHz      <- primero DIVIDE
        x  PLLN 84  =  168 MHz      <- luego MULTIPLICA
        /  PLLP 2   =   84 MHz      <- y vuelve a dividir
```

La división inicial no es un capricho: el multiplicador trabaja mejor partiendo de una frecuencia baja, y dividiendo primero se consigue un abanico de resultados mucho más amplio con los mismos tres números.



EL ÁRBOL DE RELOJ, TAL COMO LO USAN LAS PRÁCTICAS

4.4 El reparto por los buses

La salida del PLL es **SYSCLK**, el reloj del sistema. De ahí cuelgan tres ramas, cada una con su divisor:

RAMA	DIVISOR	FRECUENCIA	QUIÉN CUELGA DE ELLA
HCLK	÷1	84 MHz	El núcleo, la memoria y el SysTick
APB1	÷2	**42 MHz**	TIM2 y los periféricos lentos
APB2	÷1	84 MHz	El ADC y los periféricos rápidos

APB1 va a la mitad porque **no admite más**: es un límite del chip, no una elección. Y el ADC, aunque cuelga de APB2 a 84 MHz, tiene su propio divisor porque su máximo son 36:

```
APB2  84 MHz  /  4  =  21 MHz      <- dentro del maximo de 36
```

> **Esto es una fuente clásica de errores.** Con el divisor a 2 saldrían 42 MHz, que está fuera de especificación. El conversor probablemente seguiría dando números —no se

> apaga ni avisa—, pero serían menos fiables, y el fallo aparecería como ruido en las medidas y no como un error.

Un detalle que despista

Cuando un bus APB va dividido, los temporizadores que cuelgan de él reciben **el doble** de su frecuencia. Es una particularidad de esta familia: APB1 va a 42 MHz, pero TIM2 recibe 84. Si calculas un prescalador con 42 y el resultado sale a la mitad de lo esperado, esta es la razón.

4.5 La espera de la memoria Flash

Subir la frecuencia no es solo poner un número más alto. La memoria Flash donde vive el programa **tiene su propia velocidad**, y no puede entregar una instrucción por ciclo cuando el núcleo va a 84 MHz.

La solución es decirle al chip que espere: **dos ciclos** en este caso. Se llama latencia de la Flash y se configura junto con el reloj:

```
HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2);
```

> **Si la latencia se queda corta, el chip no arranca**, o lo hace de forma errática: el núcleo lee instrucciones antes de que la memoria las tenga listas. Lo configura CubeMX a partir del `.ioc`, así que no hay que calcularlo a mano; pero conviene saber que existe, porque explica que la frecuencia no sea un parámetro suelto.

4.6 Qué depende del reloj y qué no

Este es el apartado que hay que llevarse del capítulo. Hay dos formas de medir tiempo en un programa, y **solo una cambia si cambia el reloj**:

FORMA	¿CAMBIA CON EL RELOJ?	POR QUÉ
`HAL_Delay` y `HAL_GetTick`	**No**	La HAL ajusta el SysTick para que interrumpa cada milisegundo sea cual sea la frecuencia
Contar vueltas de un bucle	**Sí**	Cada vuelta cuesta un número fijo de ciclos, y los ciclos duran más o menos según el reloj
Prescaladores de temporizador	**Sí**	Se calculan sobre la frecuencia del bus, no sobre milisegundos
Retardos escritos en ensamblador	**Sí**	Por lo mismo: cuentan ciclos

> **Un proyecto configurado a una frecuencia distinta de la que supones no da ningún síntoma** mientras todo se apoye en `HAL_GetTick`. Solo se rompe lo que cuenta ciclos: un prescalador calculado para 84 MHz y ejecutado a 16 da **5,25 veces** el periodo pretendido. Y el error no aparece donde está la causa, sino en el temporizador, que es lo que hace que cueste tanto encontrarlo.

4.7 Cómo saber a qué frecuencia va un proyecto

Tres formas, de la más rápida a la más fiable:

CÓMO	DÓNDE MIRAR
En el fichero de configuración	La línea `RCC.SYSCLKFreq_VALUE` del `.ioc`. Es texto plano: se lee sin abrir nada
En el código	La función `SystemClock_Config` de `main.c`: los valores de `PLLM`, `PLLN` y `PLLP`

CÓMO	DÓNDE MIRAR
Preguntándoselo al chip	`HAL_RCC_GetHCLKFreq()` devuelve la frecuencia real en tiempo de ejecución. Es la única que no puede mentir

Las dos primeras pueden discrepar si alguien tocó el código sin tocar la configuración, o al revés. La tercera se ejecuta en la placa, así que dice lo que de verdad está pasando: si tienes una duda seria sobre el reloj, ponla en un punto de ruptura y míralo.

5. Entradas y salidas digitales

Encender un LED y leer un pulsador parecen las dos cosas más sencillas que puede hacer un microcontrolador. La primera casi lo es. La segunda esconde el problema que más tiempo hace perder en un laboratorio, y este capítulo lo cuenta entero.

5.1 Un pin no es solo un cable

Cada patilla del chip está conectada a un circuito que puede comportarse de varias maneras, y hay que elegir cuál antes de usarla. Estos son los modos que aparecen en la asignatura:

MODO	QUÉ HACE	DÓNDE SE USA
Salida push-pull	El pin fuerza activamente el nivel alto y el bajo	El LED, y los tres hilos del módulo
Entrada	El pin solo escucha	El pulsador de la placa
Analógico	El pin se desconecta de la lógica digital y va al conversor	El potenciómetro en P5
Función alternativa	El pin lo gobierna un periférico, no tu código	La USART2, aunque las prácticas no la usen

Hay un quinto modo, **open-drain**, que el pin solo puede tirar hacia abajo y necesita una resistencia externa para subir. No aparece en las prácticas, pero es el que usan buses como el I^2C, y explica por qué esos buses llevan siempre dos resistencias.

> El modo se elige en el fichero de configuración, no en el código. Si un pin no responde y el código parece correcto, **lo primero es comprobar en qué modo está declarado**: un pin en modo entrada no se puede escribir, y no da ningún error al intentarlo.

5.2 Por qué una entrada al aire no vale cero

Un pin configurado como entrada y sin nada conectado **no lee 0**. Lee lo que le llegue: ruido eléctrico, acoplamiento de los pines vecinos, la mano acercándose. Se dice que está *flotando*, y su valor cambia solo.

Para que eso no ocurra, el chip lleva resistencias que se pueden activar por programa:

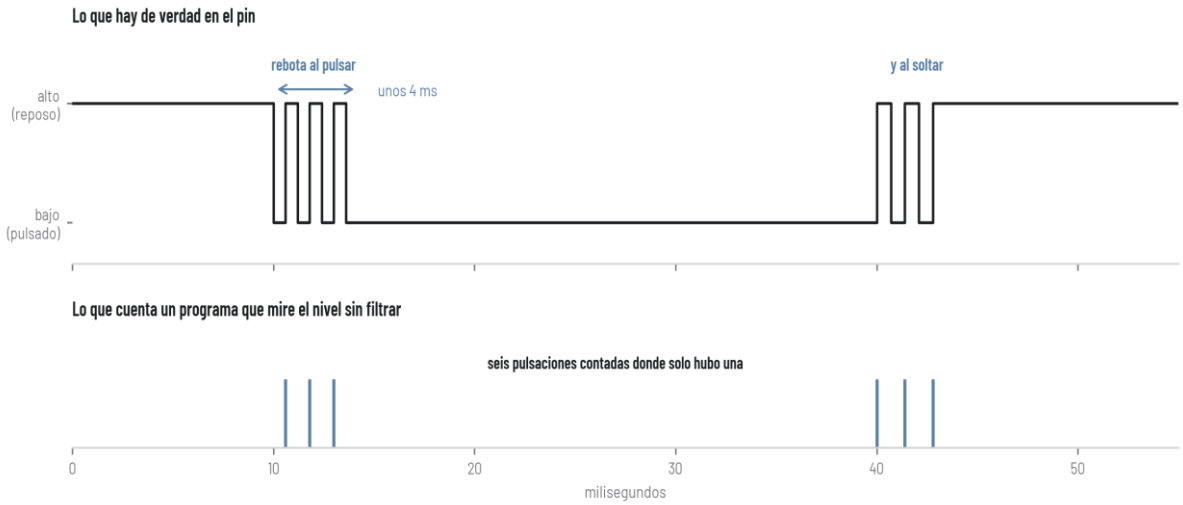
RESISTENCIA	A DÓNDE TIRA EL PIN	CUÁNDO SE USA
Pull-up	Hacia la alimentación: en reposo lee 1	Cuando el pulsador conecta a masa. Es lo habitual
Pull-down	Hacia masa: en reposo lee 0	Cuando el pulsador conecta a la alimentación
Ninguna	El pin flota	Solo si algo externo fija el nivel siempre

> El pulsador B1 de la placa **ya lleva su resistencia montada en la propia placa**, así que el proyecto lo declara sin polarización interna y funciona igual. Con un pulsador que montes tú, en cambio, hay que activar la del chip o poner una fuera: si no, el programa leerá pulsaciones que nadie ha hecho.

Esta es también la razón de que los pulsadores se conecten casi siempre a masa y no a la alimentación, y de que por tanto la lógica salga invertida, como se vio en el capítulo 2.

5.3 El rebote

Un pulsador es dos láminas de metal que se tocan. Al pulsarlo no se tocan de golpe: chocan, separan, vuelven a tocarse, varias veces, durante unos pocos milisegundos. Eléctricamente eso es una ráfaga de pulsos.



EL REBOTE DE UN PULSADOR, Y LO QUE CUENTA EL PROGRAMA

Para una persona es una sola pulsación. Para un programa que mire el pin miles de veces por segundo, son varias, y no hay forma de distinguirlas mirando solo el nivel: **todas son reales**.

SÍNTOMA	QUÉ ESTÁ PASANDO
Un contador sube de tres en tres	Se cuenta cada rebote como una pulsación
Un estado cambia y vuelve al momento	La ráfaga produce varios cambios seguidos
A veces funciona y a veces no	Depende de dónde caiga la ráfaga respecto al bucle

Filtrar sin bloquear

La idea es sencilla: **no creer un cambio hasta que se mantenga un rato**. Unos 30 milisegundos bastan, porque el rebote dura mucho menos y una persona no pulsa dos veces tan rápido.

Lo que no se puede hacer es esperar parando el programa. Un `HAL_Delay(30)` filtraría el rebote y de paso dejaría el resto del programa congelado treinta milisegundos, treinta veces por segundo si alguien mantiene el botón. La forma correcta es recordar cuándo cambió la lectura y comprobar si ya ha pasado el tiempo:

EL ANTIRREBOTE QUE USAN LAS PRÁCTICAS

```
static bool estable = false;   /* el estado que damos por bueno */
static bool ultimo  = false;   /* la ultima lectura cruda       */
static uint32_t tCambio = 0;

bool ahora = BotonPulsado();
if (ahora != ultimo) {         /* la senal se movio: */
    ultimo  = ahora;           /* reinicia la ventana */
    tCambio = HAL_GetTick();
} else if (HAL_GetTick() - tCambio >= 30u) {
    estable = ahora;           /* lleva 30 ms quieta: es de verdad */
}
```

5.4 Nivel y flanco

Filtrar el rebote no basta. Queda una distinción que confunde mucho al principio:

	QUÉ SIGNIFICA	PARA QUÉ SIRVE
Nivel	El botón *está* pulsado	Mientras lo mantengas, algo pasa: acelerar, avanzar deprisa
Flanco	El botón *acaba de* pulsarse	Ocurre una sola vez por pulsación: cambiar de estado, contar

Un programa que quiera contar pulsaciones y mire el nivel contará una por cada vuelta del bucle mientras el dedo siga encima: miles. Lo que hay que detectar es el flanco, y eso es comparar el estado estable con el que había antes:

```
if (estable && !estableAnterior) {   /* justo ahora ha pasado de suelto
a pulsado */
    contador++;
}
estableAnterior = estable;
```

> **Casi todos los fallos con pulsadores son uno de estos tres**: no filtrar el rebote, confundir nivel con flanco, o filtrar bloqueando el programa. Merece la pena comprobarlos en ese orden antes de buscar en otro sitio.

5.5 La velocidad de conmutación

Al declarar una salida se elige también su velocidad, y suele dejarse en la más baja. No es una limitación del programa: es lo rápido que el pin cambia de nivel, y afecta al consumo y al ruido que genera.

Para todo lo que hace la asignatura, la velocidad baja sobra: el protocolo del módulo va a unas decenas de kilohercios, muy lejos del límite. Subirla solo añadiría ruido.

6. El tiempo

Medir tiempo parece resuelto en cuanto existe una función que devuelve los milisegundos transcurridos. No lo está: con esa función se puede construir un cronómetro exacto o uno que atrase un 10 %, y la diferencia son dos caracteres. Este capítulo explica de dónde sale ese número y cómo se usa sin equivocarse.

6.1 El SysTick

Dentro del núcleo Cortex-M hay un temporizador pequeño y siempre presente: el **SysTick**. No es un periférico del fabricante, es parte del propio núcleo, y por eso funciona igual en cualquier chip con este núcleo.

Su funcionamiento es simple. Se le carga un número, cuenta **hacia atrás** hasta cero, genera una interrupción y vuelve a cargarse solo:

ELEMENTO	QUÉ ES
Registro de recarga	El número desde el que empieza a contar
Registro de cuenta	Por dónde va ahora mismo
Interrupción	Salta cada vez que llega a cero

> **El contador es de 24 bits.** Ese detalle, que parece menor, decide lo que se puede y lo que no se puede hacer con él, y se ve en el apartado 6.3.

6.2 Lo que hace la HAL con él

Al arrancar, `HAL_Init` configura el SysTick para que interrumpa **una vez por milisegundo**. La rutina de esa interrupción no hace más que incrementar un contador, y `HAL_GetTick()` devuelve ese contador.

```
uint32_t t = HAL_GetTick();   /* milisegundos desde que arranco la placa
*/
```

De ahí salen dos consecuencias que conviene tener claras:

- **No depende del reloj del sistema.** La HAL calcula la recarga a partir de la frecuencia real, así que un milisegundo dura un milisegundo tanto a 84 MHz como a 16.

- **Su resolución es de un milisegundo.** No sirve para medir nada más corto. Para eso hay que leer el registro de cuenta del propio SysTick, o usar un temporizador.

> El contador es de 32 bits, así que **desborda al cabo de unos 49 días** de funcionamiento continuo. No es un problema si las restas se hacen bien, y el apartado 6.5 explica por qué.

6.3 El límite de 24 bits

Si el contador es de 24 bits, el número más grande que se le puede cargar es:

```
2^24 - 1  =  16 777 215
```

Eso fija el periodo máximo que el SysTick puede generar por sí solo, y ese máximo depende de la frecuencia a la que se le alimente:

FRECUENCIA DEL NÚCLEO	PERIODO MÁXIMO	RECARGA PARA UN SEGUNDO	¿CABE?
84 MHz	**0,1997 s**	83.999.999	**No**
16 MHz	1,0486 s	15.999.999	Sí, por poco

> **Esto es una trampa doble.** A 16 MHz un periodo de un segundo cabe, la configuración se acepta y todo parece funcionar. A 84 MHz no cabe: la función de configuración devuelve un error, y si nadie mira ese valor devuelto, el SysTick se queda como estaba y el programa sigue adelante como si nada. En los dos casos el fallo es silencioso.

La solución no es forzar el SysTick, sino **contar sus interrupciones**. Si interrumpe cada milisegundo, mil interrupciones son un segundo. Es exactamente lo que hace la HAL para su propio contador, y lo que debe hacer cualquier programa que necesite periodos largos.

6.4 Una base de tiempo que no derive

Con `HAL_GetTick()` se construye la cadencia de casi todo. El patrón es este, y los dos detalles que lleva no son opcionales:

```
if (HAL_GetTick() - tAnterior >= PERIODO_MS) {
    tAnterior += PERIODO_MS;      /* sumar el periodo, NO reasignar */
    /* ...lo que toque hacer cada PERIODO_MS... */
}
```

Detalle 1: comparar con >= y no con >

Con > la condición no se cumple a los 10 ms, sino a los 11: hay que **superar** el valor. Cada periodo dura uno de más, y con periodos cortos eso es un porcentaje enorme:

PERIODO	CON > DURA	ERROR
10 ms	11 ms	**10 %**
100 ms	101 ms	1 %
1000 ms	1001 ms	0,1 %

Un cronómetro de centésimas usa un periodo de 10 ms. Con > atrasa seis segundos por minuto, que es imposible no notar; con 1000 ms el mismo error pasaría desapercibido durante mucho tiempo.

Detalle 2: sumar el periodo, no reasignar el reloj

Escribir `tAnterior = HAL_GetTick()` parece equivalente, y no lo es. El programa no comprueba la condición en el instante exacto en que se cumple, sino un poco después: al reasignar, ese sobrante se pierde, y se pierde otra vez en la siguiente vuelta, y en la siguiente.

Sumando el periodo, el sobrante se conserva y la cuenta se recupera sola. El error deja de acumularse y se queda en un pequeño temblor alrededor del valor correcto.

6.5 Por qué la resta funciona aunque el contador desborde

`HAL_GetTick()` devuelve un entero **sin signo** de 32 bits, y la resta también es sin signo. Cuando el contador desborda y vuelve a cero, la resta sigue dando el intervalo correcto por la propia aritmética de complemento a dos.

> Por eso el patrón se escribe siempre como `HAL_GetTick() - tAnterior >= PERIODO` y nunca como `HAL_GetTick() >= tAnterior + PERIODO`. La segunda forma **sí se rompe** al desbordar: la suma desborda por su cuenta y la comparación deja de tener sentido. Son dos expresiones que parecen la misma y no lo son.

6.6 Deriva y temblor

Conviene separar dos errores que se confunden:

	QUÉ ES	SE NOTA
Deriva	Un error que se suma en cada periodo y crece sin parar	Al cabo de un rato: el reloj atrasa cada vez más
Temblor	Cada periodo llega un poco antes o después, pero sin acumularse	Nunca, en la práctica: la media es correcta

El patrón del apartado 6.4 elimina la deriva, no el temblor. Y está bien así: el temblor viene de que el bucle principal tarda lo que tarda en dar la vuelta, y no hay forma de eliminarlo sin usar interrupciones.

Para comprobar cuál de los dos tienes, basta con **medir dos veces**: si el error a los cinco minutos es cinco veces el de un minuto, hay deriva. Si es parecido, es temblor.

6.7 Cuándo esto no basta

SI NECESITAS	USA
Cadencia de milisegundos en el bucle principal	`HAL_GetTick()`, como aquí
Que algo ocurra aunque el bucle esté ocupado	Un temporizador con interrupción (capítulo 8)
Resolución por debajo del milisegundo	Un temporizador, o el registro de cuenta del SysTick
Que el tiempo siga con el chip dormido	El reloj de calendario, que no se ve en la asignatura

7. Interrupciones

Hasta aquí, el programa pregunta: mira el pulsador, mira el reloj, mira otra vez. Una interrupción invierte la relación. El periférico avisa, el procesador deja lo que estaba haciendo, atiende y vuelve. Es un cambio de mentalidad, y trae problemas nuevos que no existían antes.

7.1 Preguntar o que te avisen

	CONSULTAR	INTERRUMPIR
Cómo funciona	El bucle mira el estado cada vuelta	El periférico avisa cuando pasa algo
Latencia	Lo que tarde el bucle en dar la vuelta	Prácticamente inmediata
Coste	Se gasta tiempo mirando aunque no pase nada	Cero mientras no pase nada

	CONSULTAR	INTERRUMPIR
Dificultad	Baja: todo ocurre en un sitio	Alta: hay dos flujos que comparten datos

> **Consultar no es peor.** Es más sencillo y basta casi siempre; en esta asignatura la mayoría de las prácticas lo hacen así. Las interrupciones se usan cuando la latencia importa —medir cuándo alguien pulsa, en la P7— o cuando el bucle no puede garantizar que va a pasar por ahí a tiempo.

7.2 Qué ocurre exactamente

Cuando llega una interrupción, el procesador hace esto sin que nadie se lo pida:

PASO	QUÉ PASA
1	Termina la instrucción que estaba ejecutando
2	Guarda en la pila los registros que va a necesitar recuperar
3	Busca en una tabla la dirección de la rutina que atiende esa interrupción
4	Salta ahí y la ejecuta
5	Restaura lo guardado y **vuelve al punto exacto** donde estaba

Esa tabla de direcciones vive al principio de la memoria de programa y la genera el entorno. Por eso las rutinas de atención tienen **nombres fijos**: la tabla apunta a un nombre concreto, y si escribes uno distinto la tuya no se llamará nunca. No hay error de compilación; simplemente no ocurre nada.

> En esta familia, ese guardado y restauración lo hace el propio hardware, y por eso una rutina de atención se escribe como una función normal de C, sin nada especial. En otras arquitecturas hay que declararla de forma distinta.

7.3 El controlador de interrupciones

El **NVIC** es quien decide a quién se atiende y en qué orden. Cada fuente tiene una prioridad, y la regla es la que sorprende al principio: **número más bajo, más prioridad**.

- Si llegan dos a la vez, se atiende primero la del número menor.
- Si llega una de número menor mientras se atiende otra, la interrumpe: se apila.
- Si llega una de número mayor o igual, espera a que termine la que hay en curso.

> **El SysTick también es una interrupción**, y de ella depende `HAL_GetTick()`. Si una rutina propia tiene más prioridad y tarda demasiado, el contador de milisegundos se queda atrás. Es una de las formas de que el tiempo empiece a medirse mal sin que se toque ninguna línea del código que mide.

7.4 Las líneas externas

Para que un pin genere una interrupción está el **EXTI**. Se configura qué flanco importa y a partir de ahí el pin avisa solo:

FLANCO	CUÁNDO SALTA	CON UN PULSADOR ACTIVO A CERO
Subida	El pin pasa de bajo a alto	Al **soltar** el botón
Bajada	El pin pasa de alto a bajo	Al **pulsar** el botón
Ambos	En los dos casos	Dos avisos por pulsación

Hay una limitación que conviene conocer antes de topársela: **las líneas se comparten por número de pin**. La línea 13 la usan PA13, PB13, PC13… y solo una de ellas puede generar

interrupción a la vez. Si dos pines con el mismo número la necesitan, hay que cambiar uno de sitio.

Además, varias líneas comparten la misma rutina de atención. Por eso lo primero que hace esa rutina es preguntar quién ha sido:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == B1_Pin) {   /* ha sido este, no otro */
        ...
    }
}
```

7.5 Qué se puede hacer dentro y qué no

Una rutina de atención se ejecuta **quitándole el tiempo a todo lo demás**. Mientras está dentro, el bucle principal está parado y las interrupciones de menor prioridad esperan. De ahí la regla: entrar, hacer lo mínimo, salir.

DENTRO SÍ	DENTRO NO
Anotar que algo ha pasado, en una bandera	Esperas de ningún tipo, y `HAL_Delay` menos que ninguna
Guardar el instante con `HAL_GetTick()`	Refrescar un display: es lento
Incrementar un contador	Cálculos largos, o cualquier cosa con coma flotante pesada

> **`HAL_Delay` dentro de una interrupción puede colgar el programa.** Esa función espera a que avance el contador de milisegundos, que avanza gracias a otra interrupción. Si la que está en curso tiene más prioridad, esa otra no llega nunca y la espera no termina. El programa no se cae: se queda quieto, que es peor de diagnosticar.

El patrón correcto es que la rutina solo tome nota, y que el bucle principal haga el trabajo cuando le toque:

```
/* En la rutina de atencion: lo minimo */
hayPulsacion = true;
tPulsacion   = HAL_GetTick();

/* En el bucle principal: el trabajo */
if (hayPulsacion) {
    hayPulsacion = false;
    ...actualizar el display, calcular, lo que haga falta...
}
```

7.6 La frontera: volatile

Toda variable que escriba una interrupción y lea el bucle principal —o al revés— tiene que declararse `volatile`. Sin esa palabra, el compilador puede leerla **una sola vez** y reutilizar el valor, porque desde su punto de vista nadie la modifica dentro del bucle.

```
volatile bool hayPulsacion = false;
```

> **`volatile` no hace la variable segura, solo la hace visible.** Obliga a releerla de memoria cada vez, y eso es todo. No garantiza que una lectura y una escritura no se crucen: si la variable ocupa más de una palabra, o si se modifica en varios pasos, hay que pensar en ello aparte. En esta asignatura basta con banderas y contadores de 32 bits, donde el problema no aparece.

Y hay que recordar lo del capítulo 3: **el efecto de `volatile` solo se ve compilando en Release**. En Debug no hay optimización, así que un programa al que le falte funcionará perfectamente hasta el día en que alguien lo compile para entregar.

7.7 El rebote también interrumpe

Una interrupción por flanco no filtra nada. Si el pulsador rebota seis veces, la rutina se ejecuta seis veces, y encima muy seguidas.

Con interrupciones el antirrebote se hace igual que sin ellas, pero con un detalle más: como la rutina no puede esperar, lo que se compara es el instante:

```
uint32_t ahora = HAL_GetTick();
if (ahora - tUltima >= 30u) {   /* han pasado 30 ms desde la anterior */
    tUltima = ahora;
    hayPulsacion = true;        /* esta si cuenta */
}
```

> Un aviso que cuesta caro descubrir: en la placa **una pulsación puede generar dos eventos distintos** si el programa usa a la vez la interrupción y una lectura filtrada del mismo pin. La interrupción salta con el flanco; la lectura lo confirma unas decenas de milisegundos después. Si dos partes del programa esperan una pulsación seguidas, la primera consume el flanco y la segunda recoge la confirmación de esa misma pulsación, sin que nadie haya tocado nada.

8. Temporizadores

Un temporizador es un contador con reloj propio que sigue funcionando mientras el procesador hace otra cosa. Es la diferencia entre *acordarse* de hacer algo cada cierto tiempo y *que ocurra* cada cierto tiempo, pase lo que pase en el bucle principal.

8.1 Qué hay dentro

En su uso más simple, un temporizador tiene tres piezas:

PIEZA	QUÉ HACE
Prescalador	Divide el reloj de entrada. Sirve para bajar de megahercios a algo manejable
Contador	Cuenta pulsos del reloj ya dividido
Recarga automática	El valor al que el contador vuelve a empezar. Al llegar, genera un evento

El evento puede hacer varias cosas: generar una interrupción, cambiar un pin, disparar el conversor. En la asignatura se usa lo primero.

8.2 Cómo se calcula un periodo

La cuenta es directa, y conviene saberla de memoria:

```
                    reloj de entrada
frecuencia  =  --------------------------------
               (prescalador + 1) x (recarga + 1)
```

Los «+1» están porque los dos registros cuentan desde cero: un prescalador de 0 no divide entre cero, divide entre uno. Es la fuente de error más tonta y más frecuente al empezar.

Un ejemplo con los números de las prácticas:

DATO	VALOR
Reloj que recibe TIM2	84 MHz
Prescalador	8399, es decir divide entre 8400
Frecuencia del contador	84.000.000 / 8400 = **10.000 Hz**
Recarga	9999, es decir cuenta 10.000 pulsos

DATO	VALOR
Periodo resultante	10.000 / 10.000 = **1 segundo**

> Casi siempre hay **varias combinaciones** que dan el mismo periodo. Conviene elegir un prescalador que deje una frecuencia redonda —10 kHz, 1 kHz— porque así el número de recarga se lee de un vistazo y se puede cambiar el periodo sin rehacer la cuenta.

8.3 El reloj que recibe no es el del bus

Aquí está la trampa que más despista. TIM2 cuelga del bus APB1, que va a **42 MHz**. Pero el temporizador no recibe 42, recibe **84**.

Es una particularidad de esta familia: cuando el divisor de un bus APB es distinto de 1, los temporizadores de ese bus reciben el doble de la frecuencia del bus. Si el divisor es 1, reciben la del bus sin más.

BUS	DIVISOR	FRECUENCIA DEL BUS	FRECUENCIA QUE LLEGA AL TEMPORIZADOR
APB1	÷2	42 MHz	**84 MHz**
APB2	÷1	84 MHz	84 MHz

> **Si calculas con 42 MHz, el periodo te sale el doble de lo que esperas.** Y el error no se ve en el código: la fórmula está bien, los registros están bien, lo que está mal es la frecuencia de partida. Ante un periodo que sale exactamente al doble o a la mitad, este es el primer sitio donde mirar.

8.4 Cuántos bits

No todos los temporizadores cuentan hasta el mismo sitio, y eso decide qué periodos se pueden alcanzar sin trucos:

TEMPORIZADOR	BITS	CUENTA HASTA
TIM2 y **TIM5**	32	4.294.967.295
Los demás del F401	16	65.535

Que TIM2 sea de 32 bits es la razón de que sea el que se usa en las prácticas: con 16 bits, y a 10 kHz, el periodo máximo serían unos 6,5 segundos. Con 32 no hay problema práctico.

Compáralo con el SysTick del capítulo 6, que es de **24 bits** y además está atado al reloj del núcleo: a 84 MHz no llega ni a un segundo. Ahí está la diferencia entre los dos.

8.5 El aviso llega a una función compartida

Cuando un temporizador desborda, la HAL llama a una función con nombre fijo. **La misma para todos los temporizadores**, así que lo primero es comprobar cuál ha sido:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {   /* ha sido este */
        ...
    }
}
```

> **Con un solo temporizador la comprobación parece sobrar, y no sobra.** La HAL usa uno internamente para su base de tiempo en algunas configuraciones, y el día que añadas un segundo temporizador, sin esa comprobación los dos ejecutarán el mismo código. Cuesta una línea.

Y hay un paso que se olvida a menudo: **arrancar el temporizador con la interrupción habilitada**. Configurarlo no lo pone en marcha:

```
HAL_TIM_Base_Start_IT(&htim2);   /* sin esto, el callback no se llama nunca */
```

8.6 Otros modos, en una línea

Las prácticas solo usan el temporizador como base de tiempo, pero conviene saber que puede más:

MODO	PARA QUÉ
Base de tiempo	Lo que se usa aquí: un evento periódico
PWM	Generar una señal cuadrada con anchura variable: regular un motor, atenuar un LED, hacer sonar un zumbador
Captura de entrada	Anotar el instante exacto en que cambia un pin. Es la forma de medir tiempos con resolución mejor que el milisegundo
Codificador	Contar los pulsos de un encoder rotatorio, en hardware y sin perder ninguno

8.7 Temporizador o SysTick

SI NECESITAS	USA
Cadencia de milisegundos, y el bucle es rápido	`HAL_GetTick()`. Más simple y suficiente
Que ocurra puntualmente aunque el bucle se atasque	Un temporizador con interrupción

SI NECESITAS	USA
Un periodo que no sea múltiplo del milisegundo	Un temporizador: el prescalador da cualquier frecuencia
Resolución por debajo del milisegundo	Un temporizador
Generar una señal en un pin sin intervenir	Un temporizador en modo PWM

La regla práctica: **empieza siempre por `HAL_GetTick()`**. Es más sencillo de leer y de depurar, y solo cuando algo de la lista de arriba lo impida merece la pena pasar a un temporizador.

9. Conversión analógica-digital

Un pin digital solo distingue dos estados. Un potenciómetro, un sensor de temperatura o un micrófono no dan dos estados: dan una tensión que puede tomar cualquier valor. El conversor analógico-digital es lo que traduce esa tensión a un número con el que se puede trabajar.

9.1 Cómo se convierte una tensión en un número

El conversor del F401 es de **aproximaciones sucesivas**, y funciona como el juego de adivinar un número preguntando «¿es mayor o menor?».

1. Compara la tensión de entrada con la mitad del margen. ¿Está por encima? Primer bit a 1.
2. Compara con la mitad de la mitad que ha quedado. Segundo bit.
3. Y así doce veces, una por bit, acercándose cada vez más.

De ahí salen dos consecuencias prácticas. La primera, que la conversión **no es instantánea**: son doce pasos, y cada uno lleva su tiempo. La segunda, que **tarda siempre**

lo mismo, sea cual sea la tensión, lo que hace muy predecible el momento en que estará listo el resultado.

9.2 La resolución, y el 4095 que no es 4096

Doce bits dan 4.096 valores distintos: del 0 al 4.095. El margen de entrada va de 0 V a la tensión de referencia, que en esta placa son los 3,3 V de alimentación.

TENSIÓN EN EL PIN	VALOR LEÍDO
0 V	0
La mitad, 1,65 V	unos 2.048
3,3 V	**4.095**

> **El máximo es 4095, no 4096.** Parece un detalle y no lo es: dividir entre 4096 al convertir a voltios da un valor que nunca llega a 3,3 aunque el mando esté al tope. El error es pequeño, un 0,02 %, pero es sistemático y aparece justo donde más se mira, en los extremos del recorrido.

La conversión a milivoltios, con aritmética entera y sin coma flotante:

```
uint32_t mv = (valor * 3300u) / 4095u;   /* 0 a 3300 mV */
```

El orden importa: **multiplicar antes de dividir**. Al revés, la división entera se comería los decimales y el resultado saldría a saltos.

Qué significa un bit

Cada unidad del valor leído corresponde a unos **0,8 mV**. Esa es la resolución real, y marca el límite de lo que se puede distinguir: dos tensiones que difieran menos que eso darán el mismo número.

9.3 El tiempo de muestreo

Antes de convertir, el conversor conecta la entrada a un condensador interno y espera a que se cargue. Ese tiempo se puede configurar, y no es un parámetro menor.

Si la fuente de señal tiene mucha resistencia, el condensador se carga despacio, y si no se le da tiempo se convierte una tensión que todavía no ha llegado a su valor. El resultado sale bajo, y de forma consistente, lo que lo hace difícil de reconocer como error.

> Con un potenciómetro de valor moderado, cualquier tiempo de muestreo razonable sirve. Con sensores de alta impedancia hay que consultar la hoja de características. **Ante lecturas sistemáticamente bajas y estables, el tiempo de muestreo es un sospechoso.**

9.4 El reloj del conversor

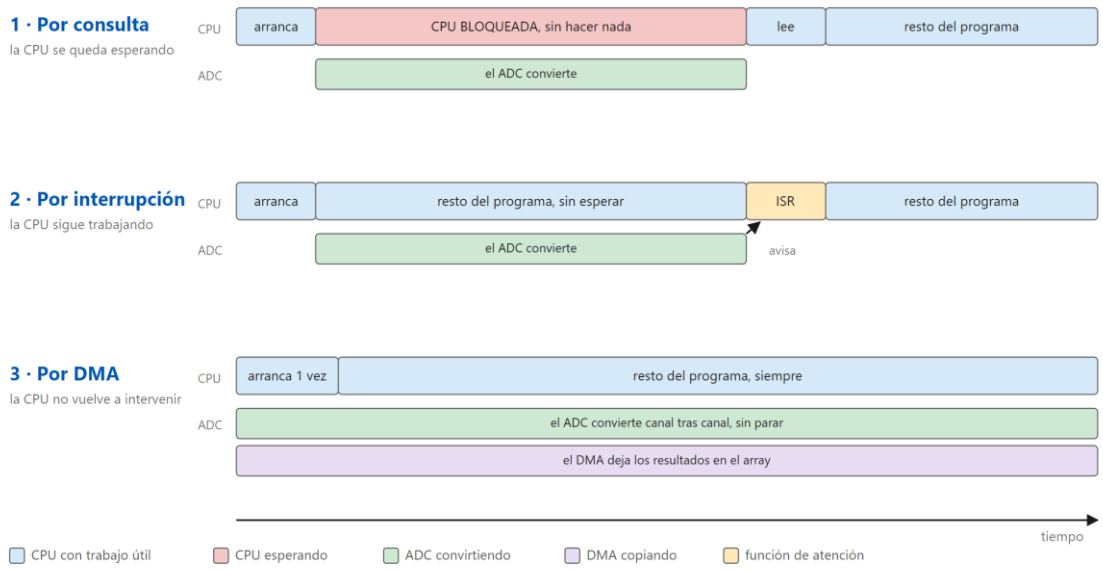
El conversor tiene su propio límite de velocidad: **36 MHz** en el F401. Como cuelga de un bus que va a 84, hace falta dividir:

```
APB2  84 MHz  /  4  =  21 MHz      <- dentro del maximo
APB2  84 MHz  /  2  =  42 MHz      <- FUERA de especificacion
```

Pasarse no apaga nada ni da un error. El conversor sigue dando números, pero deja de garantizar que sean correctos, y el efecto aparece como ruido en las medidas: exactamente donde nadie va a buscar un problema de configuración.

9.5 Las tres formas de leerlo

La conversión tarda. La diferencia entre los tres modos es **qué hace el programa mientras tanto**:



LOS TRES MODOS DE LEER EL CONVERSOR

MODO	QUÉ HACE EL PROGRAMA	CUÁNDO CONVIENE
Consulta	Espera parado a que termine	Una lectura de vez en cuando, y el programa no tiene prisa
Interrupción	Sigue con lo suyo; el conversor avisa al terminar	Cuando el bucle tiene que atender otras cosas
DMA	Ni se entera: los datos van a memoria por su cuenta	Varios canales, o muchas muestras seguidas

El **DMA** merece una explicación aparte, porque es el que suele sorprender. Es un periférico que mueve datos de un sitio a otro sin pasar por el procesador. Configurado una vez, el conversor va dejando los resultados en un array de memoria y el programa los lee cuando quiere, sin haber ejecutado ni una instrucción para traerlos.

> **Comparar los tres modos solo tiene sentido si el bucle no bloquea.** Si el programa lleva una espera dentro, la versión por interrupción no se nota, porque el tiempo que

> ahorra se pierde en la espera. Es el motivo de que las tres versiones de la Práctica 5 usen la misma cadencia no bloqueante.

9.6 Los canales de dentro del chip

No todos los canales del conversor van a un pin. Dos están conectados a fuentes internas, y las dos son útiles:

CANAL INTERNO	QUÉ MIDE	PARA QUÉ SIRVE
Referencia de tensión	Una tensión fija y conocida, generada dentro del chip	Deducir cuánto vale de verdad la alimentación
Sensor de temperatura	La temperatura del propio silicio	Medir sin montar nada. No mide el ambiente

Para qué sirve medir una tensión que ya se conoce

El conversor mide **respecto a su alimentación**. Si esa alimentación no son exactamente 3,3 V —y por USB rara vez lo son—, todas las medidas salen desplazadas.

La referencia interna resuelve eso. Como se sabe cuánto vale, y se sabe qué número devuelve el conversor al medirla, se puede despejar la alimentación real y corregir el resto de lecturas. Es autocalibración, y sale gratis.

> El **sensor de temperatura mide el chip, no la habitación**. Marcará varios grados por encima del ambiente porque el propio microcontrolador se calienta, y subirá si le pones el dedo encima. Como termómetro de una habitación no vale; para detectar que algo se está calentando de más, sí.

9.7 Por qué el valor no para quieto

Al mostrar una lectura en el display se ve enseguida: el número baila aunque nadie toque el mando. No es un fallo. Las últimas cifras recogen ruido eléctrico, y con 0,8 mV por unidad hace falta muy poco para mover una cuenta.

DÓNDE	CUÁNTO BAILA	POR QUÉ
Cerca de 0	Más	La resistencia del divisor es baja y recoge más ruido
A media escala	Menos	
En el tope	Casi nada	El valor está pegado al máximo y no puede subir

Si molesta, hay dos remedios sencillos. **Promediar** varias lecturas seguidas, que reduce el ruido pero también la rapidez de respuesta. O **no mostrar las últimas cifras**: si el usuario no necesita el milivoltio, enseñar décimas de voltio hace desaparecer el problema sin tocar nada.

10. El TM1638 por dentro

Las prácticas usan el módulo a través de una biblioteca, y eso está bien: nadie debería tener que pensar en el protocolo para escribir un número en una pantalla. Este capítulo es para cuando sí hace falta pensarlo, porque algo no sale o porque quieres hacer algo que la biblioteca no contempla.

10.1 Por qué tres hilos y no sesenta

Un display de siete segmentos con punto necesita ocho señales. Ocho dígitos serían sesenta y cuatro, más ocho LEDs, más ocho pulsadores. Ochenta hilos para un módulo que cabe en la palma de la mano.

El TM1638 resuelve eso de dos maneras a la vez:

- **Multiplexa**: no enciende los ocho dígitos a la vez, sino uno detrás de otro muy deprisa. El ojo los ve todos encendidos.
- **Habla en serie**: en vez de un hilo por dato, un hilo por el que pasan los datos uno detrás de otro, con otro hilo que marca el ritmo.

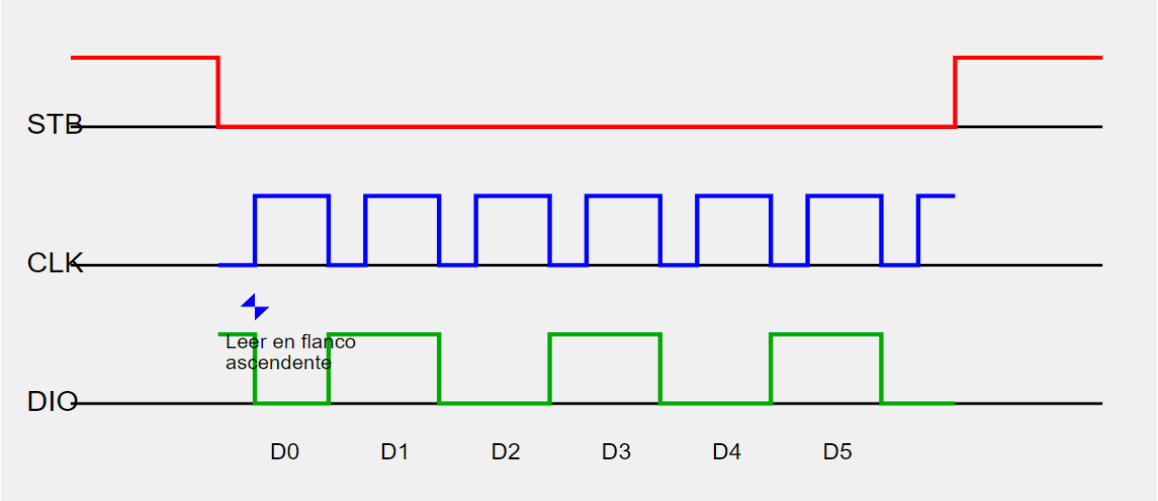
HILO	SENTIDO	QUÉ HACE
CLK	Siempre sale del micro	Marca cuándo el dato es válido
DIO	**Bidireccional**	El dato. Sale al escribir, entra al leer teclas
STB	Siempre sale del micro	Delimita cada orden. En reposo, a nivel alto



LAS TRES SEÑALES DEL PROTOCOLO

10.2 Cómo se envía un byte

El microcontrolador no tiene ningún periférico que hable este protocolo, así que se mueven los pines a mano. Para cada bit: bajar el reloj, poner el dato, subir el reloj. **El chip captura en el flanco de subida.**



EL CRONOGRAMA DE UN BYTE

> **Los bits van empezando por el menos significativo.** Es lo contrario de como se escribe un número en papel, y es la causa de que el bucle de envío recorra el byte del bit 0 al 7 y no al revés. Si alguna vez ves los segmentos encendidos «del revés», este es el primer sitio donde mirar.

Como el protocolo se genera moviendo pines desde el programa, **la velocidad del reloj la fija el código**: es el tiempo que tardan en ejecutarse las instrucciones que hay entre subir y bajar CLK. Eso tiene una consecuencia incómoda que conviene tener presente antes de tocar la biblioteca: quitar trabajo de esos bucles **acelera** el reloj, y el TM1638 admite 1 MHz como máximo. Una simplificación que parece inocente puede romper la comunicación, y no se ve leyendo el código.

10.3 Las tres familias de orden

El chip distingue de qué tipo es una orden por sus dos bits más altos:

BITS ALTOS	FAMILIA	PARA QUÉ
`01`	Datos	Anuncia si se va a escribir o a leer, y si la dirección se autoincrementa

BITS ALTOS	FAMILIA	PARA QUÉ
`10`	Display	Enciende o apaga la pantalla, y fija el brillo
`11`	Dirección	En qué posición de la memoria del display se va a escribir

Una escritura típica son tres órdenes seguidas: «voy a escribir», «en esta dirección», y los datos. Con el autoincremento activado, una sola dirección basta para escribir los dieciséis bytes de una vez, que es como la biblioteca borra la pantalla.

10.4 La memoria del display

El chip tiene **16 bytes** de memoria, y lo que hay en ellos es lo que se ve. Están intercalados de una forma que sorprende:

DIRECCIÓN	QUÉ CONTROLA
0, 2, 4 … 14 (**pares**)	Los segmentos del dígito 1, 2, 3 … 8
1, 3, 5 … 15 (**impares**)	El LED que hay encima del dígito 1, 2, 3 … 8

De ahí salen las dos cuentas que aparecen en la biblioteca y que, sin esta tabla, parecen arbitrarias:

```
direccion del digito n  =  0xC0 + 2 * (n - 1)
direccion del LED    n  =  0xC0 + 2 * n - 1
```

> Del byte de un LED solo cuenta el **bit 0**: encendido o apagado. Los otros siete no hacen nada.

10.5 Un byte por dígito

El byte de un dígito no es el número que quieres mostrar: es **qué segmentos se encienden**, uno por bit.

LOS SIETE SEGMENTOS Y SU BIT

```
 aaaa          bit 0 = a        bit 3 = d        bit 6 = g
f    b         bit 1 = b        bit 4 = e        bit 7 = punto
f    b         bit 2 = c        bit 5 = f
 gggg
e    c       Ejemplo: el 7 es a + b + c  =  0x07
e    c                el 8 son los siete =  0x7F
 dddd  o
```

Traducir un carácter a su patrón es lo único que hace la tabla de la biblioteca. Y por eso existe una función para escribir el patrón directamente: **es la vía para dibujar símbolos que no están en la tabla**, encendiendo los segmentos que hagan falta.

10.6 Por qué faltan letras

Con siete trazos hay letras que sencillamente no se pueden dibujar de forma reconocible. La biblioteca admite **38 caracteres**:

```
espacio  -  _  0123456789  ABCEFHIJLOPSU  abcdhinortuy
```

QUÉ FALTA	CUÁLES
Mayúsculas que no existen	`D`, `G`, `K`, `M`, `N`, `Q`, `R`, `T`, `V`, `W`, `X`, `Y`, `Z`
Parejas que se ven igual	`B` sale como `8` · `O` como `0` · `I` como `1` · `S` como `5`

> **Pedir un carácter que no existe no falla en silencio, pero tampoco avisa claro.** La biblioteca borra lo que ibas a mostrar y escribe `Error` seguido de un número. Quien no

> lo sepa buscará el fallo en su lógica un buen rato. El caso típico es querer poner `Go`: no hay `G`, así que hay que elegir otra palabra.

10.7 Las teclas

Leer los pulsadores es la única operación en la que DIO cambia de sentido. La secuencia es: mandar la orden de lectura, pasar DIO a entrada, sacar **32 pulsos de reloj** leyendo el nivel en cada uno, y devolver DIO a salida.

Los 32 bits que llegan **no vienen en el orden de las teclas**. El chip los organiza según su matriz interna, y las ocho que interesan caen en posiciones salteadas:

TECLAS	POSICIONES
S1, S2, S3, S4	1, 9, 17, 25
S5, S6, S7, S8	5, 13, 21, 29

Eso explica el reparto que hace la biblioteca, que de otro modo parecería arbitrario. Y explica también algo útil: como se leen las ocho a la vez, **detectar que hay dos pulsadas es gratis**. Basta con mirar dos bits en lugar de uno.

> La lectura devuelve el **nivel**, no el flanco: mientras la tecla siga pulsada, cada llamada volverá a devolver su bit. Detectar la pulsación y filtrar los rebotes es trabajo del programa, con lo que se cuenta en el capítulo 5. El módulo no filtra nada.

10.8 El brillo

La orden de display lleva tres bits de brillo, así que hay **ocho niveles, del 0 al 7**. El 0 es el más tenue; para apagar hay otra orden distinta.

> El brillo se pierde en cuanto se manda otra orden de display, y **cada escritura de un dígito o de un LED termina mandando una**. Por eso la biblioteca recuerda el último

> valor y lo vuelve a aplicar: si no lo hiciera, ajustar el brillo parecería funcionar y el efecto duraría hasta la primera escritura.

11. Depurar

Hay dos formas de averiguar por qué un programa no hace lo que debe. Una es cambiar algo, compilar, grabar, mirar la placa y repetir. La otra es **parar el programa y preguntarle** qué está pasando. La primera es la que sale sola y la que más tiempo consume; este capítulo va de la segunda.

11.1 Por qué probar sale caro

Cada vuelta de ensayo y error cuesta lo mismo, hagas el cambio que hagas:

PASO	CUÁNTO
Editar el código	Segundos
Compilar	De cinco a treinta segundos
Grabar en la placa	Unos segundos
Reproducir la situación que falla	Lo que haga falta
Total por intento	**Rara vez menos de un minuto**

Y a cambio de ese minuto obtienes **un bit de información**: o funcionó o no. Si hay cinco causas posibles, son cinco minutos en el mejor caso y muchos más si el fallo no aparece siempre.

Un punto de ruptura, en cambio, responde a preguntas concretas en segundos: si esa línea se ejecuta, cuántas veces, con qué valores, qué devolvió la función. La diferencia no es de comodidad, es de **cuánta información obtienes por minuto invertido**.

> **El síntoma de estar probando en vez de depurar** es fácil de reconocer: llevas un rato cambiando cosas y ya no recuerdas cuáles has probado, ni por qué pensabas que cada una iba a funcionar. Cuando eso pasa, para y pon un punto de ruptura.

11.2 Qué te da el depurador

HERRAMIENTA	QUÉ RESPONDE
Punto de ruptura	¿Llega el programa hasta aquí? ¿Cuántas veces?
Inspección de variables	¿Qué vale esto ahora mismo?
Paso a paso	¿Por dónde sigue: entra en el `if` o no?
Pila de llamadas	¿Quién ha llamado a esta función?
Vista de periféricos	¿Cómo está configurado de verdad el temporizador, el pin, el reloj?
Vista de memoria	¿Qué hay en esta dirección?

La vista de periféricos es la que más se desaprovecha

En un ordenador no existe nada parecido; aquí es probablemente la herramienta más útil de todas. Permite ver **los registros del chip tal como están**, sin creerte lo que dice tu código de configuración.

PREGUNTA	DÓNDE MIRAR
¿A qué frecuencia va de verdad el reloj?	Los registros del RCC
¿Este pin está en modo salida o entrada?	Los registros del puerto
¿Por dónde va el contador del temporizador?	El registro de cuenta de TIM2

PREGUNTA	DÓNDE MIRAR
¿Está habilitada la interrupción?	Los registros del controlador de interrupciones

> Es la forma de resolver la discusión entre «el código dice que está configurado así» y «pues no se comporta así». **El registro no miente.** Si el código y el registro no coinciden, el problema está en que la configuración no llegó a aplicarse, y eso es una búsqueda muy distinta.

11.3 El método

Depurar no es mirar el código hasta que se te ocurra algo. Es un procedimiento, y sirve igual con o sin depurador:

PASO	QUÉ HACER
1	Describir el síntoma con precisión: **qué esperabas y qué pasa en su lugar**
2	Formular una hipótesis que se pueda comprobar. «Está mal» no lo es; «el callback no se ejecuta» sí
3	Diseñar la comprobación más barata que la confirme o la descarte
4	Comprobarla. Si se descarta, volver al paso 2 con lo que has aprendido
5	Solo cuando sepas la causa, arreglar. **Un cambio cada vez**

Bisección: partir el problema en dos

La técnica más rentable. En vez de preguntarte dónde está el fallo, pregúntate si el programa llega **hasta la mitad** en buen estado. Un punto de ruptura a mitad de camino descarta la mitad del código en un intento.

Aplicado a un caso típico —el display no muestra lo que debe— la cadena de preguntas es:

4. ¿Se ejecuta la línea que escribe en el display? Punto de ruptura ahí.
5. Si se ejecuta: ¿con qué valor? Inspeccionar la variable.
6. Si el valor es correcto: el problema está en la biblioteca o en el cableado.
7. Si el valor es incorrecto: retroceder a donde se calcula, y repetir.

> Fíjate en que **ninguna de esas preguntas se responde mirando el código**. Todas se responden mirando lo que ocurre. Ahí está la diferencia entre depurar y releer.

11.4 Puntos de ruptura que valen más que uno normal

TIPO	PARA QUÉ	EJEMPLO
Condicional	Parar solo cuando se cumple algo, en vez de mil veces	Parar cuando el contador valga 500, o cuando la variable sea negativa
Con contador	Parar a la enésima vez que se pasa por ahí	La vuelta 100 del bucle, para ver si el error se acumula
En una rutina de interrupción	Comprobar que salta, y con qué	Confirmar que el flanco elegido es el correcto

El condicional es el que más cambia las cosas. Un punto de ruptura normal dentro de un bucle que da diez mil vueltas es inservible: paras, sigues, paras, sigues. Con una condición, paras **exactamente** en la vuelta que te interesa.

11.5 Lo que cambia al depurar un microcontrolador

Aquí hay diferencias con depurar un programa de ordenador que conviene conocer antes de sacar conclusiones equivocadas:

QUÉ PASA	CONSECUENCIA
El tiempo no se detiene	Los temporizadores del chip siguen contando mientras el programa está parado. Al reanudar, `HAL_GetTick()` habrá saltado varios segundos
Los periféricos siguen	Un motor sigue girando, un LED sigue encendido. Parar el programa no para el mundo
Parar dentro de una interrupción bloquea otras	Las de menor prioridad no se atienden mientras estés ahí, incluida la que cuenta los milisegundos
Mirar cambia lo mirado	Un fallo que depende del tiempo puede desaparecer al depurarlo, precisamente porque lo has parado

> **Ese último punto tiene nombre: efecto observador**, y es lo que hace difíciles los fallos de tiempo. Si un problema desaparece en cuanto pones un punto de ruptura, eso no es mala suerte: **es información**. Te está diciendo que el fallo depende de cuándo ocurren las cosas, y ahí el depurador ya no es la herramienta.

11.6 Cuando el depurador no llega

Para lo que hay que ver a velocidad real hay otras vías, de menos a más aparato:

TÉCNICA	CÓMO	CUÁNDO
Un LED	Encenderlo al entrar en un sitio y apagarlo al salir	Saber si se pasa por ahí, sin detener nada
Contar en una variable	Incrementarla y mirarla después, parando	Cuántas veces ocurre algo. Delata rebotes

TÉCNICA	CÓMO	CUÁNDO
Guardar una traza	Un array con los últimos valores, y mirarlo al parar	Ver la secuencia de lo que pasó, no solo el estado final
Sacar por el puerto serie	Escribir texto por la USART	Seguimiento continuo. **Cuidado: escribir es lento** y puede cambiar el comportamiento
Un pin de traza	Poner un pin a 1 al entrar y a 0 al salir, y mirarlo con un osciloscopio	Medir cuánto tarda algo de verdad, sin afectarlo

El pin de traza es la única que no perturba nada apreciable, y la única forma de medir cuánto tarda una rutina de interrupción. Cuesta dos líneas de código.

11.7 Del síntoma a la comprobación

Qué mirar primero según lo que veas, para no empezar por el sitio equivocado:

SÍNTOMA	PRIMERA COMPROBACIÓN
No pasa nada en absoluto	Punto de ruptura en la primera línea de `main`. ¿Llega siquiera?
Una función no parece ejecutarse	Punto de ruptura dentro. Si no para, no se llama: el problema está antes
Un callback de interrupción no se ejecuta	Los registros del controlador: ¿está habilitada? ¿Se arrancó el periférico?
Un valor sale raro	Inspeccionarlo justo donde se calcula, no donde se usa
Un tiempo sale mal	La vista de periféricos: la configuración del reloj y del temporizador

SÍNTOMA	PRIMERA COMPROBACIÓN
Funciona a veces	Punto de ruptura condicional en el caso que falla, o contar ocurrencias
Funciona en Debug y no en Release	No lo busques con el depurador: falta un `volatile`

11.8 Cuando el depurador es el que engaña

Tres situaciones en las que lo que ves no es lo que hay, y que conviene reconocer para no perseguir fantasmas:

LO QUE VES	QUÉ PASA
Al parar aparece **ensamblador** en vez de tu código	La ruta del proyecto lleva tildes o eñes y el depurador no encuentra los fuentes
Una variable aparece como **no disponible**	Estás en Release: el optimizador la ha guardado en un registro o la ha eliminado
Los puntos de ruptura **paran donde no toca**	El binario grabado no corresponde al código que estás viendo. Recompila y vuelve a grabar

> El segundo es el más traicionero, porque parece un fallo del programa. **En Release el código no se corresponde línea a línea con el fuente**: el optimizador reordena, funde y elimina. Para depurar, Debug; Release es para entregar y para medir.

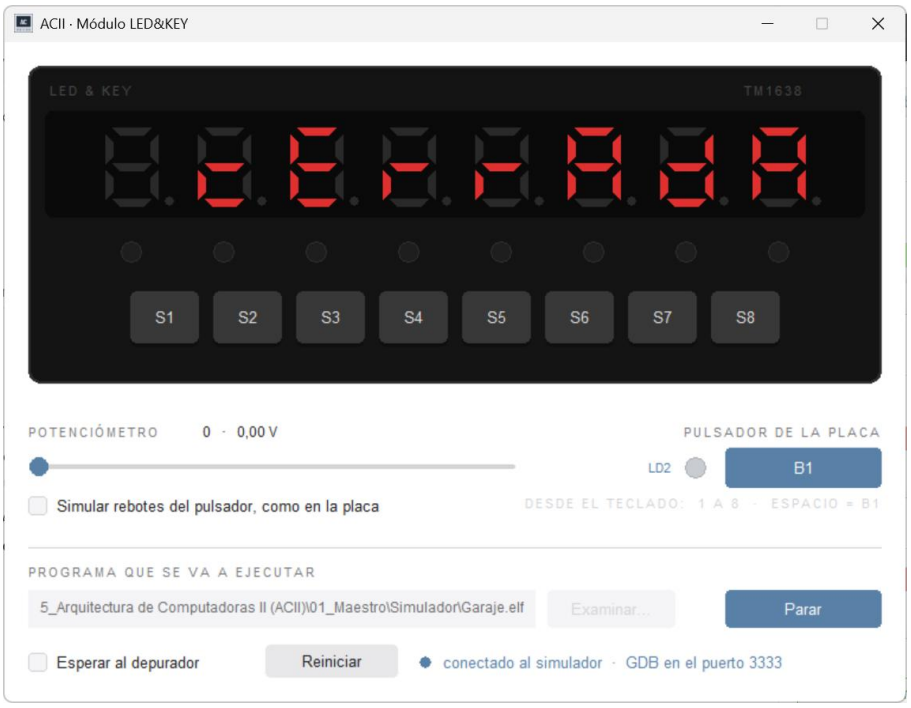
11.9 Depurar sin la placa

Depurar como se ha contado hasta aquí necesita una placa conectada. Hay una forma de no necesitarla: **un emulador** que ejecuta el mismo binario que grabarías en el

microcontrolador, con el mismo depurador y los mismos puntos de ruptura. Sirve para trabajar en casa entre sesiones de laboratorio, o cuando alguien está usando la placa.

El emulador se llama **Renode**, es libre, y ejecuta el `.elf` que produce STM32CubeIDE sin cambiarle nada: no hay una versión del programa «para simulador» y otra «para placa». Por delante hay un **panel** que dibuja el módulo LED&KEY —los ocho dígitos, los ocho LED y los ocho pulsadores—, el pulsador azul de la placa y su LED verde, para que haya algo que mirar y algo que pulsar.

El panel es lo único que hay que abrir. Él arranca el emulador, le prepara lo que necesita y se conecta solo; no hace falta consola ni escribir ninguna orden. Y deja siempre listo el servidor al que se conecta el depurador, así que no hay dos modos de arranque ni nada que recordar activar.



EL PANEL CON LA PRÁCTICA 2 EN MARCHA. JUNTO A B1 ESTÁ LD2, EL LED VERDE DE LA PLACA. LAS OCHO TECLAS Y B1 TAMBIÉN RESPONDEN AL TECLADO: 1 A 8, Y LA BARRA ESPACIADORA

De la placa está simulado lo que usan las prácticas:

QUÉ	CÓMO
El microcontrolador	`STM32F401RE` a **84 MHz**, el mismo reloj que configuran los proyectos. `HAL_Delay(1000)` tarda 1000 ms de tiempo simulado
Los puertos GPIO	Entradas y salidas con sus resistencias de pull-up, para que un pulsador sin pulsar se lea como en la placa
El módulo LED&KEY	Los ocho dígitos, los ocho LED y los ocho pulsadores, por los tres pines de siempre
El pulsador azul	En `PC13`, con la misma lógica invertida que en la placa
El LED verde, `LD2`	En `PA5`. Es el que hace parpadear la Práctica 1
El potenciómetro	Como entrada del conversor A/D, de 0 a 4095
Los temporizadores	`TIM2` y `SysTick`, con sus interrupciones
Las interrupciones de pin	`EXTI`: `B1` y las teclas avisan al programa en vez de tener que preguntarles. Es lo que se usa en la Práctica 4

El panel, control por control

La ventana tiene dos mitades. Arriba, la placa: es lo que tu programa escribe y lo que tú pulsas. Abajo, el mando del simulador: qué programa se ejecuta y cómo.

CONTROL	QUÉ ES
Los ocho dígitos y los ocho LED	El módulo LED&KEY, tal como lo escribe tu programa. Si tu programa no escribe, se quedan apagados
S1 a S8	Los ocho pulsadores del módulo, de izquierda a derecha. Se pulsan con el ratón o con las teclas **1** a **8**

CONTROL	QUÉ ES
B1	**El pulsador azul de la placa**, en PC13. No es del módulo: va directo al microcontrolador. Con el ratón, o con la **barra espaciadora**
LD2	**El LED verde de la placa**, en PA5. Tampoco es del módulo, y por eso va al lado de B1 y no con los rojos. Se enciende cuando tu programa pone ese pin a 1
Potenciómetro	La entrada del conversor A/D. La barra va de **0 a 4095**, y al lado se ve también en voltios
Simular rebotes del pulsador	Manda una ráfaga de pulsos al pulsar, como hace un pulsador de verdad, en vez de una transición limpia
Programa que se va a ejecutar	El `.elf` que se cargará. Se escribe a mano o se elige con Examinar…
Arrancar / Parar	Levanta el emulador con ese programa, o lo mata
Reiniciar	Las dos cosas seguidas: para el que hay y levanta otro. Es lo que hay que darle después de recompilar
Esperar al depurador	Arranca con el procesador congelado antes de la primera instrucción, para poder depurar desde `main`

> **La casilla de rebotes merece la pena.** En la placa, un pulsador no se cierra de golpe: rebota unos milisegundos. Un programa sin antirrebote parecerá correcto con una pulsación limpia y contará varias en el laboratorio. Con la casilla marcada, el simulador te enseña ese problema en casa en vez de escondértelo.

El teclado. Mientras el simulador está en marcha, las teclas **1** a **8** son S1 a S8 y la **barra espaciadora** es B1; también valen las del teclado numérico. Se mantienen pulsadas

mientras mantengas la tecla, así que se puede probar una pulsación larga. Si escribes en el campo del programa, los atajos se desactivan mientras el foco esté ahí —si no, no podrías teclear una ruta, que lleva dígitos y espacios—; para recuperarlos, `Intro` o un clic en la placa.

La línea de estado, abajo a la derecha, dice en qué punto está:

LO QUE PONE	QUÉ SIGNIFICA
`sin conexión`	No hay emulador; el panel está esperando a que le des a `Arrancar`
`arrancando el emulador…`	Renode está levantando la máquina. La primera vez tarda: compila los modelos
`conectado al simulador`	Todo en marcha. Al lado dice el puerto del servidor GDB
`esperando al depurador…`	Con la casilla marcada: el procesador está congelado y no ejecutará nada hasta que se conecte CubeIDE
`el simulador ha cerrado la conexión`	El emulador se ha muerto o lo han matado. El módulo se apaga para no enseñar un fotograma viejo

Ejecutar un programa

Sin depurador, solo para ver si el programa hace lo que debe:

8. **Compila** en CubeIDE como siempre, `Project` → `Build Project`. Mira en la consola que acaba con `0 errors`.
9. **Localiza el `.elf`.** Está en la carpeta `Debug` del proyecto y se llama como el proyecto: si tu proyecto es `P3_Reloj`, el fichero es `...\P3_Reloj\Debug\P3_Reloj.elf`. En el Project Explorer aparece bajo `Binaries`.
10. **Abre el panel** —en Windows `Panel\PanelACII.exe`; en Linux y en Mac, el lanzador que dejó el instalador— y elige ese `.elf` con `Examinar…`.

11. **Dale a `Arrancar`.** El panel copia lo que hace falta, lanza el emulador y se conecta solo. Cuando la línea de estado ponga `conectado al simulador`, el programa ya está corriendo.

A partir de ahí el panel es la placa: los dígitos y los LED enseñan lo que escribe el programa, y `S1`..`S8`, `B1` y el potenciómetro le llegan como si los pulsaras. Para terminar, `Parar`.

> **La primera vez tarda más.** El emulador compila al vuelo los cuatro modelos de los periféricos, y eso son unos segundos. Las siguientes va directo. Si pasa un minuto sin arrancar, algo va mal: mira el apartado de problemas.
>
> **Al recompilar hay que volver a arrancar.** El emulador carga el binario una vez, al empezar. Después de recompilar en CubeIDE, `Reiniciar` —que para el emulador y levanta otro con el binario nuevo—, o seguirás viendo el programa anterior, que es la forma más tonta de perder una tarde.

Con el simulador en marcha, el campo del programa y `Examinar…` quedan cerrados: el que se está ejecutando es el que se cargó al arrancar, y para cambiarlo hay que parar.

Depurar desde STM32CubeIDE

Se puede poner un punto de ruptura en CubeIDE y que se cumpla dentro del emulador, con su código fuente, sus variables y su pila de llamadas. Es el depurador de siempre; lo único que cambia es a qué se conecta.

El emulador **siempre** abre un servidor GDB en el puerto **3333**, estés depurando o no, así que puedes conectarte en cualquier momento. La configuración del lado de CubeIDE se hace **una vez por proyecto**, y te la deja hecha el propio panel.

Paso 1. Arranca el simulador. Elige el `.elf` y dale a `Arrancar`, igual que antes. Déjalo corriendo: el depurador se va a conectar a él.

Si lo que quieres es **parar dentro de `main`, antes de que se ejecute nada**, marca la casilla **`Esperar al depurador`** antes de darle a `Arrancar`. El emulador queda entonces con el

procesador congelado en la primera instrucción y no arranca hasta que el depurador se lo diga. El display se queda apagado y el estado pone `esperando al depurador…`: es lo que tiene que pasar.

Paso 2. La configuración de depuración. Al arrancar, el panel mira el proyecto al que pertenece el `.elf`, y si todavía no tiene configuración de depuración para el emulador se la deja hecha. Te lo dice con un aviso. Aparece en CubeIDE en `Run` → `Debug Configurations…`, bajo **`GDB Hardware Debugging`**, con el nombre **`Simulador ACII - <proyecto>`**. Lleva el nombre del proyecto porque ahí se listan las de todos los que tengas abiertos, y si se llamaran igual no se distinguirían.

> **Si el proyecto ya estaba abierto en CubeIDE, refréscalo con `F5`** —selecciona el proyecto en el Project Explorer y pulsa `F5`— para que vea el fichero nuevo. Si aun así no aparece, cierra y vuelve a abrir CubeIDE.

Viene marcada como **favorita**, así que también la tienes en la flecha de al lado del botón de depurar, arriba del todo y por delante del historial.

> **El botón de depurar a secas, sin desplegar, no sirve la primera vez.** Relanza siempre lo último que se lanzó, que será la configuración de la placa. Hay que desplegar la flecha de al lado y elegir la del simulador; a partir de ahí el botón ya repite esa.
>
> **No es la misma que la de la placa.** La de siempre, `STM32 C/C++ Application`, habla con el ST-LINK y sigue donde estaba, para cuando depures en la placa. Son dos configuraciones distintas del mismo proyecto y no se estorban.

Paso 3. Pon los puntos de ruptura y dale a `Debug`. Al conectarse, el depurador te lleva a la perspectiva de depuración. Lo que ves depende de cómo hayas arrancado:

CÓMO ARRANCASTE	DÓNDE SE DETIENE
Sin `Esperar al depurador`	Donde estuviera el programa, que casi siempre es dentro de la biblioteca del módulo: `Suspended : Signal : SIGINT`. La inicialización ya pasó
Con `Esperar al depurador`	En la primera línea de `main`, con **nada** ejecutado. Desde ahí se puede recorrer `SystemClock_Config` y los `MX_..._Init` paso a paso

A partir de ahí es el depurador de siempre: `F5` para entrar, `F6` para pasar por encima, `F8` para continuar, la vista de variables, la de registros y la pila de llamadas.



EL PROGRAMA DETENIDO DENTRO DE LA BIBLIOTECA DEL MÓDULO, CON LA PILA DE LLAMADAS QUE HA LLEVADO HASTA AHÍ

Para volver a depurar desde el principio, el botón `Reiniciar` del panel. Parar el programa en CubeIDE, o pedirle que lo relance, no sirve: al procesador solo se le puede coger antes de la primera instrucción con un emulador recién arrancado, y eso es lo que hace ese botón. Después, `Debug` otra vez.

Y el ciclo completo mientras trabajas —cambias el código, quieres volver a probarlo— es este:

12. En CubeIDE, `Terminate` para soltar el depurador.
13. `Project` → `Build Project` para recompilar.
14. En el panel, `Reiniciar`.

15. `Debug` otra vez, eligiendo `Simulador ACII - <proyecto>` en la flecha.

Lo que lleva dentro la configuración

No hay que tocarlo: el panel lo escribe. Se cuenta porque es lo que habría que poner a mano en un proyecto que no haya pasado por el panel, y porque aclara por qué el depurador y el emulador se entienden.

En la pestaña **`Main`**, el proyecto y su binario: `Debug\<nombre>.elf`.

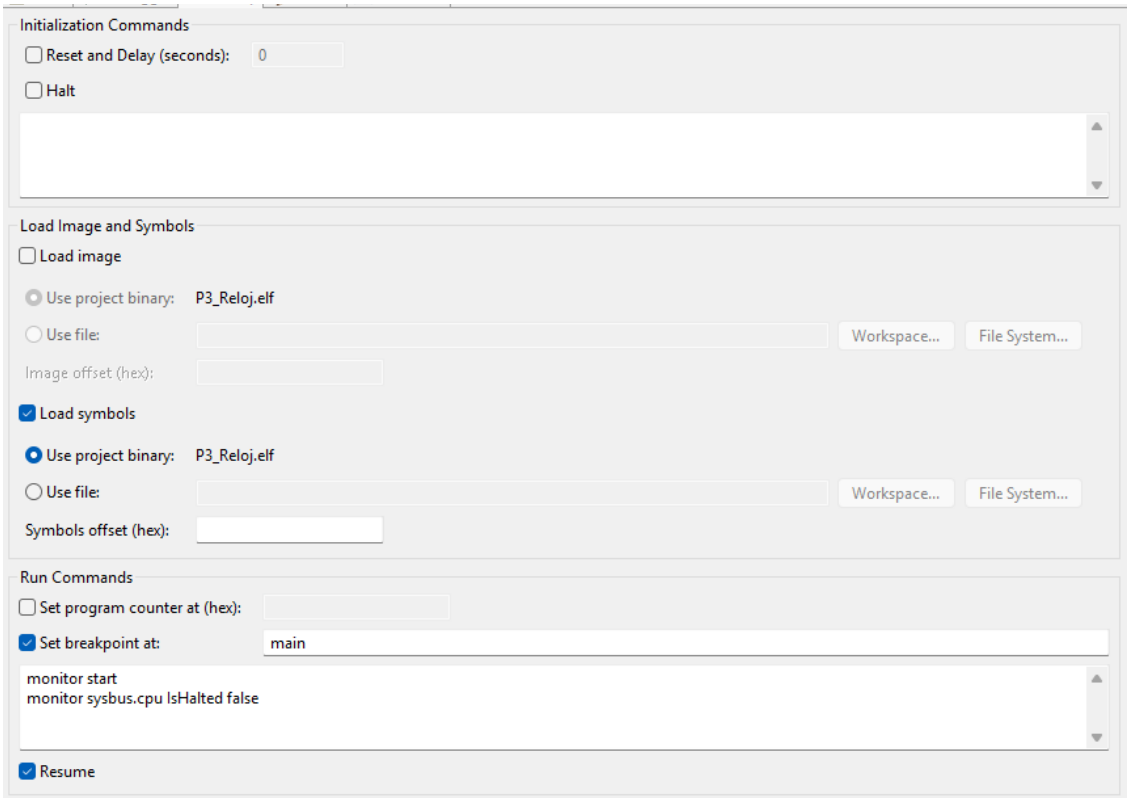
En la pestaña **`Debugger`**, a dónde conectarse:



LA PESTAÑA DEBUGGER: EL DEPURADOR HABLA CON EL EMULADOR POR UN PUERTO, EN LUGAR DE CON LA SONDA

CAMPO	QUÉ PONER
`GDB Command`	`arm-none-eabi-gdb`, el mismo que usa el entorno para la placa
`Use remote target`	**Marcado.** Es lo que habilita el resto
`Debug server`	`Generic TCP/IP`
`Protocol`	`remote`
`Connection`	`localhost:3333`

Y en la pestaña **`Startup`**, que es donde se tuerce si se deja como viene sola:



LA PESTAÑA STARTUP TAL COMO TIENE QUE QUEDAR: DE LAS CINCO CASILLAS, SOLO «LOAD SYMBOLS» Y «RESUME» MARCADAS; ABAJO, EL PUNTO DE RUPTURA EN MAIN Y LAS DOS ÓRDENES QUE ARRANCAN EL EMULADOR

CASILLA	CÓMO	POR QUÉ
`Reset and Delay`	**Desmarcada**	Es una orden de la sonda física. El emulador no la tiene
`Halt`	**Desmarcada**	Lo mismo. Además el procesador ya llega retenido
Órdenes de inicialización	Vacío	No hay nada que inicializar
`Load image`	**Desmarcada**	El programa ya lo ha cargado el emulador
`Load symbols`	**Marcada**	Es lo que da código fuente, variables y pila de llamadas
`Set breakpoint at`	`main`	Con la casilla del panel marcada, este sí se cumple

CASILLA	CÓMO	POR QUÉ
`Run Commands`	**Las dos órdenes de abajo**	Arrancan el tiempo del emulador y sueltan el procesador
`Resume`	Marcada	Que eche a andar al conectarse

En la caja **`Run Commands`**, la de abajo del todo, van estas dos líneas:

```
monitor start
monitor sysbus.cpu IsHalted false
```

La primera arranca el reloj del emulador y la segunda suelta el procesador. Van ahí y no en las órdenes de inicialización **porque esa caja se ejecuta después de poner los puntos de ruptura**: si se arrancara antes, el programa pasaría por `main` sin que hubiera nada que lo detuviera.

> **`continue` no arranca el emulador por sí solo.** El tiempo de la emulación es cosa de Renode, no del programa, y hay que pedírselo con `monitor start`. Sin esas dos líneas, con la casilla marcada, el depurador se conecta y se queda esperando para siempre.
>
> **Si al lanzar sale «Reset command not defined for debug server 'Generic TCP/IP'», es que `Reset and Delay` sigue marcada.** Desmárcala, y desmarca también `Halt` o darás con el mismo mensaje cambiando la primera palabra. Son órdenes que solo entiende una sonda de verdad.

Lo que el simulador no puede enseñar

El tiempo no es el de la placa. El emulador va del orden de dos veces más lento, y ese factor depende del ordenador. Sirve para ver **qué hace** el programa; para saber **cuánto tarda**, la placa.

ESTO SÍ	ESTO NO
Si el programa llega donde debe y con qué valores	Cuánto tarda una rutina
Si la máquina de estados hace las transiciones correctas	Calibrar un bucle de espera
Qué escribe en el display y qué LED enciende	El brillo real del display
Si una tecla se detecta y si el antirrebote funciona	Un contacto sucio o un cable flojo

> **El simulador no sustituye al laboratorio, lo complementa.** Un programa que funciona en el simulador puede fallar en la placa, y eso no es un defecto del simulador: es que el hardware tiene comportamientos que ningún modelo recoge. La entrega se comprueba en la placa.

Si algo no va

LO QUE VES	QUÉ PASA Y QUÉ HACER
«No encuentro Renode»	En Windows, falta la carpeta Renode junto a la del panel: vuelve a instalar, o a descomprimir el ZIP entero. En Linux y Mac, no está instalado
«Otra versión de Renode»	Solo en Linux y Mac. Tienes una distinta de la 1.16.1. Puedes seguir, pero si no arranca, es lo primero que hay que cambiar
«El emulador se ha cerrado nada más arrancar»	El panel enseña lo último que dijo el emulador. Casi siempre es que la ruta del proyecto lleva espacios o tildes

LO QUE VES	QUÉ PASA Y QUÉ HACER
«Ya hay un simulador corriendo»	Quedó uno de antes, de otra ventana del panel. Ciérrala y vuelve a darle a `Arrancar`
«El emulador no ha abierto el puerto a tiempo»	Ha tardado más de dos minutos y medio. Mira el registro: casi siempre es una pieza que falta en la carpeta `plataforma`
El panel arranca pero el display no cambia	El programa no escribe en el módulo. Es un fallo del programa, no del simulador: pon un punto de ruptura donde escribe
El display se queda apagado y no reacciona	Si marcaste `Esperar al depurador`, es lo esperado: no ejecutará nada hasta que conectes CubeIDE
`Simulador ACII - ...` no aparece en CubeIDE	Refresca el proyecto con F5. Si sigue sin salir, cierra y abre CubeIDE
El depurador se conecta y se queda colgado	Faltan las dos líneas de `Run Commands`. Ver el apartado anterior
Al depurar, un punto de ruptura en la inicialización no se cumple	Arrancaste sin `Esperar al depurador`: cuando te conectas ya había pasado. `Parar`, marcar la casilla y `Arrancar`
Las teclas 1..8 no hacen nada	El foco está en el campo del programa. `Intro`, o un clic en la placa

Dónde mirar cuando nada de lo anterior encaja. El emulador deja todo lo que va diciendo en un registro, y el panel deja ahí también el guion con el que lo ha lanzado. La carpeta de trabajo es `C:\ACII_sim` en Windows y `~/.acii_sim` en Linux y en Mac:

FICHERO	QUÉ ES
`<carpeta de trabajo>/renode.log`	Todo lo que ha dicho el emulador, incluido por qué se ha cerrado
`<carpeta de trabajo>/simula.resc`	Las órdenes con las que se ha lanzado: qué placa, qué modelos y qué `.elf`
`<carpeta de trabajo>/*.elf`	La copia del programa que se está ejecutando. Si la fecha no es la de tu última compilación, no le diste a `Reiniciar`

11.10 Lo que conviene llevarse

- Antes de tocar nada, **describe el síntoma**: qué esperabas y qué pasa.
- Convierte la sospecha en una **pregunta que se pueda responder**, y respóndela con un punto de ruptura, no releyendo.
- **Parte el problema en dos** en vez de buscar la línea culpable.
- Ante una duda sobre configuración, mira **los registros**, no el código que los escribe.
- Si el fallo desaparece al depurarlo, eso **es un dato**: depende del tiempo.
- Y un cambio cada vez. Dos a la vez, y no sabrás cuál fue.
- Sin placa a mano, **el emulador te da el mismo depurador**; lo que no te da son los tiempos.

> Todo esto tiene un coste de entrada de unos minutos —aprender dónde están los botones— y se amortiza el primer día. **Un fallo que por ensayo y error cuesta media hora, acorralado cuesta cinco minutos**, y además dejas de arreglar cosas sin saber por qué funcionaban mal.

12. Cómo se construye una biblioteca

El capítulo 10 explica lo que hace el chip. Este explica lo que hay entre ese chip y la línea que escribes tú:

```
tm1638_DisplayTxt(&TM, "ACII");
```

Entre esa llamada y las patillas del microcontrolador hay unos centenares de líneas de C. Merece la pena mirarlas, porque **escribir una biblioteca es el trabajo más habitual de un programador de sistemas embebidos**: casi nunca se parte de una función que ya hace lo que quieres, y casi siempre se parte de una hoja de características.

12.1 El problema que resuelve

La hoja de características del TM1638 dice cosas como que el bit menos significativo se transmite primero, o que el chip captura en el flanco de subida del reloj. Eso no es código: es una descripción de qué tiene que ocurrir en tres cables y en qué orden.

Sin biblioteca, escribir `ACII` en el display significa, cada vez:

16. Bajar STB para abrir la orden.
17. Enviar el byte del modo de escritura, bit a bit, moviendo CLK entre bit y bit.
18. Enviar la dirección donde escribir, sabiendo que los dígitos están en las direcciones pares y los LEDs en las impares.
19. Traducir cada letra a los siete segmentos que hay que encender.
20. Enviar cada byte de segmentos.
21. Subir STB, y mandar además la orden que enciende el display.

> **Ese es el trabajo que una biblioteca hace una vez y esconde para siempre.** No lo hace más rápido ni más corto: lo hace **una sola vez**, en un sitio donde se puede corregir, en lugar de repetido y ligeramente distinto en cada programa.

12.2 La idea central: capas

La biblioteca no es una lista de funciones sueltas. Está partida en **niveles**, y cada uno solo habla con el de abajo. El propio fichero los rotula:

NIVEL	QUÉ HACE	EJEMPLO
0 · Mover un pin	Envoltorios de una línea sobre la HAL. No aportan lógica	`tm1638_CLKhigh`
1 · Enmarcar y enviar	Abrir y cerrar una orden, y mandar un byte bit a bit	`tm1638_SendData`
2 · Display y teclado	Lo que tú llamas: escribir texto, encender un LED, leer las teclas	`tm1638_DisplayTxt`

El nivel 0 parece inútil —¿para qué una función de una línea?— y es donde está la mitad del valor. Compara las dos formas de escribir la misma secuencia:

```
/* Sin nivel 0 */
HAL_GPIO_WritePin(tm->clk_port, tm->clk_pin, GPIO_PIN_RESET);
HAL_GPIO_WritePin(tm->dio_port, tm->dio_pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(tm->clk_port, tm->clk_pin, GPIO_PIN_SET);

/* Con nivel 0 */
tm1638_CLKlow(tm);
tm1638_SDOhigh(tm);
tm1638_CLKhigh(tm);
```

> La segunda versión **se lee como el cronograma de la hoja de características**. Y eso importa más de lo que parece: cuando algo no funciona, quieres comparar tu código con el dibujo del fabricante línea a línea, no traducir mentalmente cada `HAL_GPIO_WritePin`.

12.3 Qué se enseña y qué se esconde

La cabecera `TM1638.h` es **el contrato**: lo que declara es lo que puedes usar. Todo lo demás está marcado `static` en el `.c`, que en C significa que eso no existe fuera de ese fichero.

	CUÁNTAS	DÓNDE
Funciones **públicas**, las que puedes llamar	**11**	Declaradas en `TM1638.h`
Funciones **internas**, el andamiaje	**12**	`static` dentro de `TM1638.c`

La regla para decidir de qué lado va cada función se enuncia en una línea:

> **Público es lo que resuelve un problema de quien llama. Interno es todo lo que existe solo porque el chip es como es.** A quien escribe una práctica le importa «escribe este texto»; no le importa que el byte viaje empezando por el bit menos significativo.

La ventaja práctica llega el día que hay que cambiar algo. Si el protocolo cambiara entero, las 12 funciones internas podrían reescribirse de arriba abajo y **ningún programa de las prácticas se enteraría**, porque ninguno las nombra.

12.4 Por qué una estructura y no unas constantes

Los tres pines del módulo podrían fijarse con `#define` y ahorrarse el primer argumento de todas las funciones. En lugar de eso hay una estructura:

```
typedef struct
{
    GPIO_TypeDef *clk_port;   /* puerto del reloj  */
    GPIO_TypeDef *dio_port;   /* puerto del dato   */
    GPIO_TypeDef *stb_port;   /* puerto del strobe */
    uint16_t clk_pin;
    uint16_t dio_pin;
    uint16_t stb_pin;
} TM1638;
```

Y todas las funciones reciben un puntero a ella. Cuesta un argumento en cada llamada, y a cambio da tres cosas:

CON #DEFINE	CON ESTRUCTURA
Los pines se deciden al compilar	Se deciden al ejecutar: puedes leerlos de una configuración
Un solo módulo por programa	**Varios módulos a la vez**, cada uno con su estructura
Cambiar de pines obliga a recompilar la biblioteca	La biblioteca no se toca: cambian tres líneas de tu `main.c`

> **Es la diferencia entre una biblioteca y un trozo de código copiado.** Un trozo de código sirve para tu proyecto. Una biblioteca sirve también para el siguiente, y eso se consigue no dejándole decidir cosas que no le corresponden.

12.5 Cuando la biblioteca tiene que recordar

El TM1638 exige que, después de escribir en su memoria, se le mande la orden que enciende el display. Y esa orden lleva **el brillo dentro del mismo byte**. Es decir: cada vez que escribes algo hay que volver a decirle el brillo.

Si ese valor no se guardara en ningún sitio habría dos salidas, las dos malas: poner siempre el máximo, y entonces ajustar el brillo no serviría de nada, o exigirlo como argumento en todas las funciones. La solución es que la biblioteca lo recuerde:

```
static uint8_t s_brightness = 7;   /* arranca al maximo para que se vea
*/

static void tm1638_ApplyDisplayControl(TM1638 *tm)
{
    tm1638_SendData(tm, DISPLAY_ON | s_brightness);
}
```

> **Guardar estado tiene un precio, y conviene saberlo.** Esa variable es única para todo el programa, así que dos módulos conectados a la vez compartirían brillo. Es una limitación aceptada a cambio de una API más simple; el sitio correcto para arreglarla, si hiciera falta, sería mover el brillo dentro de la estructura `TM1638`.

12.6 La parte que no se puede tocar

Hay un trozo de la biblioteca donde **la corrección no depende de la lógica sino del tiempo**. El reloj del protocolo se genera moviendo un pin a mano, así que la anchura del pulso es, literalmente, lo que tardan en ejecutarse las instrucciones que hay entre subir y bajar `CLK`.

Mira este bucle y pregúntate qué se podría simplificar:

EL ENVÍO DE UN BYTE, BIT A BIT

```
for (uint8_t j = 0; j < 8; j++)
{
    ByteData[j] = (Data & (0x01 << j)) && 1;
}

for (int8_t j = 0; j < 8; j++)
{
    tm1638_CLKlow(tm);
    if (ByteData[j] == GPIO_PIN_SET) { tm1638_SDOhigh(tm); }
    else                             { tm1638_SDOlow(tm);  }
    tm1638_CLKhigh(tm);
}
```

Sobra un array entero: se podría calcular cada bit dentro del segundo bucle y ahorrarse el primero. Sería más corto, más elegante, y **podría romper la comunicación**, porque quitar trabajo de ese bucle acelera el reloj y el TM1638 admite **1 MHz como máximo**.

> **Es el caso extremo de algo que aparece en toda la asignatura: el código no dice todo lo que hace.** Aquí, además, el fallo no se ve leyendo ni depurando, porque parar el programa destruye precisamente lo que habría que medir. Hace falta un osciloscopio. Por eso esa parte lleva un aviso escrito encima y la biblioteca se entrega cerrada.

12.7 Qué hacer con lo imposible

¿Qué debe pasar si alguien pide escribir en el dígito 9, cuando hay 8? Las opciones habituales son devolver un código de error, no hacer nada, o avisar. Aquí las funciones públicas devuelven `void`, así que la primera queda descartada por diseño.

La biblioteca **avisa en el propio display**, escribiendo un texto de error. Es discutible, y tiene una razón concreta: en una placa sin consola, el display es el único sitio donde el programa puede decir algo. Un fallo silencioso obliga a depurar; un fallo visible se lee desde el otro lado del laboratorio.

> **Toda biblioteca decide qué hacer con lo imposible, aunque no lo decida a propósito.** Si no lo piensas, la respuesta acaba siendo comportamiento indefinido, que es la peor de todas.

12.8 La tabla de caracteres, y sus límites

Traducir una letra a segmentos se hace con una tabla de **38 caracteres**. Podría ser un array indexado por el código del carácter, que sería más rápido; es una cadena de casos, que resulta más lenta y **mucho más fácil de leer y de ampliar**. Con una tabla que se recorre una vez por dígito, esa diferencia no se nota.

Con ocho dígitos de siete segmentos y 38 caracteres, media parte del alfabeto no existe. La biblioteca no puede resolver eso: es una limitación del hardware, y lo que le toca es **ser honesta sobre ella** en vez de dibujar una `K` que se parezca vagamente a una `K`. El Anexo A tiene la lista completa de lo que se puede escribir.

> **Ese es el último criterio, y el más importante: una biblioteca no debe disimular las limitaciones de lo que hay debajo.** Si el chip no sabe pintar una letra, la respuesta correcta no es una aproximación silenciosa, es decirlo.

12.9 Si tuvieras que escribir una

El orden en que se construye una biblioteca así, y que sirve para casi cualquier periférico:

22. Leer la hoja de características hasta poder dibujar el cronograma a mano.
23. Escribir el nivel 0: mover cada pin, una función por señal. Comprobar con un LED o un osciloscopio que se mueven.
24. Escribir el envío de un byte y probar **una sola orden**, de las más simples: encender el display entero, por ejemplo.
25. Solo entonces, construir encima lo que quiere quien la va a usar.
26. Declarar en la cabecera únicamente eso último, y marcar `static` todo lo demás.

> El error típico es empezar por el paso 4, escribir la función bonita que muestra texto, y descubrir que no funciona sin saber si el fallo está en la tabla de caracteres, en el orden de los bits o en un pin mal declarado. **Construyendo de abajo arriba, cada capa se comprueba cuando solo puede fallar ella.**

PARTE II

Las prácticas

Práctica 1 · Primer proyecto: el .ioc, CubeMX y el reloj

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 1 de 13
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	Ninguno. Es la primera sesión
Material	Placa NUCLEO-F401RE · cable USB · STM32CubeMX · STM32CubeIDE · un cronómetro
Entregables	Proyecto CubeIDE, tabla de medidas y memoria (apartado 8)
Plazo	Una semana desde el final de la sesión, por Moovi

2. Qué se construye

Un programa muy corto: el LED verde de la placa parpadea, y el botón azul cambia **cómo se mide la espera** entre parpadeo y parpadeo.

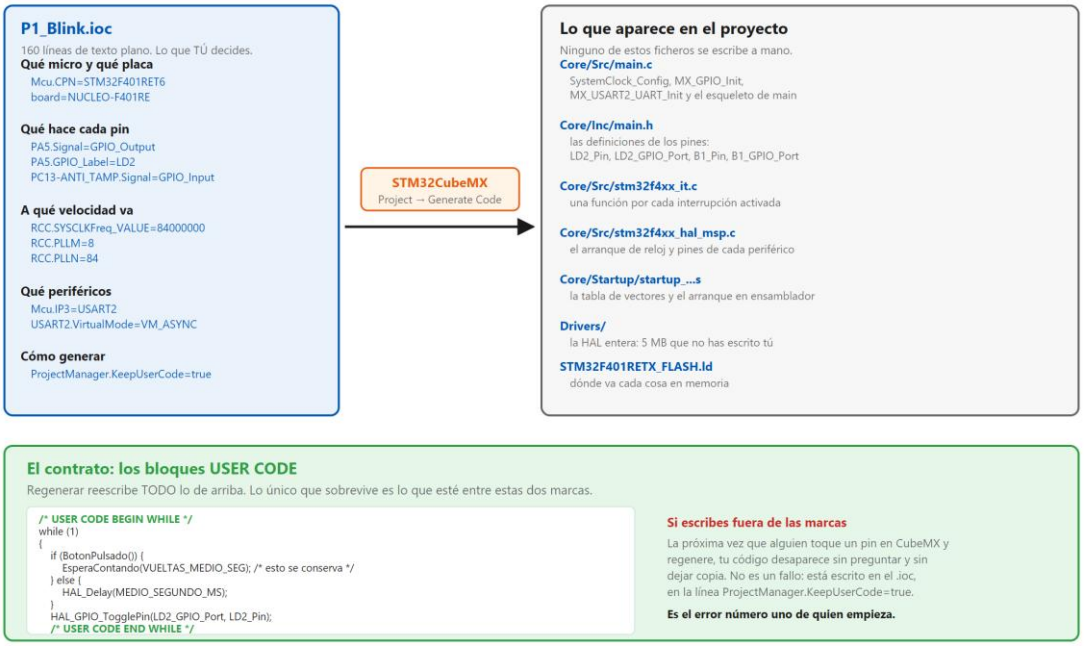
BOTÓN	CÓMO SE ESPERA	QUÉ CUENTA
Suelto	`HAL_Delay(500)`	milisegundos
Pulsado	Un bucle vacío que da muchas vueltas	vueltas de bucle

Bien ajustado, los dos parpadeos van igual. Después se cambia **una sola línea** de la configuración —la frecuencia a la que va el microcontrolador— y se vuelve a mirar: uno de los dos parpadeos sigue igual y el otro ha cambiado.

> Suena a poca cosa para dos horas, y esa es la idea. El programa es corto **a propósito**, para que quepa tiempo de mirar de dónde ha salido: quién escribió las 250 líneas de `main.c` que no has escrito tú, dónde está guardado que PA5 es una salida, y por qué el mismo código se comporta distinto según un número que no aparece en ninguna línea de C.

3. De dónde sale el código

Al crear un proyecto para la NUCLEO-F401RE aparecen de golpe unos 5 MB de ficheros. Conviene saber quién los ha puesto ahí antes de tocar nada.



EL .IOC ES LA FUENTE; EL RESTO LO GENERA CUBEMX A PARTIR DE ÉL.

3.1 El .ioc es un fichero de texto

`P1_Blink.ioc` no es un formato binario ni una base de datos: son **159 líneas de texto plano**, con la forma `clave=valor`. Se puede abrir con el Bloc de notas, y conviene hacerlo al menos una vez.

OCHO LÍNEAS DEL .IOC DE ESTA PRÁCTICA

```
Mcu.CPN=STM32F401RET6
PA5.Signal=GPIO_Output
PA5.GPIO_Label=LD2
PC13-ANTI_TAMP.Signal=GPIO_Input
PC13-ANTI_TAMP.GPIO_Label=B1
RCC.SYSCLKFreq_VALUE=84000000
ProjectManager.KeepUserCode=true
board=NUCLEO-F401RE
```

Ahí está **todo lo que has decidido**: qué micro, qué hace cada pin, a qué velocidad va y cómo se genera el proyecto. La ventana gráfica de CubeMX es una forma cómoda de editar este fichero, nada más.

3.2 Qué genera CubeMX y qué no

LO GENERA CUBEMX	LO ESCRIBES TÚ
`SystemClock_Config`, `MX_GPIO_Init`, `MX_USART2_UART_Init`	La lógica del programa, dentro del `while(1)`
Las definiciones `LD2_Pin`, `B1_GPIO_Port`… en `main.h`	Tus funciones y tus constantes
`stm32f4xx_it.c`: una función por interrupción activada	El contenido de esas funciones
`Drivers/`: la biblioteca HAL entera	Nada. No se toca
El fichero de enlazado `.ld` y el arranque en ensamblador	Nada. No se toca

3.3 El contrato: los bloques USER CODE

Repartidos por los ficheros generados hay parejas de comentarios como estas:

LA MARCA

```
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */
```

Cada vez que se pulsa **Project → Generate Code**, CubeMX reescribe los ficheros generados de arriba abajo. Lo único que respeta es lo que esté **entre** un `BEGIN` y su `END`.

> **Escribir fuera de las marcas es el error número uno de quien empieza.** El código funciona, se compila, se graba en la placa; y desaparece sin previo aviso el día que alguien cambia un pin y regenera. No hay copia de seguridad ni pregunta de confirmación: está escrito en el propio `.ioc`, en la línea `ProjectManager.KeepUserCode=true`.

4. El reloj: el dato que no está en el código

El STM32F401 puede funcionar a distintas frecuencias. La que use en cada momento se decide en el `.ioc`, en la pestaña **Clock Configuration**, y de ahí sale el `SystemClock_Config` que genera CubeMX.

CONFIGURACIÓN	FRECUENCIA DEL NÚCLEO	CÓMO SE OBTIENE
Oscilador interno directo	16 MHz	El HSI tal cual, sin multiplicar
Con el multiplicador PLL	**84 MHz**	16 / 8 = 2 MHz, × 84 = 168, / 2 = 84

84 MHz es el máximo del F401 y es la configuración que usan todas las prácticas de la asignatura. Pero fíjate en que ese número **no aparece en ninguna línea del programa que escribes tú**. Está en el `.ioc`, y de rebote en una función generada que nadie suele abrir.

> De ahí sale la pregunta de la práctica: **¿qué partes de un programa cambian si cambia el reloj, y cuáles no?** Los pasos 5 y 6 la responden midiendo, no razonando.

5. Desarrollo paso a paso

Paso 1. Crear el proyecto

Desde la versión 2.0, **CubeIDE ya no trae CubeMX dentro**: un proyecto creado desde CubeIDE sale **sin `.ioc`**, y sin él esta práctica no se puede hacer. El proyecto se crea en **STM32CubeMX** y después se abre en CubeIDE:

27. En CubeMX: **File** → **New Project**, pestaña **Board Selector**, busca `NUCLEO-F401RE` y selecciónala. Al preguntar si se inicializan los periféricos en su modo por defecto, responde **que sí**: interesa ver qué añade el generador por su cuenta.
28. En la pestaña **Project Manager**: pon el nombre y la carpeta del proyecto, en **Toolchain / IDE** elige **STM32CubeIDE** y deja marcado **Generate Under Root**.
29. Pulsa **Generate Code**.
30. En CubeIDE: **File** → **Import** → **General** → **Existing Projects into Workspace**, y elige esa carpeta.

> **CubeMX propone por defecto otro entorno, EWARM, que es el de IAR.** Si no lo cambias, genera un proyecto que CubeIDE no sabe abrir, y la importación no encuentra nada. En el `.ioc` se ve en la línea `ProjectManager.TargetToolchain`, que tiene que decir `STM32CubeIDE`.
>
> **Guarda el proyecto en una carpeta sin letras acentuadas.** Si la ruta lleva alguna —una carpeta llamada «Práctica 1», o un usuario que se llame «José»— el programa compilará y se grabará en la placa sin problema, pero **al depurar CubeIDE no encontrará el `main.c`** y solo te ofrecerá ver ensamblador. El compilador guarda la ruta dentro del ejecutable en UTF-8 y el depurador la lee con la codificación de Windows: para cualquier letra acentuada las dos no coinciden. Los espacios y los paréntesis no molestan; solo las tildes y la eñe.

Paso 2. Abrir el .ioc con un editor de texto

Antes de tocar la ventana gráfica, abre el `.ioc` como texto plano y búscale estas cuatro cosas:

31. La línea que dice qué micro es.
32. Qué pines ha configurado CubeMX **sin que se lo pidieras**, solo por haber elegido la placa. Hay al menos cinco.
33. A qué frecuencia ha quedado el sistema.
34. La línea que decide si tu código se conserva al regenerar.
35. La línea que dice **para qué entorno** se genera el proyecto.

Anota las respuestas: la primera cuestión de la memoria va sobre esto.

Paso 3. El LED parpadeante

En CubeMX, configura **PA5** como `GPIO_Output` y ponle la etiqueta `LD2`. Genera el código y, **en CubeIDE, refresca el proyecto con F5**: si no, sigue viendo los ficheros de antes. Después escribe el parpadeo dentro del bucle infinito, **entre las marcas USER CODE**:

DENTRO DE USER CODE BEGIN WHILE

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
HAL_Delay(500);
```

Compila, graba y comprueba que parpadea una vez por segundo: medio segundo encendido y medio apagado.

Paso 4. El botón

En CubeMX, configura **PC13** como `GPIO_Input` con la etiqueta `B1`, genera y refresca como antes. Es el botón azul de la placa, y está cableado **activo a cero**: pulsado se lee `GPIO_PIN_RESET`.

LA LECTURA

```
if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET) {
    /* pulsado */
}
```

> Léelo tal cual, sin filtrar nada. Un pulsador real no cambia de estado limpiamente: rebota durante unos milisegundos. Aquí da igual, porque solo se mira si está pulsado o no. En la Práctica 2 sí importará, y ahí se verá cómo se trata.

Paso 5. Las dos esperas

Las dos esperas del título son estas. **La primera ya la estás usando**: es la del Paso 3, la que viene hecha en la biblioteca. Se le pide un tiempo en milisegundos.

LA ESPERA DE LA BIBLIOTECA

```
HAL_Delay(500);   /* medio segundo */
```

La segunda hay que escribirla, y no usa la HAL para nada: espera **contando vueltas** de un bucle vacío.

LA ESPERA «A PELO»

```
static void EsperaContando(uint32_t vueltas)
{
    for (volatile uint32_t i = 0; i < vueltas; i++) {
        /* a proposito, aqui no hay nada */
    }
}
```

> El `volatile` es imprescindible. Sin él, el compilador tiene permiso para ver un bucle cuyo resultado no usa nadie y **borrarlo entero**: el programa quedaría sin espera y el LED parpadearía a una velocidad invisible.

Puedes comprobarlo, pero **hay que hacerlo en la configuración Release**, no en Debug. Y esa diferencia es parte de la lección:

CONFIGURACIÓN	OPTIMIZACIÓN	CON VOLATILE	SIN VOLATILE
Debug	`-O0`	el bucle está	**el bucle sigue estando**
Release	`-Os`	el bucle está	**el bucle desaparece**

> En Debug el compilador no optimiza nada, así que quitar `volatile` no cambia el resultado y parece que la palabra sobra. **Sobra hasta el día que compilas con optimización**, y entonces el programa hace algo distinto sin que hayas tocado una línea. Por eso `volatile` no es una recomendación de estilo: es la única forma de decirle al compilador que ese bucle importa aunque no lo parezca.

Ahora haz que el botón elija entre las dos esperas, y **ajusta el número de vueltas con el cronómetro** hasta que el parpadeo con el botón pulsado se parezca al de sin pulsar. Anota el número al que has llegado.

Paso 6. Cambiar el reloj y volver a medir

Abre el `.ioc` en CubeMX, ve a **Clock Configuration** y cambia el origen del reloj del sistema del PLL al oscilador interno directo: de 84 MHz a 16 MHz. Regenera el código, refresca en CubeIDE, compila y graba **sin tocar ni una línea del programa**.

Mide otra vez las dos velocidades de parpadeo con el cronómetro y rellena esta tabla:

	PARPADEO CON HAL_DELAY	PARPADEO CONTANDO VUELTAS
A 84 MHz		
A 16 MHz		
¿Ha cambiado?		

Antes de mirar el resultado, apunta qué esperas que ocurra. Luego mídelo. **Si no coincide, esa diferencia es justo lo que hay que explicar en la memoria.**

> Cuando termines, **vuelve a dejar el proyecto a 84 MHz**: es la configuración con la que trabajan las siete prácticas de la asignatura.

Paso 7. Comprueba el reloj sin fiarte del código

En el paso anterior has deducido la frecuencia por el comportamiento del parpadeo. Ahora compruébala **mirándola**, que es distinto.

36. Lanza el programa en modo depuración y párala en cualquier punto.
37. Abre la vista de registros de periféricos y busca los del **RCC**.
38. Localiza los valores del PLL y comprueba que son los que dice tu configuración.

> **El registro no miente.** Si el código dice una cosa y el registro dice otra, la configuración no llegó a aplicarse, y ese es un problema muy distinto de un error de cálculo. Poder distinguir los dos casos en treinta segundos es la razón de aprender a usar esta vista.

En la memoria, indica **qué frecuencia has leído en el registro** y si coincide con la que esperabas.

6. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Encender la placa con el programa grabado.	El LED verde parpadea una vez por segundo, sin el depurador conectado.
CA-2	Mantener pulsado el botón azul.	El LED sigue parpadeando a un ritmo **parecido**, mediante la espera contada.
CA-3	Quitar `volatile` del bucle de espera y compilar **en Release**.	El parpadeo con el botón pulsado se vuelve **invisible**: el compilador ha borrado el bucle. En **Debug** la misma prueba no cambia nada, y hay que saber decir por qué.
CA-4	Volver a poner `volatile`. Cambiar el reloj a 16 MHz y regenerar.	El parpadeo con `HAL_Delay` **no cambia**; el de las vueltas se hace unas **cinco veces más lento**.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-5	Regenerar el código desde CubeMX sin cambiar nada.	El programa sigue estando ahí, porque se escribió dentro de los bloques USER CODE.
CA-6	Poner un punto de ruptura en el `HAL_GPIO_TogglePin` y depurar.	El depurador para en **esa línea de `main.c`**, no en ensamblador.

> CA-6 no es un adorno: es la comprobación de que la ruta del proyecto no tiene tildes. Si te salta a una ventana de ensamblador, revisa el aviso del Paso 1 antes de seguir con la Práctica 2.

7. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Proyecto y entorno	CA-1 y CA-6: el proyecto se crea, compila, graba y **depura**.	Compila y graba, pero el depurador no encuentra las fuentes.	1,5
2. Configuración de pines	PA5 salida y PC13 entrada, hechos desde el `.ioc` y con etiqueta.	Los pines funcionan pero se configuraron a mano en el código.	1,5
3. El contrato del generador	CA-5: todo el código propio dentro de los bloques USER CODE.	Hay código fuera de las marcas, aunque funcione.	2,0
4. Las dos esperas	CA-2 y CA-3: la espera contada funciona, y se entiende que	Funciona, pero no sabe explicar por qué hace falta	2,0

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
	`volatile` solo se nota con el optimizador activado.	`volatile`, o cree que en Debug también se nota.	
5. La medida del reloj	CA-4: la tabla del Paso 6, con las cuatro medidas tomadas.	La tabla está a medias o los valores no son creíbles.	2,0
6. Memoria	Las dos cuestiones del apartado 8.	Responde una de las dos, o sin justificar.	1,0

8. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin las carpetas Debug ni Release**. Nombre: ACII_P1_Apellido1_Apellido2_Nombre.zip
- **La tabla del Paso 6** con las cuatro medidas y el número de vueltas al que llegaste.
- **Memoria breve**, máximo dos páginas:
    - Enumera los pines que CubeMX configuró sin que se lo pidieras al elegir la placa, y explica para qué sirve cada uno. ¿Qué pasaría si desactivaras los dos del depurador?
    - `HAL_Delay(500)` espera lo mismo a 16 que a 84 MHz. ¿Cómo puede ser, si el micro va cinco veces más despacio? Averigua de dónde saca `HAL_Delay` su base de tiempo y explícalo.

9. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.

- **No sustituir la medida.** Los tiempos del Paso 6 los tomas tú, con la placa y un cronómetro.

Anexo A. Las claves del .ioc que conviene reconocer

CLAVE	QUÉ SIGNIFICA
`Mcu.CPN`	La referencia exacta del chip: `STM32F401RET6`.
`board`	La placa elegida. De aquí salen los pines que aparecen solos.
`Mcu.IPn`	Cada periférico activado: NVIC, RCC, SYS, USART2…
`Mcu.Pinn`	La lista de pines en uso.
`PA5.Signal`	Qué papel juega el pin: `GPIO_Output`, `GPIO_Input`, `USART2_TX`…
`PA5.GPIO_Label`	El nombre que tendrá en `main.h`. De aquí salen `LD2_Pin` y `LD2_GPIO_Port`.
`RCC.SYSCLKFreq_VALUE`	La frecuencia del núcleo, en hercios.
`RCC.PLLM`, `PLLN`, `PLLP`	Los tres divisores y el multiplicador que la producen.
`NVIC.*_IRQn`	Qué interrupciones están activadas.
`ProjectManager.KeepUserCode`	Si se conserva lo que hay entre las marcas USER CODE.
`ProjectManager.ProjectName`	El nombre del proyecto. Ojo: puede no coincidir con el de la carpeta.
`ProjectManager.TargetToolchain`	Para qué entorno se genera. Tiene que ser `STM32CubeIDE`: CubeMX propone `EWARM`, el de IAR.
`ProjectManager.UnderRoot`	Si el proyecto se genera en la raíz de la carpeta, que es donde lo busca la importación de CubeIDE. Tiene que valer `true`.

Anexo B. Qué hay en la carpeta del proyecto

RUTA	QUÉ CONTIENE	¿SE TOCA?
`Core/Src/main.c`	El programa. Aquí trabajas, dentro de las marcas.	sí
`Core/Inc/main.h`	Las definiciones de los pines.	no
`Core/Src/stm32f4xx_it.c`	Las funciones de atención a interrupciones.	dentro de las marcas
`Core/Src/stm32f4xx_hal_msp.c`	El arranque de reloj y pines de cada periférico.	no
`Core/Startup/`	El arranque en ensamblador y la tabla de vectores.	no
`Drivers/`	La biblioteca HAL de ST.	no
`*.ld`	Dónde se coloca cada cosa en memoria.	no
`*.ioc`	La configuración. La fuente de todo lo anterior.	sí, con CubeMX
`Debug/`, `Release/`	Resultado de compilar. **No se entrega.**	no

Anexo C. Conexionado

Esta práctica no necesita ningún componente externo: todo está ya en la placa.

SEÑAL	PIN	DETALLE
LED verde LD2	PA5	Activo a 1: se enciende escribiendo un uno.
Botón azul B1	PC13	Activo a 0: pulsado se lee `GPIO_PIN_RESET`.

SEÑAL	PIN	DETALLE
Depurador SWD	PA13 / PA14	Los pone CubeMX solo. **No los desactives.**
Puerto serie	PA2 / PA3	USART2, conectado al ST-LINK. Aquí no se usa.

Práctica 2 · Control de una puerta de garaje

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesiones 2 y 3 de 13
Duración	4 horas presenciales, en dos sesiones
Modalidad	Individual, en el laboratorio
Requisitos previos	Práctica 1 superada: crear un proyecto en STM32CubeIDE, configurar GPIO y compilar
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · biblioteca TM1638.c / TM1638.h
Entregables	Proyecto CubeIDE comprimido, diagrama de estados y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la segunda sesión, por Moovi

2. Objetivos de aprendizaje

Al terminar esta práctica serás capaz de:

- Traducir un enunciado en lenguaje natural a una **máquina de estados finitos** explícita, con sus estados, transiciones y acciones separadas.
- Escribir un bucle de control **no bloqueante**: el programa atiende todas las entradas en todo momento, sin quedarse esperando dentro de una espera.
- Detectar **flancos** en una entrada digital y filtrar los rebotes mecánicos de un pulsador sin detener el programa.

- Medir tiempo con **HAL_GetTick()** y construir una base de tiempo periódica que no acumule deriva.
- Gobernar los LEDs y el display de siete segmentos de un módulo TM1638 desde una biblioteca ya escrita.

> Esta práctica no va de mover una puerta. Va de **separar el estado del sistema de la lógica que lo cambia**. Es el mismo patrón que encontrarás en un controlador de ascensor, en un cajero o en el arranque de un motor industrial.

3. El sistema que hay que construir

Un automatismo de puerta de garaje tiene que resolver tres cosas a la vez: saber en qué situación está la puerta, reaccionar al mando a distancia y no aplastar a nadie. Vas a construir ese automatismo completo, sustituyendo el motor y los sensores reales por el módulo LED&KEY.

La puerta puede estar cerrada, abierta, moviéndose en cualquiera de los dos sentidos o detenida a mitad de camino. El mando a distancia es un único pulsador: no dice "abre" ni "cierra", solo dice "cambia". Qué signifique ese "cambia" depende de lo que la puerta estuviera haciendo, y ahí está el problema que resuelve la máquina de estados.

4. Modelo del sistema

Todo lo que sigue es de obligado cumplimiento. Los valores concretos importan: los criterios de aceptación del apartado 7 se comprueban con ellos.

4.1 Recorrido, velocidad y posición

MAGNITUD	VALOR	CONSTANTE EN EL CÓDIGO
Recorrido total de la hoja	300 cm (3,00 m)	RECORRIDO_CM
Avance en cada paso	3 cm	PASO_CM

MAGNITUD	VALOR	CONSTANTE EN EL CÓDIGO
Periodo del paso	100 ms	TICK_MOV_MS
Velocidad resultante	30 cm/s	—
Duración del recorrido completo	10,0 s	—

La variable **posicion** guarda la posición de la hoja en centímetros: vale **0 con la puerta cerrada** y **300 con la puerta completamente abierta**. Se actualiza únicamente cuando el motor está en marcha, sumando o restando PASO_CM una vez por cada periodo de 100 ms.

> Ojo con la unidad. La posición se lleva **en centímetros**, no en "número de LED". Los ocho LEDs son una representación de la posición, no la posición misma. Confundirlos es el error más frecuente en esta práctica.

4.2 Entradas

ENTRADA	DISPOSITIVO FÍSICO	PAPEL EN EL SISTEMA
MANDO	Pulsador azul de usuario de la NUCLEO (PC13). Activo a nivel bajo.	Mando a distancia. Cada pulsación es una orden de "cambia lo que estés haciendo".
FOTOCÉLULA	Tecla **S1** del módulo LED&KEY.	Detector de presencia. Mientras esté pulsada hay un obstáculo bajo la puerta.

> **Cuidado al cablear: estas dos entradas se llaman MANDO y FOTOCÉLULA, no B0 y B1.** En el serigrafiado de la placa **B1 es el pulsador azul**, así que usar esos nombres lleva a conectar la entrada equivocada.

El pulsador es un contacto mecánico y **rebota**: al cerrarse produce durante unos milisegundos una ráfaga de transiciones. Si cuentas flancos sin filtrar, una sola pulsación se convierte en varias órdenes y la puerta hace cosas absurdas. Tendrás que filtrarlo, y el apartado 6 te dice cómo hacerlo sin bloquear.

4.3 Salidas

SALIDA	SOPORTE	QUÉ REPRESENTA
motorAbrir / motorCerrar	Dos variables lógicas	Las dos órdenes del motor. **Las dos a 0 significa motor parado**, y por tanto puerta quieta. Nunca pueden estar las dos a 1 al mismo tiempo.
Barra de posición	LED 1 a LED 8 del módulo	Cuántos LEDs encendidos, más abierta está la puerta: 0 LEDs con la puerta cerrada, 8 con la puerta abierta.
Display	Los 8 dígitos de siete segmentos	El estado en reposo, o la cuenta atrás en segundos mientras la puerta se mueve.

Que las dos órdenes del motor sean variables y no LEDs es deliberado: son la **salida real** de la máquina de estados, y en un equipo de verdad irían a un contactor. En la memoria tendrás que justificar en qué estados vale 1 cada una.

5. Comportamiento exigido

5.1 Estados

ESTADO	SITUACIÓN	DISPLAY	MOTOR
CERRADO	Hoja abajo, posicion = 0	cErrAdA	parado
ABRIENDO	Hoja subiendo	cuenta atrás	abrir

ESTADO	SITUACIÓN	DISPLAY	MOTOR
ABIERTO	Hoja arriba, posicion = 300	AbiErtA	parado
CERRANDO	Hoja bajando	cuenta atrás, o bArrErA si hay obstáculo	cerrar
PARADO_ABRIENDO	Detenida a mitad; venía abriendo	PArAdA	parado
PARADO_CERRANDO	Detenida a mitad; venía cerrando	PArAdA	parado

> Hacen falta **dos** estados de parada, no uno. El enunciado exige que la puerta invierta la marcha, así que el sistema tiene que recordar en qué sentido iba cuando se detuvo. Con un único estado PARADO esa información se pierde y el requisito no se puede cumplir.

5.2 Transiciones

La única entrada que provoca transiciones es el flanco de pulsación del MANDO, salvo los dos finales de recorrido.

ESTADO ACTUAL	SUCESO	ESTADO SIGUIENTE
CERRADO	pulsación de MANDO	ABRIENDO
ABIERTO	pulsación de MANDO	CERRANDO
ABRIENDO	pulsación de MANDO	PARADO_ABRIENDO
ABRIENDO	posicion alcanza 300 cm	ABIERTO
CERRANDO	pulsación de MANDO	PARADO_CERRANDO

ESTADO ACTUAL	SUCESO	ESTADO SIGUIENTE
CERRANDO	posicion alcanza 0 cm	CERRADO
PARADO_ABRIENDO	pulsación de MANDO	CERRANDO
PARADO_CERRANDO	pulsación de MANDO	ABRIENDO

5.3 La fotocélula

La fotocélula **solo actúa durante el cierre**. Es lo que hace una instalación real: al abrirse, la puerta se aleja del obstáculo y no hay peligro; al cerrarse, lo aplasta.

- Mientras haya obstáculo y el estado sea CERRANDO, la orden **motorCerrar pasa a 0** y la hoja deja de moverse de inmediato.
- El sistema **no cambia de estado**: sigue en CERRANDO, esperando. En el display aparece bArrErA.
- En cuanto desaparece el obstáculo, el cierre **se reanuda solo**, sin necesidad de tocar el mando.
- Durante la apertura la fotocélula se ignora por completo.

6. Desarrollo paso a paso

Trabaja en este orden y comprueba cada paso en la placa antes de seguir con el siguiente. Un paso que no funciona contamina todos los que vienen detrás.

Paso 1. Proyecto y biblioteca

Crea el proyecto en STM32CubeMX para la NUCLEO-F401RE e impórtalo en STM32CubeIDE, igual que en el Paso 1 de la P1 —con **STM32CubeIDE** como Toolchain/IDE— . Configura en CubeMX los tres pines del módulo como salidas (CLK, STB, DIO), genera, añade TM1638.c y TM1638.h a Core/Src e inicialízalo. El anexo A tiene la referencia de la biblioteca y el anexo B el conexionado.

Si usas **sprintf** o **snprintf**, incluye <stdio.h> en el bloque USER CODE Includes. Sin esa línea el compilador de STM32CubeIDE 2.x **da error**, no aviso, y el proyecto no construye.

Guarda el proyecto en una carpeta sin letras acentuadas. Si la ruta lleva alguna —una carpeta llamada «Práctica 2», o un usuario que se llame «José»— el programa compilará y se grabará en la placa sin problema, pero **al depurar CubeIDE no encontrará el main.c** y solo te ofrecerá ver ensamblador. El compilador guarda la ruta dentro del ejecutable en UTF-8 y el depurador la lee con la codificación de Windows: para cualquier letra acentuada las dos no coinciden. Los espacios y los paréntesis no molestan; solo las tildes y la eñe. Si ya te ha ocurrido, mueve el proyecto a una ruta limpia y recompila desde cero con Project → Clean.

Paso 2. Base de tiempo de 100 ms

Antes de tocar la máquina de estados, consigue un pulso periódico fiable. La forma correcta es comparar HAL_GetTick() contra una marca previa, y avanzar la marca sumándole el periodo:

BASE DE TIEMPO SIN DERIVA

```
static uint32_t tMov = 0;

bool tick = (HAL_GetTick() - tMov) >= TICK_MOV_MS;
if (tick) {
    tMov += TICK_MOV_MS;   /* sumar el periodo, NO asignar HAL_GetTick()
*/
}
```

Si en lugar de sumar el periodo asignases tMov = HAL_GetTick(), cada vuelta perdería los microsegundos que el bucle tarda en llegar hasta ahí, y el error se acumularía: al cabo de un minuto el reloj iría notablemente atrasado. Explica esta diferencia en la memoria.

Paso 3. Flanco del mando, con antirrebote no bloqueante

Necesitas detectar el instante en que el pulsador pasa a pulsado, una sola vez por pulsación. La técnica es esperar a que la lectura lleve estable un tiempo mínimo antes de darla por buena:

ANTIRREBOTE POR VENTANA DE ESTABILIDAD

```
static bool     estable = false;   /* estado ya validado   */
static bool     ultimo  = false;   /* ultima lectura cruda */
static uint32_t tCambio = 0;

bool crudo = (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET);

if (crudo != ultimo) {             /* la senal se movio: reinicia la
ventana */
    ultimo  = crudo;
    tCambio = HAL_GetTick();
}
if ((HAL_GetTick() - tCambio) >= ANTIRREBOTE_MS && crudo != estable) {
    estable = crudo;
    return estable;                /* true solo al pulsar, no al soltar
*/
}
return false;
```

> **Prohibido usar HAL_Delay para el antirrebote.** Durante esa espera el programa no lee la fotocélula ni mueve la puerta: el sistema se queda ciego. Un HAL_Delay(500) con pasos de 100 ms pierde cinco pasos de movimiento. Cualquier entrega que use HAL_Delay dentro del bucle principal suspende el criterio 5 del baremo.

Paso 4. La máquina de estados

Escribe el bucle principal en cinco bloques, siempre en este orden. Mantenerlos separados es lo que hace que el programa se pueda leer y depurar:

39. **Leer las entradas** una sola vez por vuelta y guardarlas en variables locales.
40. **Transiciones**: un switch sobre el estado que solo decide cuál será el estado siguiente. Aquí no se toca nada más.

41. **Órdenes del motor**: calcular motorAbrir y motorCerrar a partir del estado y del obstáculo.
42. **Movimiento**: si toca tick y el motor está en marcha, actualizar posicion.
43. **Visualización**: display y barra de LEDs, según el estado y la posición.

La tentación es mezclar la decisión con la acción dentro del mismo switch. Resiste: en cuanto el sistema crece, ese código se vuelve imposible de seguir.

Paso 5. Visualización

La barra de LEDs se obtiene escalando la posición al número de LEDs, y la cuenta atrás dividiendo los centímetros que faltan entre la velocidad. Los dos son cálculos con enteros: cuida el orden de las operaciones para no perder precisión.

ESCALADO DE LA BARRA

```
uint8_t encendidos = (pos_cm * N_LEDS) / RECORRIDO_CM;   /* multiplicar
ANTES de dividir */
```

Paso 6. Mira la máquina de estados moverse

Una máquina de estados es fácil de escribir y difícil de comprobar: desde fuera solo se ve la puerta. Con el depurador se ve el estado.

44. Pon un **punto de ruptura condicional** en el bucle principal, con la condición `estado == stParadoAbriendo || estado == stParadoCerrando`: son los dos estados de parada que declara el `typedef enum` del proyecto.
45. Provoca esa transición con el pulsador y comprueba que el programa se detiene ahí, y solo ahí.
46. Inspecciona el estado y la posición: ¿son los que esperabas en ese instante?

> Un punto de ruptura **sin condición** en esa línea es inservible: para en cada transición y acabas pulsando «continuar» veinte veces. La condición es lo que convierte la herramienta en útil.

Aprovecha para comprobar una cosa más: pon un punto de ruptura en el antirrebote y observa **cuántas veces se ejecuta** por cada pulsación tuya. Anótalo en la memoria.

7. Criterios de aceptación

Tu programa se comprobará con estas seis pruebas, en la placa. Están redactadas para que puedas pasarlas tú mismo antes de entregar. Si alguna falla, todavía no has terminado.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Desde la puerta cerrada, una pulsación de MANDO y se deja correr.	La puerta abre por completo. El recorrido dura **10,0 s ± 0,3 s** y termina con los 8 LEDs encendidos y AbiErtA en el display.
CA-2	Durante la apertura, pulsar MANDO a mitad de recorrido; después, pulsar otra vez.	La puerta se detiene en el acto y muestra PArAdA. La segunda pulsación la pone a **cerrar**, no a abrir.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-3	Durante el cierre, mantener pulsada S1 unos segundos y después soltarla.	La hoja se detiene mientras S1 está pulsada, el display muestra bArrErA y la posición no cambia. Al soltar, **el cierre se reanuda solo**.
CA-4	Durante la apertura, mantener pulsada S1.	La apertura continúa con normalidad: la fotocélula no interviene al abrir.
CA-5	Observar la barra de LEDs a lo largo de un recorrido completo.	Progresa de 0 a 8 LEDs de forma monótona, y a mitad de recorrido hay exactamente **4 LEDs** encendidos.
CA-6	Observar el display durante un recorrido completo.	La cuenta atrás arranca en 10.0, decrece de forma continua y **nunca marca 0.0 con la puerta todavía en movimiento**.

8. Criterios de evaluación

> **Condición previa.** El proyecto entregado debe compilar en STM32CubeIDE **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

La práctica se califica sobre 10 puntos con el siguiente baremo. Cada criterio se valora en tres niveles: no alcanzado (0 puntos), alcanzado a medias (mitad del peso) y alcanzado (peso completo).

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Recorrido y temporización	CA-1: la puerta completa el recorrido en el tiempo previsto.	Abre y cierra, pero el tiempo se desvía más de 0,3 s del exigido.	2,0

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
2. Parada e inversión	CA-2: la pulsación detiene el movimiento y la siguiente invierte el sentido.	Detiene correctamente, pero al reanudar sigue en el mismo sentido.	2,0
3. Fotocélula	CA-3 y CA-4: detiene el cierre, reanuda al liberar y no afecta a la apertura.	Detiene el cierre pero exige tocar el mando para reanudar, o interviene también al abrir.	2,0
4. Visualización	CA-5 y CA-6: barra de LEDs y cuenta atrás correctas.	Uno de los dos elementos es correcto y el otro no.	1,5
5. Calidad del código	Bucle no bloqueante, sin HAL_Delay; transiciones y acciones separadas; constantes con nombre en lugar de números sueltos.	Funciona y está ordenado, pero queda algún HAL_Delay o algún número mágico.	1,5
6. Diagrama y memoria	Diagrama de estados coherente con el código entregado y justificación de las dos cuestiones del apartado 9.	Diagrama correcto pero sin justificar, o justificación sin diagrama.	1,0

9. Entregables

- **Proyecto de STM32CubeIDE** comprimido en un único archivo, **sin la carpeta Debug**. Nombre: ACII_P2_Apellido1_Apellido2_Nombre.zip
- **Diagrama de estados** con los seis estados y las ocho transiciones de la tabla del apartado 5.2. Vale a mano y fotografiado, siempre que se lea.
- **Memoria breve**, máximo dos páginas, que responda a estas dos preguntas:
    - Por qué hacen falta dos estados de parada y no uno.

- Qué diferencia hay entre avanzar la marca de tiempo sumándole el periodo y reasignarle HAL_GetTick(), y qué efecto tiene a lo largo de varios minutos.

Entrega por Moovi en el plazo indicado en la ficha. La práctica se defiende en el laboratorio: se te pedirá ejecutar los criterios de aceptación sobre tu propia placa y explicar cualquier parte del código.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué lo has usado.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques cualquier línea del código o que la modifiques en el momento. No saber explicar el propio código equivale a no haberlo entregado.
- **No sustituir el trabajo de diseño.** El diagrama de estados y las respuestas de la memoria son tuyos.

Anexo A. La biblioteca TM1638

El módulo LED&KEY se gobierna con TM1638.c y TM1638.h, que se te entregan ya escritos. No hay que modificarlos.

A.1 Inicialización

```
/* USER CODE BEGIN 0 */
TM1638 TM;
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
TM.clk_port = CLK_GPIO_Port;  TM.clk_pin = CLK_Pin;
TM.stb_port = STB_GPIO_Port;  TM.stb_pin = STB_Pin;
TM.dio_port = DIO_GPIO_Port;  TM.dio_pin = DIO_Pin;

tm1638_TurnOn(&TM, 7);        /* brillo maximo */
tm1638_DisplayClear(&TM);
/* USER CODE END 2 */
```

A.2 Funciones

FUNCIÓN	QUÉ HACE
tm1638_TurnOn(&TM, brillo)	Enciende el display. brillo de 0 a 7.
tm1638_DisplayClear(&TM)	Apaga todos los dígitos.
tm1638_Led(&TM, n, on)	Enciende (1) o apaga (0) el LED n, de 1 a 8.
tm1638_DisplayTxt(&TM, "texto")	Escribe una cadena, alineada a la derecha. Un punto se une al dígito anterior.
tm1638_DisplayChar(&TM, n, c, punto)	Escribe un carácter en la posición n.
tm1638_ScanButtons(&TM)	Lee de una vez el estado de las ocho teclas y lo devuelve en un byte.

FUNCIÓN	QUÉ HACE
tm1638_KeyState(&TM, teclas, n)	Sobre el byte anterior, indica si la tecla n está pulsada.
tm1638_ReadKey(&TM, teclas)	Sobre el byte anterior, devuelve el número de la tecla pulsada.

> El display **no puede representar cualquier letra**: son siete segmentos. Están disponibles las cifras, el espacio, A B C E F H I J L O P S U, y a b c d h i n o r t u y. Una letra fuera de esa lista aparece como un dígito en blanco o con basura. Comprueba tus cadenas antes de darlas por buenas.

Para leer las teclas, el patrón correcto es escanear una vez por vuelta de bucle y consultar después ese byte tantas veces como haga falta, en lugar de escanear una vez por cada consulta:

```
uint8_t teclas = tm1638_ScanButtons(&TM);
bool obstaculo = tm1638_KeyState(&TM, teclas, 1);   /* S1 = fotocelula
*/
```

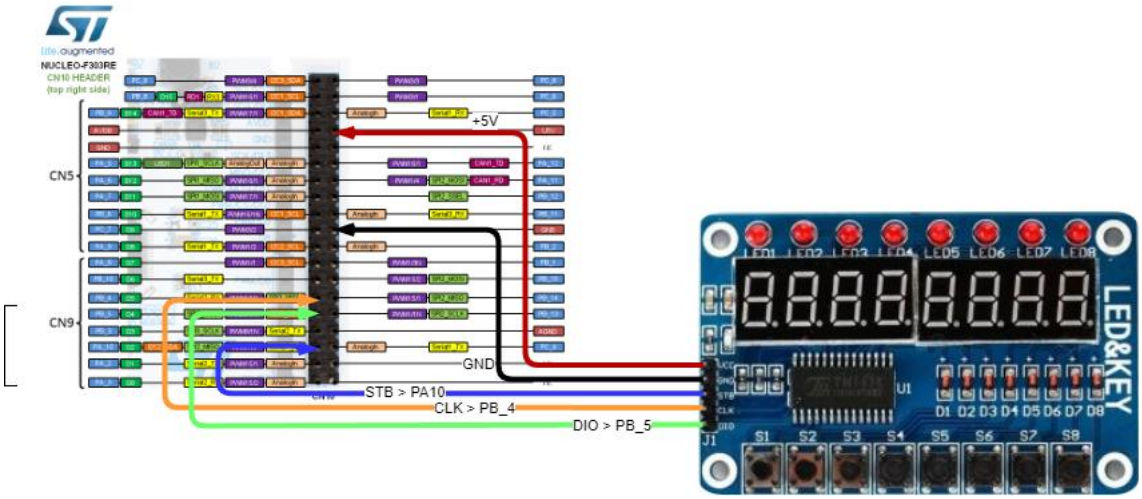
Anexo B. Conexionado

El módulo se alimenta a 3,3 V desde la propia placa y usa tres líneas de datos. Esta es la asignación de pines del proyecto de referencia; declara estas mismas etiquetas en CubeMX y los nombres CLK_Pin, STB_Pin y DIO_Pin aparecerán solos en main.h.

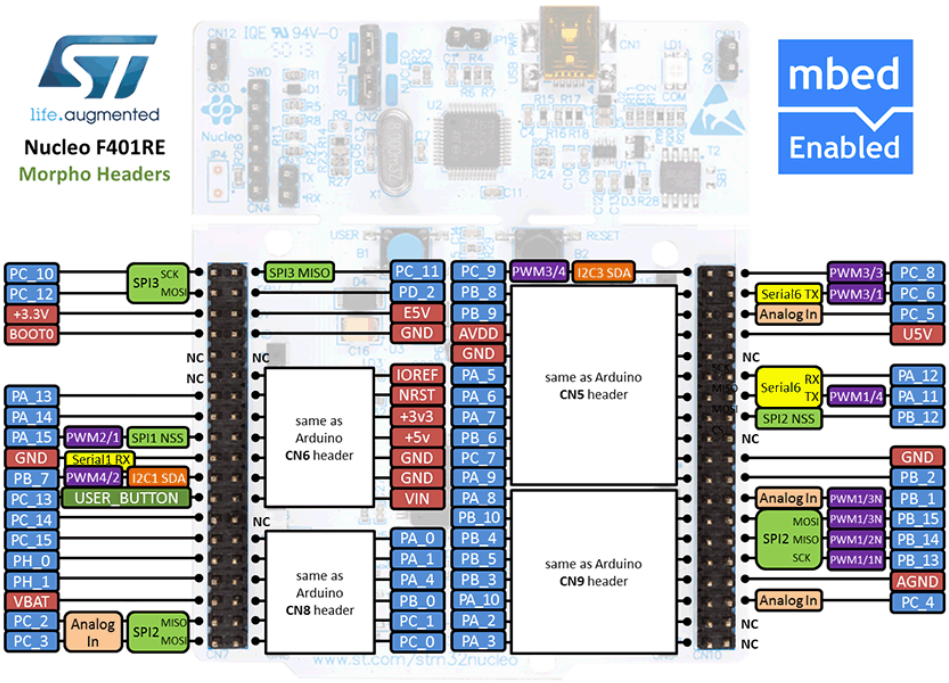
SEÑAL	PIN DEL STM32	ETIQUETA EN CUBEMX	MODO
STB	PA10	STB	GPIO_Output
CLK	PB4	CLK	GPIO_Output
DIO	PB5	DIO	GPIO_Output

SEÑAL	PIN DEL STM32	ETIQUETA EN CUBEMX	MODO
MANDO	PC13	B1	Entrada (pulsador azul de la placa)

> La línea **DIO es bidireccional**: la biblioteca la conmuta a entrada por su cuenta cuando toca leer las teclas y la devuelve a salida al terminar. Por eso en CubeMX se declara como salida y aun así funciona la lectura. En esta práctica el MANDO se lee **por consulta**, dentro del bucle; la versión por interrupción llega en la Práctica 4.



CONEXIÓN DEL MÓDULO LED&KEY A LA NUCLEO-F401RE.

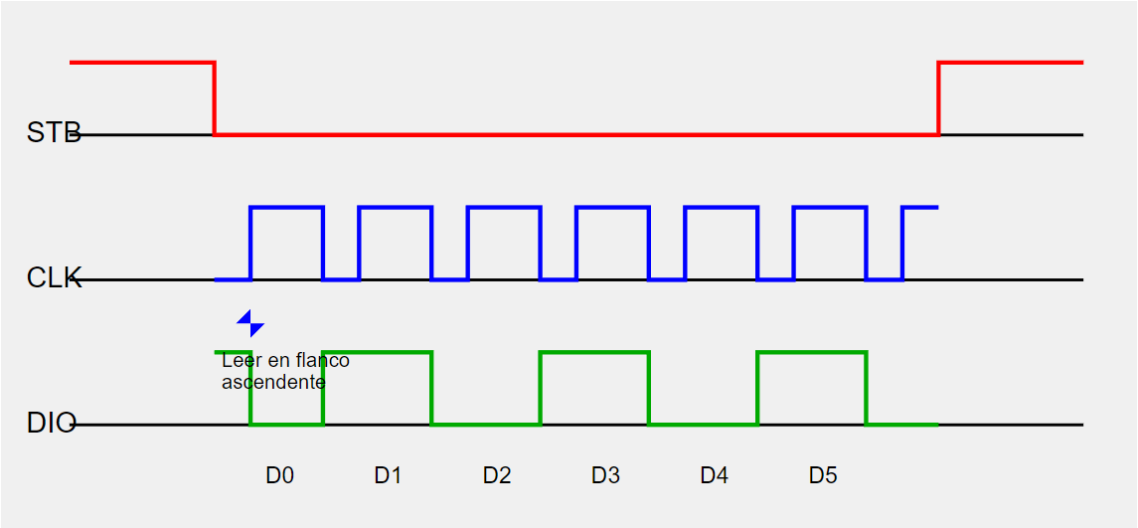


PINOUT DE LOS CONECTORES MORPHO DE LA NUCLEO-F401RE, PARA LOCALIZAR PA10, PB4 Y PB5.

Anexo C. El protocolo del TM1638

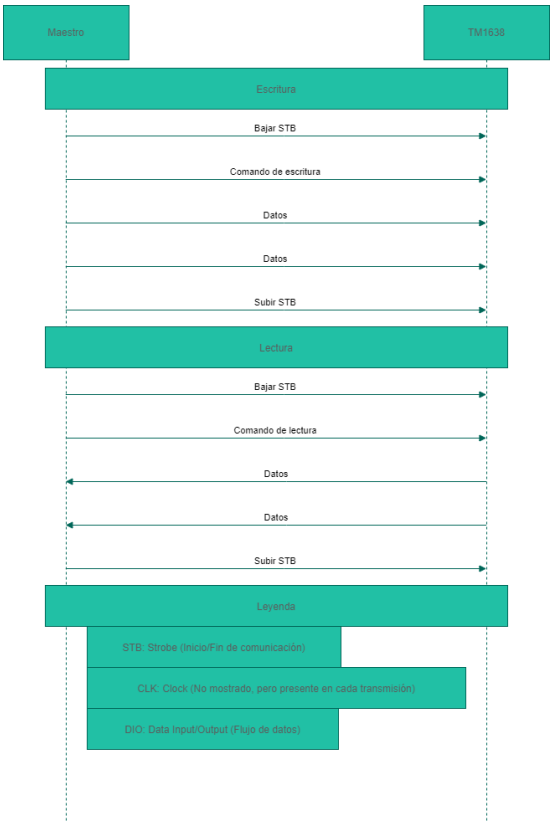
No necesitas implementarlo —la biblioteca ya lo hace—, pero sí entenderlo: es un protocolo serie síncrono de tres hilos, y aparece en el examen.

- **STB (Strobe)** delimita la trama. El maestro lo baja para empezar y lo sube para terminar; al subirlo, el TM1638 procesa lo recibido.
- **CLK (Reloj)** marca cada bit. El dato se lee en el **flanco de subida**.
- **DIO (Datos)** transporta los bits, en grupos de 8 y **empezando por el menos significativo**.
- El primer byte tras bajar STB es siempre un **comando**; los siguientes son datos.



CRONOGRAMA DE UNA TRAMA: STB EN ROJO, CLK EN AZUL, DIO EN VERDE. EL DATO SE LEE EN EL FLANCO ASCENDENTE DEL RELOJ. EL DIBUJO MUESTRA SEIS BITS POR CLARIDAD; EL TM1638 TRABAJA CON BYTES COMPLETOS.

La diferencia entre escribir y leer está en el comando inicial y en quién gobierna la línea DIO. Al escribir, la controla el maestro de principio a fin. Al leer, después del comando el microcontrolador conmuta su pin DIO a entrada y es el TM1638 el que emite.



SECUENCIA COMPLETA DE UNA ESCRITURA Y DE UNA LECTURA.

Práctica 3 · Cronómetro con precisión de centésimas

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 4 de 13
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	P2: bucle no bloqueante y biblioteca TM1638
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · biblioteca TM1638
Entregables	Proyecto CubeIDE, medida del error y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la sesión, por Moovi

2. Objetivos de aprendizaje

Al terminar esta práctica serás capaz de:

- Construir una **base de tiempo periódica** con `HAL_GetTick()` que no acumule deriva.
- Explicar por qué > y >= no son intercambiables cuando se comparan tiempos, y cuánto cuesta equivocarse.
- Descomponer una magnitud en cifras y **propagar el acarreo** entre unidades.
- Refrescar una pantalla **solo cuando el dato cambia**, en lugar de en cada vuelta del bucle.
- Medir el error de tu propio reloj contra una referencia externa y expresarlo en porcentaje.

> Esta práctica parece trivial y no lo es. Un cronómetro son veinte líneas, pero dos caracteres mal puestos hacen que **atrase casi seis segundos por minuto**. Y lo peor: funciona, se ve bonito y nadie lo nota hasta que lo compara con un reloj de verdad.

3. El problema

Medir tiempo en un microcontrolador parece fácil porque hay una función que da la hora: `HAL_GetTick()` devuelve los milisegundos transcurridos desde el arranque. Lo difícil no es leerla, es **usarla para marcar un ritmo** sin que el ritmo se vaya desviando.

En P2 ya construiste una base de tiempo de 100 ms para mover una puerta. Allí una desviación pequeña no se notaba: la puerta tardaba un poco más y ya está. Aquí sí se nota, porque lo que estás construyendo **es** el instrumento de medida.

4. Modelo del sistema

4.1 La base de tiempo

MAGNITUD	VALOR	CONSTANTE EN EL CÓDIGO
Resolución de `HAL_GetTick()`	1 ms	—
Periodo de cuenta	10 ms = 1 centésima	`TICK_MS`
Alcance del cronómetro	000,00 a 999,99 s	—
Al llegar al máximo	vuelve a 000,00	—

4.2 Cómo se guarda el tiempo

El cronómetro se lleva **descompuesto en cifras**, en un array de cuatro posiciones, y no como un entero de milisegundos. No es la forma más eficiente, y es deliberado: el objeto de la práctica es manejar el desbordamiento de cada unidad, que es lo que ocurre en cualquier contador real.

ÍNDICE	QUÉ GUARDA	RANGO
`c[0]`	Centésimas	0 a 99
`c[1]`	Unidades de segundo	0 a 9
`c[2]`	Decenas de segundo	0 a 9
`c[3]`	Centenas de segundo	0 a 9

> Ojo con `c[0]`: cuenta **centésimas de 0 a 99**, no décimas. Al desbordar incrementa directamente las unidades de segundo, porque 100 centésimas son un segundo. Confundir centésimas con décimas es el origen más frecuente de acarreos mal planteados.

4.3 Qué se ve en el display

Seis caracteres, en el formato `SSS.CC`. El punto no ocupa posición propia: la biblioteca TM1638 lo une al dígito anterior, así que en pantalla se ven seis dígitos con el punto entre el tercero y el cuarto.

EJEMPLOS DE LO QUE DEBE VERSE

```
000.00     al arrancar
009.99     nueve segundos y noventa y nueve centesimas
010.00     la centesima siguiente
999.99     el maximo, justo antes de volver a cero
```

5. Comportamiento exigido

47. Al arrancar, el cronómetro muestra **000.00** y empieza a contar de inmediato.
48. Cada **10 ms** avanza una centésima.
49. Los acarreos se propagan unidad por unidad: 100 centésimas hacen un segundo, 10 segundos una decena, 10 decenas una centena.

50. Al pasar de **999.99** vuelve a **000.00** sin saltos ni valores intermedios raros.
51. El display se actualiza **solo cuando el valor cambia**, no en cada vuelta del bucle.
52. El bucle principal **no contiene ninguna espera bloqueante**. Ni `HAL_Delay`, ni bucles de espera activa.

6. Desarrollo paso a paso

Paso 1. Mostrar un valor fijo

Antes de contar nada, consigue pintar `000.00` en el display. Escribe una función que reciba el array de cifras y componga la cadena de seis caracteres. Pruébala con valores inventados: 0, 123 y 999 con sus centésimas.

> Escribe esa función **fuera de `main`**, como función de fichero. Definir funciones dentro de otra función es una extensión de GNU: compila con este compilador y no compila con otros. Es el mismo aviso que en P2.

Paso 2. La base de tiempo

Ahora haz que avance una centésima cada 10 ms. Este es el corazón de la práctica y donde están las dos trampas:

BASE DE TIEMPO CORRECTA

```
static uint32_t tCuenta = 0;

tCuenta = HAL_GetTick();          /* una vez, antes del bucle */

if (HAL_GetTick() - tCuenta >= TICK_MS) {
    tCuenta += TICK_MS;
    /* avanzar una centesima */
}
```

> **Trampa 1: >= y no >.** Con > la condición no se cumple hasta los 11 ms, así que cada centésima dura un 10 % de más. El cronómetro atrasa **5,5 segundos por minuto**. Se ve perfecto y mide mal.

> **Trampa 2: sumar el periodo, no reasignar.** `tCuenta += TICK_MS` conserva el desfase con el que se llegó a comprobar la condición; `tCuenta = HAL_GetTick()` lo descarta, y ese error se acumula vuelta tras vuelta. Cuanto más tarde el bucle en dar una pasada, peor: con un bucle de 3 ms el atraso llega a 10 s por minuto.

Paso 3. Los acarreos

Cada vez que avanza una centésima hay que propagar el acarreo hacia arriba. Cuatro comprobaciones encadenadas, en orden, de la unidad más pequeña a la más grande.

Compruébalo en los puntos difíciles, que son siempre los cambios de decena: 0,99 → 1,00; 9,99 → 10,00; 99,99 → 100,00; y 999,99 → 000,00.

Paso 4. Refrescar solo cuando cambia

Si escribes en el display en cada vuelta del bucle, lo estarás redibujando miles de veces por segundo. El TM1638 se gobierna por un puerto serie lento, así que eso consume tiempo para nada y además **empeora la medida**, porque alarga la vuelta del bucle.

Mueve la actualización del display dentro del bloque que se ejecuta cada 10 ms.

Paso 5. Mide tu error

Arranca el cronómetro a la vez que el de tu móvil y déjalos correr **cinco minutos**. Anota la diferencia y exprésala en porcentaje. Ese número es el entregable del apartado 9.

> Si te sale un error en torno al **9 %**, casi seguro que tienes la trampa 1. Si te sale pequeño pero crece con el tiempo, es la trampa 2. Si te sale por debajo del 0,1 %, lo has hecho bien.

Paso 6. Caza el acarreo con el depurador

Los acarreos son el sitio donde más fallan los cronómetros, y esperar a que ocurran mirando la pantalla es lento. Se pueden provocar a voluntad.

53. Pon un **punto de ruptura condicional** dentro de `CronometroAvanza`, con la condición `c[0] == 99`. Ojo al nombre: dentro de la función el array llega como `c`, aunque fuera se llame `cronometro`.
54. Deja correr. El programa se detendrá justo antes del acarreo.
55. Avanza **paso a paso** y mira cómo se propaga: centésimas a cero, unidades más uno.

> Esto convierte una espera de hasta un segundo en algo instantáneo, y permite comprobar los cuatro acarreos —centésimas, unidades, decenas y la vuelta a cero— sin esperar mil segundos al último.

Aprovecha para observar un efecto del capítulo 11: al reanudar después de haber estado parado un rato, **el cronómetro no salta**. Explica en la memoria por qué, sabiendo cómo se construye la base de tiempo.

6.1 Ampliación opcional: el cronómetro de la bomba

Para quien termine pronto o quiera trabajarlo en casa. Sobre el cronómetro anterior:

- Cuenta **atrás** desde 10,00 s en lugar de hacia delante.
- Permite introducir un código de 4 bits con los pulsadores S1 a S4 del módulo.
- El flanco de S8 confirma el código.
- Cuatro intentos como máximo, indicados con los LEDs que quedan encendidos.
- Si se agota el tiempo o los intentos, los ocho LEDs parpadean sin parar.

No entra en la evaluación de esta práctica, pero es un buen entrenamiento para el proyecto final.

7. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Encender la placa y observar el display.	Muestra **000.00** y empieza a contar de inmediato.
CA-2	Cronometrar 60 s con un reloj externo y comparar.	La diferencia es **menor de 0,5 s**, es decir un error por debajo del 1 %.
CA-3	Cronometrar 5 minutos con un reloj externo.	La diferencia **no crece de forma apreciable** respecto a la de un minuto. Si crece, hay deriva.
CA-4	Observar el paso de 9,99 a 10,00 y de 99,99 a 100,00.	Los acarreos se propagan bien, sin dígitos intermedios erróneos.
CA-5	Dejarlo correr más de 1000 s.	Al pasar de **999.99** vuelve a **000.00** y sigue contando con normalidad.
CA-6	Revisar el código entregado.	No hay `HAL_Delay` ni esperas activas en el bucle, y las funciones auxiliares están **fuera de `main`**.

8. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar en STM32CubeIDE **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

La práctica se califica sobre 10 puntos. Cada criterio se valora en tres niveles: no alcanzado (0 puntos), alcanzado a medias (mitad del peso) y alcanzado (peso completo).

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Precisión	CA-2: el error en un minuto es menor del 1 %.	Cuenta, pero con un error de entre el 1 % y el 10 %.	2,5
2. Ausencia de deriva	CA-3: el error no crece con el tiempo.	El error en cinco minutos es más de tres veces el de un minuto.	2,0
3. Acarreos	CA-4 y CA-5: propagación correcta y vuelta a cero.	Falla solo en la vuelta a cero, o solo en un cambio de decena.	2,0
4. Código no bloqueante	CA-6: sin `HAL_Delay` ni esperas activas.	Sin `HAL_Delay`, pero el display se refresca en cada vuelta del bucle.	1,5
5. Calidad del código	Funciones fuera de `main`, constantes con nombre, sin avisos.	Funciona pero quedan números sueltos o alguna función anidada.	1,0
6. Medida del error	Paso 5: dato experimental propio, expresado en porcentaje.	Aporta la medida pero sin calcular el porcentaje, o sin comparar el minuto con los cinco.	1,0

9. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin las carpetas Debug ni Release**. Nombre: ACII_P3_Apellido1_Apellido2_Nombre.zip
- **Medida del error**: la diferencia observada en un minuto y en cinco minutos, y el porcentaje correspondiente.
- **Memoria breve**, máximo dos páginas, que responda a estas dos preguntas:

- Qué le pasa al cronómetro si se escribe > en lugar de >=, y por qué el error resulta ser justo del 9,1 %.
- Por qué `tCuenta += TICK_MS` no acumula deriva y `tCuenta = HAL_GetTick()` sí, y en qué situación se nota más la diferencia.

Entrega por Moovi en el plazo de la ficha. La práctica se defiende en el laboratorio: se te pedirá ejecutar los criterios de aceptación sobre tu propia placa y explicar cualquier parte del código.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué lo has usado.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques cualquier línea o que la modifiques en el momento.
- **No sustituir el trabajo de medida.** El error de tu cronómetro lo mides tú, con un reloj y un papel.

Anexo A. La función HAL_GetTick()

Devuelve el número de milisegundos transcurridos desde que arrancó el sistema. La mantiene el temporizador SysTick, que la biblioteca HAL configura por su cuenta para interrumpir cada milisegundo.

CARACTERÍSTICA	VALOR
Tipo devuelto	`uint32_t`
Resolución	1 milisegundo
Desbordamiento	A los 2^{32} ms, es decir unos **49,7 días**

> La resta `HAL_GetTick() - tCuenta` **funciona correctamente aunque el contador desborde**, siempre que las dos variables sean `uint32_t`. La aritmética sin signo da el resultado correcto módulo 2^{32}. Lo que **no** funciona es comparar directamente `HAL_GetTick() > tCuenta + TICK_MS`: ahí el desbordamiento sí rompe la comparación. Resta primero, compara después.

Anexo B. Por qué el error es exactamente del 9,1 %

Merece la pena hacer la cuenta, porque explica por qué un fallo de un solo carácter da un número tan concreto.

Con >=, la condición se cumple cuando han pasado 10 ms: cada centésima dura **10 ms** y todo cuadra.

Con >, la condición exige **más** de 10 ms, y como `HAL_GetTick()` avanza de milisegundo en milisegundo, el primer valor que la cumple es 11. Cada centésima dura entonces 11 ms en lugar de 10.

LA CUENTA

```
tiempo mostrado / tiempo real  =  10 / 11  =  0,909...
```

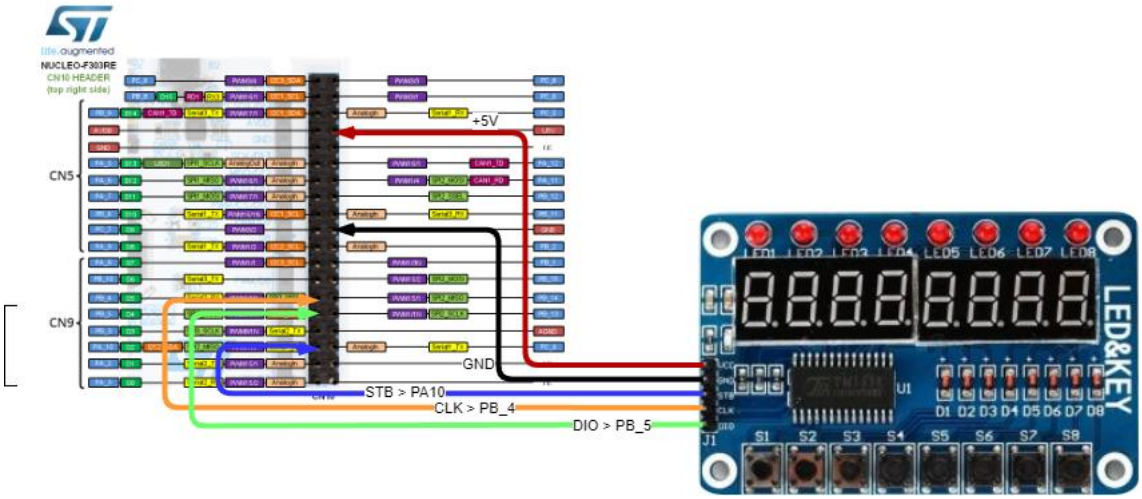
```
error  =  1 - 0,909  =  0,0909  =  9,1 %
```

En un minuto real el cronómetro marca 54,5 s: pierde 5,5 s. En cinco minutos pierde 27 s. Y sigue siendo un error **constante en porcentaje**, no acumulativo, que es lo que lo distingue del problema de la deriva.

Anexo C. Conexionado

El mismo de las prácticas anteriores. No hace falta cablear nada nuevo.

SEÑAL	PIN
TM1638 · STB	PA10
TM1638 · CLK	PB4
TM1638 · DIO	PB5



CONEXIÓN DEL MÓDULO LED&KEY A LA NUCLEO-F401RE.

Práctica 4 · Interrupciones: el pulsador, SysTick y TIM2

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesiones 6 y 7 de 13
Duración	4 horas presenciales, en dos sesiones
Modalidad	Individual, en el laboratorio
Requisitos previos	P1, P2 y P3. Se da por sabido el TM1638 y la cadencia con `HAL_GetTick()`
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · un reloj
Entregables	Un archivo con los cuatro proyectos, tabla de medidas y memoria (apartado 7)
Plazo	Una semana desde el final de la segunda sesión, por Moovi

2. Qué se construye

Hasta ahora todo lo que has escrito **preguntaba**: el bucle miraba una y otra vez si el pulsador había cambiado, o si ya habían pasado 100 ms. En esta sesión se da la vuelta al planteamiento: es el hardware el que **avisa**.

Se construyen cuatro programas, de menos a más:

PROYECTO	QUÉ HACE	QUÉ MECANISMO ENSEÑA
`P4_BtnInterrupcion`	El botón azul conmuta el LED verde	**EXTI**: un pin avisa al cambiar

PROYECTO	QUÉ HACE	QUÉ MECANISMO ENSEÑA
`P4_SYSTickInterrupcion`	El LED parpadea una vez por segundo	**SysTick**, el temporizador del núcleo
`P4_TIM2Interrupcion`	Lo mismo, con otro mecanismo	**TIM2**, un temporizador de periférico
`P4_Reloj`	Cronómetro completo con el TM1638	Los tres, trabajando juntos

> Fíjate en que los tres primeros tienen el **bucle principal vacío**. Todo ocurre en las funciones de atención a la interrupción. No es un truco de la práctica: es lo normal en un sistema empotrado bien hecho, y es justo lo contrario de lo que se hacía en P2 y P3.

3. Los tres mecanismos, y por qué son tres

MECANISMO	QUIÉN DISPARA	PERIODO	CUÁNDO CONVIENE
EXTI	Un pin cambia de estado	El que imponga el mundo exterior	Reaccionar a un pulsador o a un sensor sin encuestarlo.
SysTick	El propio núcleo Cortex-M	Corto y fijo	La base de tiempo del sistema. Es la que ya usa la HAL.
TIMx	Un periférico del microcontrolador	Muy amplio, gracias al prescalador	Periodos largos, frecuencias concretas y PWM.

La tercera columna es la que más importa y la que se descubre midiendo en el Paso 2. **SysTick no tiene prescalador**; los temporizadores de periférico sí. Esa diferencia decide cuál se puede usar para qué.

3.1 La palabra clave volatile

Toda variable que escriba una interrupción y lea el bucle principal —o al revés— tiene que declararse `volatile`:

LO QUE CRUZA LA FRONTERA

```
volatile uint32_t milisegundos = 0;
volatile uint32_t segundos = 0;
```

> Sin `volatile`, el compilador puede guardar la variable en un registro y no releerla nunca de memoria: el bucle esperaría un cambio que **sí ocurrió** pero que no ve. El programa compila, no avisa de nada y se queda colgado. Es el error más repetido de toda la asignatura.

`volatile` **no** hace la variable atómica ni la protege de accesos simultáneos: solo obliga a releerla. Para variables de 8, 16 o 32 bits en un Cortex-M4 el acceso ya es atómico por construcción, así que aquí basta.

4. Desarrollo paso a paso

Paso 1. El pulsador que avisa

Proyecto `P4_BtnInterrupcion`. Configura **PC13** en modo `GPIO_EXTI13` y activa su interrupción en la pestaña **NVIC**. El código va en el callback, no en el bucle:

LA FUNCIÓN DE ATENCIÓN

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}
```

Deja el `while(1)` vacío, compila y pruébalo. Pulsa el botón unas cuantas veces seguidas y **fíjate bien en lo que pasa**: es la primera cuestión de la memoria.

Paso 2. SysTick, y hasta dónde llega

Proyecto `P4_SYSTickInterrupcion`. El objetivo es que el LED conmute **una vez por segundo**. Empieza por lo que parece obvio:

EL INTENTO DIRECTO

```
/* Un segundo = tantas cuentas como ciclos de reloj hay en un segundo */
uint32_t resultado = HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq());
```

Guarda el valor de retorno en una variable y míralo con el depurador antes de seguir. Luego comprueba en la placa si el LED parpadea como esperabas.

> Si no funciona, no lo arregles a ciegas. Busca en el manual de referencia del Cortex-M4 **cuántos bits tiene el registro de recarga de SysTick**, calcula qué periodo máximo permite a la frecuencia a la que va tu placa, y de ahí sale tanto el porqué del fallo como la forma de resolverlo. Es la segunda cuestión de la memoria.

Cuando lo tengas, configura SysTick a un periodo que **sí** quepa y compón el segundo contando interrupciones dentro del callback:

COMPONER EL SEGUNDO

```
void HAL_SYSTICK_Callback(void)
{
    milisegundos++;
    if (milisegundos >= SYSTICK_POR_SEGUNDO) {
        milisegundos = 0;
        segundos++;
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}
```

> **Cuidado con una trampa:** definir `HAL_SYSTICK_Callback` no basta para que se ejecute. Alguien tiene que llamar a `HAL_SYSTICK_IRQHandler()` desde `SysTick_Handler`, en `stm32f4xx_it.c`. Si falta esa línea tu función es código muerto, y **el compilador no avisa**: la función existe, simplemente no la llama nadie.

Paso 3. El mismo segundo, con TIM2

Proyecto `P4_TIM2Interrupcion`. Configura TIM2 desde el `.ioc` para que interrumpa **una vez por segundo** y conmute el LED en `HAL_TIM_PeriodElapsedCallback`. Arráncalo con `HAL_TIM_Base_Start_IT`, no con `HAL_TIM_Base_Start`: sin `_IT` el temporizador cuenta pero no interrumpe.

Para elegir el prescalador y el periodo necesitas saber **a qué frecuencia le llega el reloj a TIM2**. No es la del núcleo: TIM2 cuelga del bus APB1. Mira la pestaña *Clock Configuration* y anota el número.

> El callback es común a **todos** los temporizadores, así que hay que comprobar cuál ha sido: `if (htim->Instance == TIM2)`. Sin esa comprobación, el día que añadas un TIM3 el programa hará el trabajo dos veces por periodo.

Compara este paso con el anterior: **el mismo segundo, dos mecanismos, y solo uno de los dos podía darlo directamente.**

Paso 4. El cronómetro

Proyecto `P4_Reloj`. Junta lo anterior en una aplicación con el TM1638:

- TIM2 marca la base de tiempo e incrementa el contador **dentro de la interrupción**.
- El bucle principal solo hace dos cosas: pintar el display y leer las teclas.
- Las teclas del módulo controlan el cronómetro:

TECLA	ACCIÓN
S1	Poner el contador a cero
S2	Parar
S3	Arrancar
S4	Contar hacia adelante
S5	Contar hacia atrás

> En cuenta atrás, al llegar a cero el cronómetro **se detiene solo**. Ese es el «fin de temporización» que convierte un contador en algo útil: un temporizador de cocina, un semáforo, un ciclo de lavado.

Paso 5. Confirma quién llama a tu código

Con interrupciones aparece una duda nueva: cuando una función se ejecuta, ¿quién la ha llamado? La pila de llamadas lo responde.

56. Pon un punto de ruptura dentro de la rutina de atención del pulsador.
57. Púlsalo. Cuando el programa se detenga, mira la **pila de llamadas**.
58. Repite con el callback del temporizador y compara las dos pilas.

Verás que ninguna de las dos viene de tu bucle principal: las dos aparecen colgando de la entrada de interrupción. Es la comprobación directa de que ese código **no lo llama nadie de los tuyos**.

> Comprueba también algo del capítulo 11: mientras estás parado dentro de una rutina de atención, **el contador de milisegundos no avanza**. Párate ahí un rato, anota el valor de `HAL_GetTick()`, continúa y vuelve a pararte. Explica en la memoria lo que has visto.

5. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Pulsar el botón azul con el bucle principal vacío.	El LED conmuta. La interrupción funciona sin que nadie encueste el pin.
CA-2	Comprobar el valor devuelto por el intento directo de configurar SysTick.	Se sabe **cuánto vale y por qué**, y está justificado con el tamaño del registro de recarga.
CA-3	Cronometrar el parpadeo de `P4_SYSTickInterrupcion` durante un minuto.	Sesenta conmutaciones, con menos de **1 s de diferencia** frente a un reloj externo.
CA-4	Lo mismo con `P4_TIM2Interrupcion`.	Igual de exacto, y el alumno sabe decir de dónde sale el reloj de TIM2.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-5	En `P4_Reloj`, arrancar, parar, poner a cero y cambiar de sentido con las teclas.	Las cinco teclas hacen lo suyo y el display responde sin retrasos apreciables.
CA-6	Poner la cuenta atrás cerca de cero y dejarla llegar.	Se detiene **sola** en 00:00 y no sigue contando en negativo.

> CA-2 no pide que el programa funcione: pide **entender por qué el primer intento no funcionaba**. Es el contenido más valioso de la sesión y se evalúa aunque el proyecto acabe bien.

6. Criterios de evaluación

> **Condición previa.** Los cuatro proyectos deben compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Interrupción externa	CA-1: EXTI configurada desde el `.ioc` y atendida en el callback.	Funciona, pero el pin se lee además en el bucle.	1,5
2. El límite de SysTick	CA-2: se explica por qué falla el intento directo, con números.	Sabe que falla, pero no sabe decir por qué.	2,5
3. Bases de tiempo correctas	CA-3 y CA-4: el segundo es un segundo con los dos mecanismos.	Uno de los dos parpadea a un ritmo que no es el pedido.	2,0

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
4. La aplicación	CA-5 y CA-6: el cronómetro responde y la cuenta atrás se detiene sola.	Cuenta bien pero alguna tecla no responde, o no para en cero.	2,0
5. Uso de `volatile`	Está en todo lo que cruza la frontera, y sabe decir por qué.	Lo usa porque lo pone el enunciado, sin saber qué evita.	1,0
6. Memoria	Las dos cuestiones del apartado 7.	Responde una de las dos, o sin justificar.	1,0

7. Entregables

- **Los cuatro proyectos** de STM32CubeIDE comprimidos, **sin las carpetas Debug ni Release**. Nombre: ACII_P4_Apellido1_Apellido2_Nombre.zip
- **Tabla de medidas**: lo cronometrado en CA-3 y CA-4, y el valor que devolvió el intento directo de CA-2.
- **Memoria breve**, máximo dos páginas:
    - En el Paso 1, una sola pulsación a veces conmuta el LED más de una vez. Explica por qué ocurre y qué habría que hacer para evitarlo. ¿Basta con la interrupción?
    - El intento directo de configurar SysTick para un segundo no funciona. Explica con números por qué es imposible a la frecuencia de tu placa, y qué periodo máximo sí admite.

8. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.

- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.
- **No sustituir la medida.** Los tiempos de CA-3 y CA-4 los cronometras tú, con la placa delante.

Anexo A. Lo mínimo dentro de una interrupción

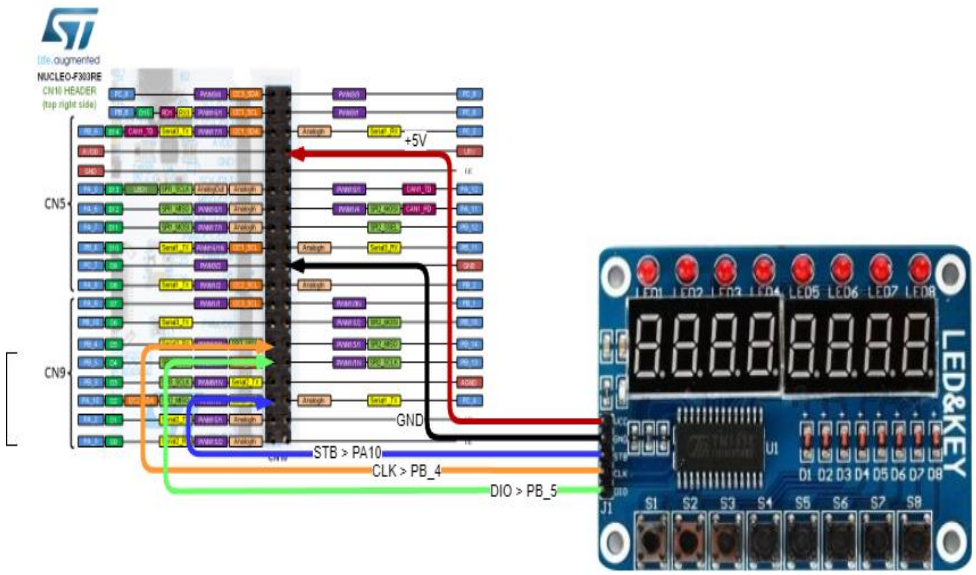
Una interrupción detiene lo que estuviera ejecutándose. Todo lo que se haga dentro se paga en latencia para el resto del sistema, así que la regla es: **contar, marcar una bandera y salir.** El trabajo de verdad se hace en el bucle principal, cuando le toca.

Por eso en `P4_Reloj` el callback solo incrementa contadores, y es el bucle el que escribe en el display. Pintar los ocho dígitos son decenas de transmisiones serie: dentro de una interrupción que se dispara mil veces por segundo, el sistema no levantaría cabeza.

Anexo B. Versión para PC

En `_simulacion_pc/reloj_interrupciones_pc.c` hay un reloj equivalente escrito para ordenador, con las interrupciones emuladas mediante señales de UNIX. Compila con `gcc` y sirve para entender el concepto sin pelearse con la configuración del microcontrolador.

Anexo C. Conexionado



CONEXIÓN DEL MÓDULO LED&KEY A LA NUCLEO-F401RE.

SEÑAL	PIN	DETALLE
LED verde LD2	PA5	Activo a 1

SEÑAL	PIN	DETALLE
Botón azul B1	PC13	Activo a 0. En esta práctica, como fuente de EXTI
TM1638 · STB	PA10	Solo en `P4_Reloj`
TM1638 · CLK	PB4	
TM1638 · DIO	PB5	

Práctica 5 · Conversión analógica-digital

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 8 de 13
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	P2 (máquina de estados y bucle no bloqueante) y P4 (interrupciones de temporizador)
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · potenciómetro · STM32CubeIDE · biblioteca TM1638
Entregables	Un archivo con los tres proyectos, tabla comparativa y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la sesión, por Moovi

2. Objetivos de aprendizaje

Al terminar esta práctica serás capaz de:

- Configurar el **conversor analógico-digital** de un STM32 y entender qué significan su resolución, su rango y su tiempo de muestreo.
- Leer el mismo canal de **tres formas distintas** —consulta, interrupción y DMA— y explicar qué le cuesta a la CPU cada una.
- Convertir las **cuentas** que devuelve el ADC en magnitudes físicas: tensión en voltios y temperatura en grados.

- Usar el **sensor de temperatura interno** del microcontrolador y la **referencia interna de tensión** para corregir la medida.
- Decidir con criterio cuál de los tres métodos conviene a un problema concreto.

> El objetivo real de esta práctica no es leer un potenciómetro: es **entender el coste de una lectura**. Los tres métodos dan el mismo número. Lo que cambia es cuánta CPU se queda esperando, y esa es la decisión que tendrás que tomar en cualquier sistema empotrado.

3. Del mundo físico al número

Un microcontrolador solo entiende números. La temperatura de una sala, la posición de un mando, la corriente que atraviesa un motor: todo eso son tensiones que varían de forma continua, y hace falta algo que las traduzca. Ese algo es el conversor analógico-digital.

En esta práctica vas a medir tres cosas a la vez con el mismo conversor: la posición de un potenciómetro, la temperatura del propio chip y su tensión de referencia interna. Las dos últimas no necesitan ningún cable: están dentro del microcontrolador.

4. El ADC del STM32F401RE

4.1 Parámetros

PARÁMETRO	VALOR	QUÉ SIGNIFICA
Resolución	12 bits	Devuelve un entero de **0 a 4095**. Nada intermedio: 4096 escalones y ya está.
Rango de entrada	0 a 3,3 V	Por encima de 3,3 V no mide más: satura, y puedes dañar el pin.
Canales usados	3	Potenciómetro, sensor de temperatura interno y referencia interna.

PARÁMETRO	VALOR	QUÉ SIGNIFICA
Escalón	≈ 0,8 mV	3,3 V / 4096. Es el mínimo cambio que el conversor puede distinguir.

De ahí sale la conversión básica de cuentas a voltios:

DE CUENTAS A TENSIÓN

```
tension = cuentas * 3.3 / 4095.0;      /* voltios */
```

> Se divide entre **4095**, no entre 4096. El valor máximo que devuelve un ADC de 12 bits es 4095, y ese es el que debe corresponder a 3,3 V. Dividir entre 4096 introduce un error sistemático pequeño pero real.

4.2 Qué se mide

ORDEN	CANAL	QUÉ ES	CABLEADO
1.º	VREFINT	Referencia interna de tensión, unos 1,21 V.	Ninguno: está dentro del chip.
2.º	TEMPSENSOR	Sensor de temperatura interno del microcontrolador.	Ninguno: está dentro del chip.
3.º	IN1 (PA1)	Potenciómetro. Es la única medida que viene de fuera.	Cursor del potenciómetro a **PA1**.

> **El potenciómetro va a PA1, no a PA0.** PA1 es el canal ADC1_IN1. Si lo conectas a PA0 no leerás nada, y el fallo es difícil de ver porque el programa funciona: simplemente mide un pin al aire.

El orden importa. Cuando se leen varios canales en secuencia, el ADC los recorre en el orden configurado y los deja en un array en ese mismo orden. Si te confundes de índice, medirás la temperatura creyendo que es el potenciómetro.

4.3 De cuentas a magnitudes

El potenciómetro se escala a un porcentaje de recorrido:

POSICIÓN DEL POTENCIÓMETRO

```
posicion = crudo[2] / 4095.0 * 100.0;   /* 0,0 a 100,0 % */
```

La temperatura es menos directa. El sensor interno entrega una tensión que crece de forma aproximadamente lineal con la temperatura, y el fabricante da dos constantes para interpretarla:

CONSTANTE	VALOR	SIGNIFICADO
V25	0,76 V	Tensión que da el sensor a 25 °C
Avg_Slope	2,5 mV/°C	Cuánto sube esa tensión por cada grado
VREFINT	1,21 V	Valor nominal de la referencia interna

DE CUENTAS A GRADOS

```
V_sensor = crudo[1] * V_ref / 4095.0;
temperatura = (V_sensor - 0.76) * 1000.0 / 2.5 + 25.0;   /* grados C */
```

Queda una pregunta: ¿de dónde sale `V_ref`? Ahí entra el tercer canal, y es el apartado siguiente.

4.4 Para qué sirve medir la referencia interna

Lo natural sería suponer que la alimentación son 3,3 V exactos y usar ese número. Pero la alimentación real de la placa no es exactamente 3,3 V, y además cambia con la temperatura, con la carga y con el USB del que cuelgue.

El truco es que **VREFINT vale siempre lo mismo**, unos 1,21 V, pase lo que pase con la alimentación. Así que midiéndolo se puede deducir cuánto vale de verdad la referencia del conversor:

REFERENCIA REAL, DEDUCIDA

```
V_ref = 4095.0 * 1.21 / crudo[0];   /* la alimentacion real, en voltios
*/
```

> Esto es autocalibración, y es una técnica de uso corriente en instrumentación. Sin ella, un error del 3 % en la alimentación se traslada íntegro a todas las medidas. En la memoria tendrás que razonar por qué esta corrección mejora la medida de temperatura y **no** mejora la del potenciómetro.

5. Las tres formas de leer el ADC

El conversor tarda un tiempo en hacer su trabajo. Lo que distingue a los tres métodos es qué hace la CPU mientras tanto.

5.1 Por consulta

La CPU arranca la conversión y **se queda esperando** a que termine, preguntando una y otra vez.

LECTURA POR CONSULTA

```
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);   /* aqui se bloquea
*/
valor = HAL_ADC_GetValue(&hadc1);
```

Sencillísimo de escribir y de depurar, y por eso es por donde se empieza. El problema está en la segunda línea: mientras dura, el programa no hace nada más. Si tu bucle tiene que atender botones o mover algo, esa espera se nota.

5.2 Por interrupción

La CPU arranca la conversión y **sigue con lo suyo**. Cuando el ADC termina, avisa, y una función de atención recoge el dato.

LECTURA POR INTERRUPCIÓN

```
HAL_ADC_Start_IT(&hadc1);          /* arranca y vuelve de inmediato */

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    valor = HAL_ADC_GetValue(hadc);
    listo = 1;                     /* volatile: lo modifica una
interrupcion */
}
```

> La bandera que comparten el bucle principal y la función de atención tiene que declararse **volatile**. Sin esa palabra el compilador puede suponer que nadie más la

> toca, guardarla en un registro y no releerla nunca: el bucle se queda esperando para siempre un cambio que sí ocurrió. Es el mismo problema que viste en P4.

5.3 Por DMA

El **acceso directo a memoria** es un periférico que copia datos sin molestar a la CPU. Se le dice una vez dónde dejar los resultados y se despreocupa: el ADC va convirtiendo canal tras canal y el DMA los va dejando en un array.

LECTURA POR DMA

```
uint16_t crudo[3];
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)crudo, 3);   /* una sola vez */
```

Es el único de los tres que escala bien a varios canales. Con consulta, leer tres canales cuesta tres esperas; con DMA cuesta lo mismo que leer uno, porque la CPU no participa.

5.4 Cuál usar

MÉTODO	COSTE PARA LA CPU	COMPLEJIDAD	CUÁNDO CONVIENE
Consulta	Alto: espera bloqueado	Mínima	Una medida puntual, o un programa que no tiene nada mejor que hacer.
Interrupción	Bajo	Media	Medidas esporádicas mientras el bucle atiende otras cosas.
DMA	Prácticamente nulo	Alta al configurar	Muestreo continuo, varios canales, o frecuencias altas.

No hay un ganador absoluto: hay una elección que justificar. Eso es lo que se te va a pedir en la memoria.

6. Desarrollo paso a paso

Tres programas, de menos a más. Cada uno se prueba en la placa antes de pasar al siguiente.

Paso 1. Consulta, un canal

Configura el ADC1 con un único canal en PA1, resolución de 12 bits, y muestra en el display el valor crudo. Comprueba que al girar el potenciómetro recorre de 0 a 4095 aproximadamente.

Después añade la conversión a tensión y muestra los voltios con un decimal.

Paso 2. El mismo canal, por interrupción

Parte del proyecto anterior. Sustituye la espera por el arranque en modo interrupción y la función de atención. La bandera compartida, **volatile**.

El resultado en pantalla debe ser idéntico al del paso 1: si cambia, algo va mal.

Paso 3. Tres canales por DMA, con temperatura

Añade los canales de temperatura y referencia interna, y activa el DMA en modo circular. Muestra en el display, alternando con el pulsador, la posición del potenciómetro y la temperatura del chip. Antepón un prefijo —`P` o `t`— para que se sepa qué magnitud se está viendo.

> Para comprobar que la temperatura responde de verdad, pasa primero el display a modo temperatura con el pulsador y luego aprieta el dedo contra el encapsulado del microcontrolador unos segundos. Debe subir algunas décimas. Si no se mueve, o estás leyendo el índice equivocado del array o el sensor no está habilitado en la configuración.

Paso 4. La comparación

Con los tres funcionando, mide cuánto tiempo consume cada lectura. Una forma sencilla: contar cuántas vueltas da el bucle principal en un segundo con cada método, usando `HAL_GetTick()`. Esa cifra es tu dato experimental.

Paso 5. Separa el dato de su presentación

Cuando un valor sale raro en pantalla hay dos sospechosos: la lectura y la conversión a unidades. El depurador los separa en un minuto.

59. Pon un punto de ruptura en el `return` de `LeerPotenciometro`, que es donde acaba de leerse el conversor.
60. Inspecciona el **valor crudo**, el número de 0 a 4095, antes de convertirlo.
61. Continúa e inspecciona el valor ya convertido a milivoltios.

Con el mando en el tope, el crudo debe ser 4095. Si lo es y la tensión mostrada no llega a 3,3 V, el fallo está en la conversión, no en la lectura. Si el crudo tampoco llega, el fallo está antes: en el montaje o en la configuración.

> Mira además, en la vista de periféricos, la configuración del conversor: **la resolución y el divisor de su reloj**. Comprueba que la frecuencia resultante está dentro del máximo del chip. Indícalo en la memoria.

7. Criterios de aceptación

Tu entrega se comprobará con estas pruebas, en la placa.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Girar el potenciómetro de un extremo a otro, en el programa por consulta.	El valor recorre de **0 a 4095** de forma continua, sin saltos ni valores congelados.
CA-2	Mostrar la tensión con el potenciómetro al mínimo, a la mitad y al máximo.	Aproximadamente **0,0 V, 1,6 V y 3,3 V**, con tolerancia de ±0,2 V.
CA-3	Comparar la lectura del programa por consulta con la del programa por interrupción.	Los dos muestran el mismo valor para la misma posición, dentro de ±2 cuentas.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-4	Alternar con el pulsador entre potenciómetro y temperatura.	El display cambia de magnitud, el prefijo indica cuál se ve (`P` o `t`), y **ambas siguen actualizándose**.
CA-5	Con el display en modo temperatura, apretar el dedo sobre el chip 10 segundos.	La temperatura **sube** de forma apreciable y vuelve a bajar al soltar. Requiere haber pasado CA-4.
CA-6	Revisar el código de la versión por interrupción.	La bandera compartida está declarada **volatile** y el bucle principal no contiene ninguna espera bloqueante.

8. Criterios de evaluación

> **Condición previa.** Los tres proyectos deben compilar en STM32CubeIDE **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

La práctica se califica sobre 10 puntos. Cada criterio se valora en tres niveles: no alcanzado (0 puntos), alcanzado a medias (mitad del peso) y alcanzado (peso completo).

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Lectura por consulta	CA-1 y CA-2: valor crudo y conversión a tensión correctos.	Lee el valor crudo pero la conversión a voltios está mal escalada.	2,0
2. Lectura por interrupción	CA-3 y CA-6: mismo resultado, sin bloqueo y con la bandera volatile.	Funciona pero falta volatile, o queda una espera bloqueante en el bucle.	2,0

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
3. DMA y temperatura	CA-4 y CA-5: tres canales, alternancia en pantalla y temperatura que responde.	El DMA funciona pero la temperatura no responde o el índice del array está cambiado.	2,5
4. Comparación medida	Paso 4: dato experimental del coste de cada método, no una opinión.	Compara los métodos pero sin haber medido nada.	1,5
5. Calidad del código	Sin números mágicos, constantes con nombre, sin avisos del compilador.	Funciona pero quedan valores literales sueltos o algún aviso.	1,0
6. Memoria	Las dos cuestiones del apartado 9, razonadas.	Responde una de las dos, o las dos sin justificar.	1,0

9. Entregables

- **Los tres proyectos de CubeIDE** comprimidos en un único archivo, **sin las carpetas Debug ni Release**. Nombre: ACII_P5_Apellido1_Apellido2_Nombre.zip
- **Tabla comparativa** de los tres métodos con tu medida experimental del paso 4.
- **Memoria breve**, máximo dos páginas, que responda a estas dos preguntas:
    - Por qué medir la referencia interna mejora la medida de temperatura y no mejora la del potenciómetro.
    - Qué método elegirías para muestrear un micrófono a 8000 muestras por segundo, y por qué los otros dos no sirven.

Entrega por Moovi en el plazo de la ficha. La práctica se defiende en el laboratorio: se te pedirá ejecutar los criterios de aceptación sobre tu propia placa y explicar cualquier parte del código.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué lo has usado.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques cualquier línea o que la modifiques en el momento. No saber explicar el propio código equivale a no haberlo entregado.
- **No sustituir el trabajo de análisis.** La comparación medida y las respuestas de la memoria son tuyas.

Anexo A. Configuración del ADC en CubeMX

Los tres programas parten de la misma configuración básica. Lo que cambia entre ellos es el modo de lectura y el número de canales.

AJUSTE	VALOR	POR QUÉ
Resolution	12 bits	La máxima del F401. Menos bits, menos precisión.
Scan Conversion Mode	Enabled con varios canales	Recorre los canales en secuencia. Con un solo canal, deshabilitado.
Continuous Conversion Mode	Enabled en DMA	El conversor no para. Con consulta, deshabilitado.
DMA Continuous Requests	Enabled en DMA	Sin esto el DMA transfiere una vez y se detiene.
DMA Mode	Circular	Reescribe el array indefinidamente. En modo Normal se para tras la primera vuelta.
Sampling Time	El más largo disponible	Los canales internos son lentos: con tiempos cortos la lectura sale ruidosa.

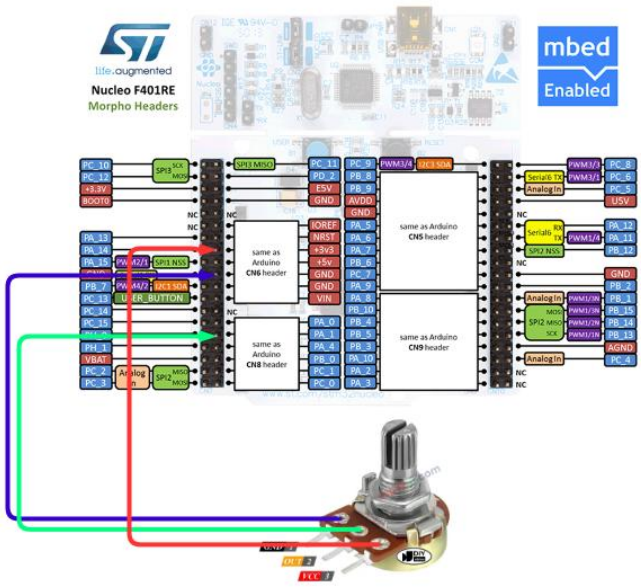
> El sensor de temperatura y la referencia interna hay que **habilitarlos explícitamente** en CubeMX, en la sección del ADC. Si no aparecen como canales disponibles, es que esa casilla está sin marcar.

Anexo B. Conexionado

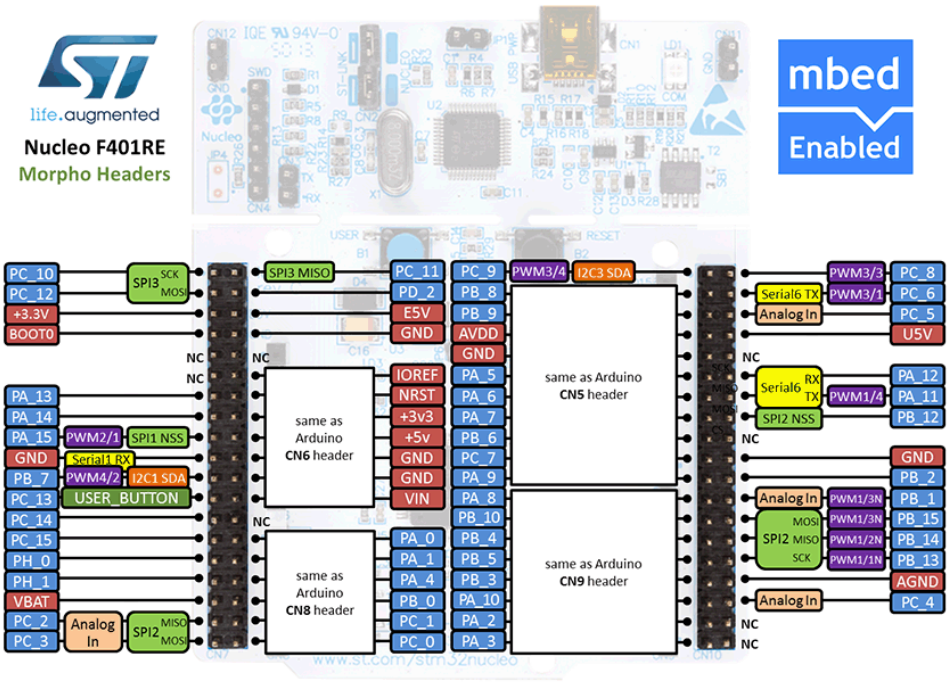
Solo hay un componente que cablear: el potenciómetro. Los otros dos canales están dentro del chip.

SEÑAL	PIN	NOTAS
Potenciómetro, cursor	**PA1**	Canal ADC1_IN1. Es el pin que usan los cuatro proyectos de referencia.
Potenciómetro, extremos	3,3 V y GND	Nunca a 5 V: el pin no lo tolera.
TM1638 · STB	PA10	Igual que en las prácticas anteriores
TM1638 · CLK	PB4	Igual que en las prácticas anteriores
TM1638 · DIO	PB5	Igual que en las prácticas anteriores

> **Los pines del módulo son los mismos que en las prácticas 2 y 4.** Si ya lo tienes cableado de una sesión anterior no hay que cambiar nada: basta con añadir el potenciómetro a PA1.



CONEXIÓN DEL POTENCIÓMETRO A LA NUCLEO-F401RE.



PINOUT DE LOS CONECTORES MORPHO, PARA LOCALIZAR PA1.

Anexo C. El sensor de temperatura interno

El STM32F401 lleva dentro un sensor conectado a un canal del ADC. No sirve para medir la temperatura ambiente: mide la del propio silicio, que está siempre algo por encima porque el chip se calienta al trabajar.

La conversión completa, con corrección por referencia interna, es la siguiente:

TEMPERATURA DEL CHIP, PASO A PASO

```
/* 1. Cuanto vale de verdad la referencia del conversor */
V_ref = 4095.0 * 1.21 / crudo[0];

/* 2. Tension que entrega el sensor */
V_sensor = crudo[1] * V_ref / 4095.0;

/* 3. De tension a grados, con las constantes del fabricante */
temperatura = (V_sensor - 0.76) * 1000.0 / 2.5 + 25.0;
```

> La precisión de este sensor es de varios grados: sirve para detectar tendencias y sobrecalentamientos, no para termometría. Las constantes V25 y Avg_Slope son valores típicos, no calibrados unidad por unidad, y varían de un chip a otro. No esperes que coincida con un termómetro.

Práctica 6 · Ordenador de a bordo

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesiones 9 y 10 de 13 · primera parte del proyecto final
Duración	4 horas presenciales, en dos sesiones
Modalidad	Individual, en el laboratorio
Requisitos previos	P2, P3, P4 y P5. Esta práctica las integra todas
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · potenciómetro · STM32CubeIDE
Entregables	Proyecto CubeIDE, tabla de comprobación del modelo y memoria (ver apartado 8)
Plazo	Una semana desde el final de la segunda sesión, por Moovi

2. Qué se construye

El cuadro de instrumentos de un coche. El potenciómetro hace de **pedal del acelerador**, un temporizador integra la física del vehículo diez veces por segundo, y el display muestra la magnitud que se elija con los pulsadores del módulo: velocidad, distancia recorrida, combustible restante, consumo medio, autonomía.

Los ocho LEDs son el **indicador de nivel de combustible**: se van apagando conforme se vacía el depósito.

> Es la primera de las dos sesiones del proyecto final. No hay concepto nuevo: hay que **integrar** lo de las cuatro prácticas anteriores en un sistema que hace varias cosas a la vez y que tiene que seguir respondiendo mientras las hace.

3. El modelo del vehículo

Todo lo que sigue es de obligado cumplimiento: los criterios de aceptación se comprueban con estos números.

MAGNITUD	VALOR	CONSTANTE
Capacidad del depósito	5,0 litros	`MAX_TANQUE`
Consumo de referencia	5,0 l / 100 km	`LITROS_REF_100KM`
Autonomía con el depósito lleno	100 km	`MAX_KM`
Periodo de integración	0,1 s (una décima)	`t_interval`
Retención al soltar el acelerador	2,8 m/s^2	`ACC * 10`

Los tres primeros valores son coherentes entre sí y conviene comprobarlo: cinco litros a cinco litros cada cien kilómetros dan exactamente cien kilómetros de autonomía. Esa coherencia es lo que permite validar el modelo sin salir a la carretera.

3.1 Cómo evoluciona

Cada décima de segundo, disparado por la interrupción del temporizador:

62. Si el acelerador está pisado, la **velocidad aumenta** en proporción a cuánto se pise, y se consume combustible en proporción a la velocidad.
63. Si no lo está, la velocidad **baja** a razón de 2,8 m/s^2, que es la retención del motor. Nunca por debajo de cero.
64. La **distancia** aumenta según la velocidad actual.
65. El **consumo medio** se recalcula como litros gastados por cada 100 km recorridos.

66. La **autonomía** sale del combustible que queda y del consumo medio.

> Fíjate en que el consumo medio y la autonomía **no son constantes del vehículo**: son magnitudes calculadas a partir de la historia del viaje. Si conduces suave mejoran, y si aceleras mucho empeoran. Eso es exactamente lo que hace un ordenador de a bordo real, y es lo que distingue esta práctica de un simple contador.
>
> **El consumo medio depende de cómo conduzcas, y eso es deliberado.** El motor solo gasta mientras el pedal está pisado, pero el coche sigue avanzando cuando lo sueltas. Por eso, acelerando sin soltar el consumo medio se queda clavado en **5,0 l/100 km** —a cualquier velocidad— y en cuanto levantas el pie empieza a bajar. Es lo que hace un coche real al ir en retención: recorre kilómetros sin gastar combustible.
>
> **El modelo no tiene velocidad máxima.** No se ha incluido resistencia al avance, así que con el pedal a fondo el coche pasa de 300 km/h y sigue subiendo. Tampoco existe una velocidad de crucero que puedas mantener: pisando sube, soltando baja. Añadir un rozamiento proporcional al cuadrado de la velocidad arregla las dos cosas y está propuesto en el anexo B.

3.2 Las ocho páginas del display

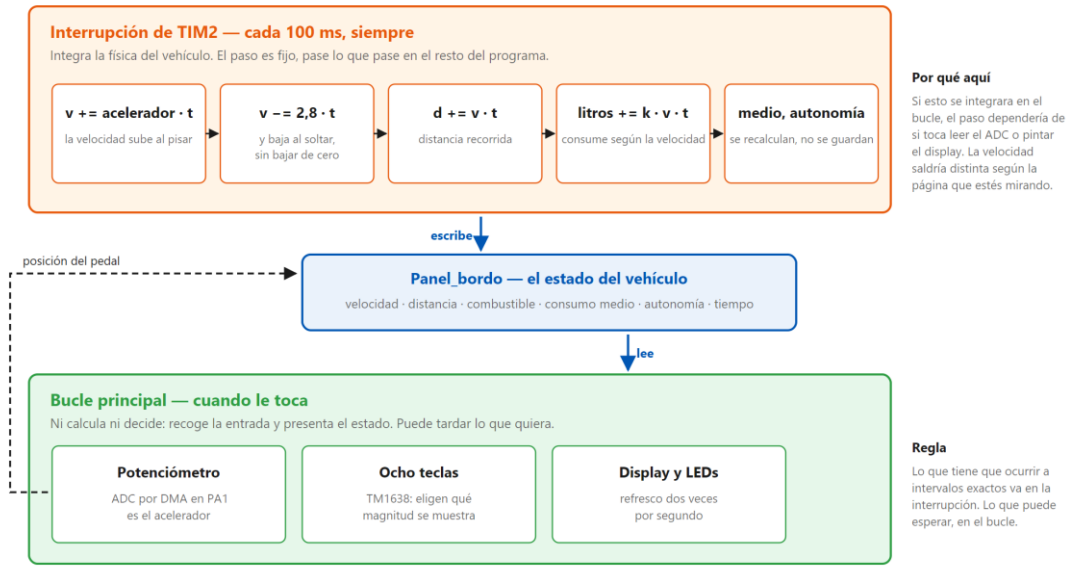
Cada tecla del módulo selecciona una magnitud. La letra inicial dice cuál se está viendo:

TECLA	MAGNITUD	PREFIJO	UNIDAD
S1	Temperatura del chip	`t`	°C
S2	Aceleración (posición del pedal)	`A`	0 a 10
S3	Velocidad	`v`	km/h
S4	Distancia recorrida	`d`	metros
S5	Combustible restante	`F`	litros

TECLA	MAGNITUD	PREFIJO	UNIDAD
S6	Autonomía	u	km
S7	Consumo medio	c	l / 100 km
S8	Tiempo de viaje		segundos

4. Por qué la interrupción es imprescindible aquí

La física se integra **dentro de la interrupción del temporizador**, cada décima de segundo exacta. Y tiene que ser así.



DOS RITMOS: LA INTERRUPCIÓN INTEGRA A INTERVALOS EXACTOS, EL BUCLE PRESENTA CUANDO PUEDE.

Si se integrase en el bucle principal, el paso de integración dependería de lo que el bucle estuviera haciendo: leer el ADC, escanear los pulsadores, escribir en el display. La velocidad calculada saldría distinta según qué página estuvieras mirando, que es un absurdo físico.

> Este es el argumento que justifica toda la Práctica 4. Un temporizador por interrupción da un paso de integración **constante y garantizado**, independiente de la carga del programa. La simulación es correcta porque el paso es fijo, no porque el código sea rápido.

5. Desarrollo paso a paso

Paso 1. Leer el acelerador

Lee el potenciómetro por DMA, como en P5, y muestra en el display la posición del pedal escalada de 0 a 10. Comprueba que responde de forma suave al girarlo.

Paso 2. Velocidad y distancia

Configura TIM2 para interrumpir cada 100 ms e integra la velocidad y la distancia. Muestra la velocidad y comprueba dos cosas: que al pisar sube, y que al soltar baja hasta pararse.

Paso 3. Combustible y consumo

Añade el consumo y el combustible restante. Es el momento de validar el modelo: con el pedal quieto en la posición que dé 100 km/h, el consumo medio debe estabilizarse cerca de **5 l/100 km**. Si no, algo está mal escalado.

Paso 4. Las páginas y los LEDs

Selecciona la magnitud con los pulsadores y saca el nivel de combustible por los ocho LEDs. El paso de una página a otra debe ser inmediato: si tarda, es que el bucle está bloqueado en algún sitio.

Paso 5. Valida el modelo

Con el cronómetro del móvil, comprueba que el contador de tiempo de viaje mide segundos de verdad, y que la distancia recorrida en un minuto a velocidad conocida coincide con lo que dice la física.

Paso 6. Atrapa un instante concreto de la simulación

El modelo cambia diez veces por segundo, así que mirar valores en la pantalla solo permite ver dónde acaba, no cómo llega. Con un punto de ruptura condicional se puede congelar un instante elegido.

67. Abre `Core/Src/Bordo.c` —la física **no está en `main.c`**— y pon un **punto de ruptura condicional** en la línea que actualiza la velocidad en km/h, con la condición `Panel_bordo.v_kmh > 100.0f`.
68. Acelera. El programa se detendrá exactamente al cruzar esa velocidad.
69. Inspecciona la distancia recorrida, el combustible y el consumo medio en ese punto, y comprueba si son coherentes entre sí.

> Aquí se nota bien lo que el capítulo 11 llama **efecto observador**: al parar, el coche de la simulación se para, pero el reloj del chip no. Al reanudar, la base de tiempo se pone al día de golpe. Observa qué le pasa al tiempo de viaje y explícalo en la memoria.

6. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Girar el potenciómetro con el vehículo parado.	La página de aceleración sigue el mando de forma continua, entre 0 y 10.
CA-2	Pisar a fondo unos segundos y soltar.	La velocidad sube al pisar y baja al soltar hasta **quedarse en 0**, sin pasar a negativo.
CA-3	Desde 100 km/h, soltar el acelerador y cronometrar.	Tarda del orden de **10 segundos** en pararse. Si tarda 1 s, la deceleración está mal escalada.
CA-4	Acelerar **sin soltar** desde parado y mirar el consumo medio. Después soltar el pedal un buen rato y volver a mirarlo.	Acelerando sin soltar se queda en **5,0 l/100 km**, sea cual sea la velocidad. Al soltar **baja**, porque el coche sigue avanzando sin consumir.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-5	Comparar el tiempo de viaje con un cronómetro externo, un minuto.	Coincide con menos de **1 s de diferencia**.
CA-6	Cambiar de página con los pulsadores mientras el coche circula.	El cambio es inmediato y **la simulación no se detiene** ni da saltos.

7. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Lectura del acelerador	CA-1: el ADC alimenta la simulación de forma continua.	Lee el potenciómetro pero a saltos o con la escala mal.	1,5
2. Física del vehículo	CA-2 y CA-3: velocidad y deceleración creíbles.	Acelera bien pero la deceleración está mal escalada.	2,0
3. Coherencia del modelo	CA-4: acelerando sin soltar el consumo medio da 5,0, y se entiende por qué al levantar el pie baja.	Calcula el consumo pero no sabe explicar por qué cambia según se conduzca.	2,0
4. Base de tiempo	CA-5: la integración va por interrupción y el tiempo es correcto.	Integra en el bucle principal en lugar de en la interrupción.	2,0

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
5. Interfaz	CA-6: páginas y LEDs, sin bloquear la simulación.	Las páginas funcionan pero cambiar de una a otra frena la simulación.	1,5
6. Memoria	Las dos cuestiones del apartado 8.	Responde una de las dos, o sin justificar.	1,0

8. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin Debug ni Release**. Nombre: ACII_P6_Apellido1_Apellido2_Nombre.zip
- **Tabla de validación**: los valores medidos en CA-3, CA-4 y CA-5 frente a los esperados.
- **Memoria breve**, máximo dos páginas:
    - Explica por qué la física tiene que integrarse en la interrupción y no en el bucle principal. Describe qué observarías si estuviera en el bucle.
    - El consumo medio se calcula dividiendo litros entre distancia. ¿Qué ocurre en los primeros instantes, cuando la distancia recorrida es casi cero? ¿Cómo lo evita el código?

9. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.
- **No sustituir la validación.** Las medidas de la tabla las tomas tú, con la placa y un cronómetro.

Anexo A. Versión para PC

En `_simulacion_pc/` hay una versión del mismo modelo escrita para ordenador, con las interrupciones emuladas mediante señales de UNIX e `itimerval`. Compila con `gcc` y se ejecuta en un terminal.

Sirve para dos cosas: entender el modelo sin pelearse con el hardware, y comparar cómo se expresa la misma idea —una rutina que se ejecuta a intervalos fijos— en dos entornos muy distintos. Es también la referencia contra la que se validó la versión de la placa.

Anexo B. Ampliaciones

- **Limitar la velocidad máxima.** Tal como está, el modelo no tiene tope: con el pedal a fondo el vehículo pasa de 300 km/h. Añadir rozamiento proporcional al cuadrado de la velocidad lo arregla y es físicamente correcto.
- **Aviso de reserva**: que los LEDs parpadeen cuando queden menos de 0,5 litros.
- **Consumo instantáneo** además del medio, y comparar cómo reacciona cada uno al pisar.
- **Cuentakilómetros parcial** que se ponga a cero con un pulsador, conservando el total.
- **Registro por el puerto serie** de velocidad y consumo cada segundo, para dibujar la curva después.

Anexo C. Conexionado

SEÑAL	PIN	PAPEL
Potenciómetro	PA1	Pedal del acelerador (ADC1_IN1)
TM1638 · STB	PA10	Display y LEDs
TM1638 · CLK	PB4	
TM1638 · DIO	PB5	

Práctica 7 · Semáforo de salida: tiempo de reacción

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 11 de 13 · práctica final
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	P2, P3, P4 y P5. Esta práctica las usa todas
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · biblioteca TM1638
Entregables	Proyecto CubeIDE, tabla de tiempos y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la sesión, por Moovi

2. De qué va

En la parrilla de un Gran Premio se encienden ocho luces rojas, una a una. Cuando llevan todas encendidas un rato **que nadie sabe cuánto va a durar**, se apagan de golpe. Ese apagón es la señal de salida, y lo que separa a un piloto bueno de uno normal son un par de centésimas.

Vas a construir exactamente eso con los ocho LEDs del módulo, y a medir tu propio tiempo de reacción en milisegundos. La máquina guarda el mejor tiempo de la sesión.

> Es la práctica final y no introduce ningún concepto nuevo: **usa los cuatro anteriores a la vez**. Máquina de estados de P2, medida de tiempo sin deriva de P3, interrupción

> externa de P4, y una pregunta que viene de P5: de dónde se saca azar en un chip que no tiene generador de números aleatorios.

3. Cómo es una partida



LAS CUATRO FASES DE UNA PARTIDA Y LO QUE SE MIDE.

70. **LISTO.** Display en `----`, LEDs apagados. Una pulsación arranca la partida.
71. **SEMÁFORO.** Los LEDs se encienden uno a uno, de izquierda a derecha, cada 400 ms. En total 3,2 segundos.
72. **ARMADO.** Las ocho encendidas. Aquí la máquina espera **entre 1 y 4 segundos**, y esa duración cambia en cada partida.
73. **MIDIENDO.** Los LEDs se apagan de golpe: esa es la señal. El cronómetro arranca en ese instante.
74. **RESULTADO.** Al pulsar, el display muestra el tiempo de reacción. Otra pulsación empieza de nuevo.

Y la trampa, que es lo que convierte el ejercicio en juego: **pulsar antes de que se apaguen es salida falsa**. El display muestra `SALIdA` y la partida se pierde. Como la espera es impredecible, no se puede anticipar: hay que reaccionar de verdad.

3.1 Lo que se ve en el display

SITUACIÓN	DISPLAY
En reposo	`----`
Justo al apagarse los LEDs	`AHOrA`
Resultado, por ejemplo 234 ms	`r0.234`
Salida falsa	`SALIdA`
Más de 2 s sin reaccionar	`LEnto`

> **Un display de siete segmentos no puede con todas las letras**, así que elige los textos entre los que la biblioteca sabe pintar: el espacio, `-`, los diez dígitos, `ABCEFHIJLOPSU`, `_` y `abcdhinortuy`. No existen la `G`, la `K`, la `M`, la `Q`, la `R`, la `V`, la `W`, la `X`, la `Z`, ni las mayúsculas `D`, `N`, `T` e `Y`. Si pides un carácter que no está, `tm1638_DisplayTxt` no falla en silencio: escribe `Error` seguido de la posición. Por eso el mensaje de salida es `AHOrA` y no `Go`.

4. El problema interesante: ¿de dónde sale el azar?

Si la espera del estado ARMADO fuese siempre la misma, el juego no valdría nada: a la tercera partida cualquiera la cuenta y se anticipa. Hace falta que sea impredecible.

Y ahí aparece un problema de verdad: **el STM32F401 no tiene generador de números aleatorios por hardware**. Algunos STM32 mayores sí lo llevan; este no. Un microcontrolador recién arrancado es la máquina más predecible del mundo: si el programa es el mismo y el estado inicial es el mismo, la secuencia es idéntica siempre.

> Prueba a poner una semilla fija y jugar tres partidas seguidas después de pulsar el botón de reinicio. Las esperas serán exactamente las mismas, en el mismo orden. Es un buen momento para entender qué significa «pseudoaleatorio».

Hay que buscar la entropía fuera del chip. Algunas fuentes posibles:

FUENTE	QUÉ TAL	POR QUÉ
Instante en que el jugador arranca	**buena**	Nadie pulsa dos veces en el mismo milisegundo. Es entropía humana, y no cuesta nada.
Bits bajos del ADC con la entrada al aire	razonable	Ruido térmico real, pero hay que configurar el ADC y el sesgo no es despreciable.
`HAL_GetTick()` en el arranque	mala	Vale casi lo mismo siempre: el programa tarda lo mismo en llegar hasta ahí.
Semilla fija	inútil	La misma partida una y otra vez.

La solución de referencia usa la primera: toma el valor de `HAL_GetTick()` en el momento exacto en que el jugador arma la salida y lo mezcla con la semilla. A partir de ahí, un generador congruencial lineal produce la secuencia.

GENERADOR CONGRUENCIAL LINEAL

```
semilla = semilla * 1664525u + 1013904223u;
```

Esas dos constantes no son arbitrarias: están elegidas para que el generador recorra los 2^{32} valores posibles antes de repetirse. Con otras cualesquiera el periodo puede ser ridículamente corto.

5. Medir bien el tiempo de reacción

Un tiempo de reacción humano ronda los **200 a 300 milisegundos**. Un buen piloto de Fórmula 1 baja de 200. Para que la medida signifique algo, el error del sistema tiene que ser muy inferior a eso.

El instante de la pulsación **se sella dentro de la función de atención a la interrupción**, no en el bucle principal:

CAPTURA POR INTERRUPCIÓN

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin == B1_Pin) {
        tPulsacion = HAL_GetTick();   /* se sella aqui, no en el bucle
*/
        hayPulsacion = true;
    }
}
```

> Si la marca de tiempo se tomase en el bucle principal, el resultado dependería de lo que el bucle estuviera haciendo en ese momento: escribir en el display, por ejemplo, tarda milisegundos. La interrupción entra al instante, sea cual sea el punto del programa. **Esto es lo que P4 servía para poder hacer.**

En la memoria tendrás que estimar cuánto error introduce cada eslabón de la cadena: el rebote del pulsador, la latencia de la interrupción y la resolución de 1 ms de `HAL_GetTick()`.

6. Desarrollo paso a paso

Paso 1. La secuencia de luces

Sin juego todavía: al pulsar, que se enciendan los ocho LEDs uno a uno cada 400 ms y se queden encendidos. Cadencia con `HAL_GetTick()`, como en P3, sin `HAL_Delay`.

Paso 2. El apagón y la medida

Añade una espera fija de 2 s con todo encendido, luego apaga los LEDs y arranca la medida. Muestra el tiempo al pulsar. Con espera fija podrás comprobar que la medida es correcta antes de meter el azar.

Paso 3. La espera impredecible

Sustituye los 2 s fijos por un valor aleatorio entre 1 y 4 s, con la semilla tomada del instante en que el jugador arranca la partida. Comprueba que dos partidas seguidas dan esperas distintas.

Paso 4. La salida falsa

Detecta la pulsación durante SEMÁFORO y durante ARMADO, y trátala como salida falsa. Cuidado con un detalle: al llegar a MIDIENDO hay que **descartar cualquier pulsación pendiente**, o un rebote de la anterior contará como reacción de 0 ms.

Paso 5. El récord y el torneo

Guarda el mejor tiempo de la sesión. Y entonces la parte divertida: comparad los resultados de la clase.

Paso 6. Comprueba dónde se sella el instante

Todo el sentido de esta práctica está en que el tiempo se capture dentro de la interrupción y no en el bucle. Es una afirmación comprobable.

75. Pon un punto de ruptura en la línea de la rutina de atención que guarda el instante.
76. Juega una partida. Al pulsar, el programa se detendrá ahí.
77. Inspecciona el valor que acaba de guardarse y compáralo con el instante en que se apagaron los LEDs.

> **Y ahora fíjate en lo que le pasa a la medida.** El tiempo que el juego muestre al continuar será absurdo, porque has estado parado varios segundos en medio de la medición. No es un fallo: es el ejemplo más claro de que hay cosas que no se pueden medir mientras se observan. Explícalo en la memoria en dos líneas.

Para comprobar la afirmación de verdad —que capturar en la interrupción da mejores tiempos que capturar en el bucle— no sirve el depurador: hay que medir a velocidad real, que es lo que se hace en el torneo del paso anterior.

7. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Arrancar una partida y observar la secuencia.	Los ocho LEDs se encienden uno a uno, y la secuencia completa dura **3,2 s ± 0,2 s**.
CA-2	Jugar cinco partidas seguidas y anotar la espera con todo encendido.	Las cinco esperas son **distintas** y todas caen entre 1 y 4 segundos.
CA-3	Reaccionar normalmente varias veces.	El tiempo mostrado está en el rango humano razonable, **entre 150 y 400 ms**. Nunca 0 ni valores absurdos.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-4	Pulsar durante la secuencia de luces, y en otra partida con las ocho encendidas.	En los dos casos muestra `SALIdA` y no da ningún tiempo.
CA-5	Tras una salida falsa, jugar otra partida normal.	Funciona con normalidad: la salida falsa no deja el sistema en mal estado.
CA-6	Revisar el código.	El instante de la pulsación se toma **en la interrupción**, y el bucle principal no contiene esperas bloqueantes.

8. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Secuencia y temporización	CA-1: los ocho LEDs con la cadencia correcta.	La secuencia funciona pero con `HAL_Delay` en lugar de cadencia no bloqueante.	2,0
2. Espera impredecible	CA-2: esperas distintas en el rango pedido.	Es aleatoria pero con semilla fija: la misma serie tras cada reinicio.	2,0
3. Medida fiable	CA-3 y CA-6: captura en la interrupción, tiempos coherentes.	Mide bien pero sellando el instante en el bucle principal.	2,5

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
4. Salida falsa	CA-4 y CA-5: se detecta en las dos fases y el sistema se recupera.	La detecta en una sola de las dos fases, o queda colgado después.	1,5
5. Calidad del código	Máquina de estados legible, constantes con nombre, `volatile` donde toca.	Funciona pero mezcla decisiones y acciones, o faltan constantes.	1,0
6. Memoria	Las dos cuestiones del apartado 9, razonadas.	Responde una de las dos, o sin justificar.	1,0

9. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin Debug ni Release**. Nombre: ACII_P7_Apellido1_Apellido2_Nombre.zip
- **Tabla con diez tiempos** de reacción propios, su media y el mejor.
- **Memoria breve**, máximo dos páginas, con estas dos cuestiones:
    - Estima el error total de tu medida: rebote del pulsador, latencia de la interrupción y resolución de `HAL_GetTick()`. ¿Cuál domina, y es despreciable frente a los 200 ms que tarda una persona?
    - Explica por qué tomar la semilla del instante en que el jugador arranca la partida da mejores resultados que tomarla de `HAL_GetTick()` nada más encender la placa.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.

- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.
- **No sustituir el trabajo de medida.** Los diez tiempos los consigues tú, jugando.

Anexo A. Ampliaciones, para quien acabe pronto

- **Media de las últimas cinco partidas**, además del mejor tiempo. Obliga a llevar un buffer circular.
- **Modo a dos jugadores**: uno usa el pulsador azul y otro una tecla del módulo. Gana quien reaccione antes, y el display dice cuál.
- **Resolución por debajo del milisegundo**, leyendo el registro de cuenta de SysTick además de `HAL_GetTick()`. Con 84 MHz la resolución teórica baja a 12 ns; discute si eso significa algo cuando lo que se mide es una persona.
- **Dificultad ajustable con el potenciómetro** de P5: que gobierne la velocidad de la secuencia o el rango de la espera.
- **Detección de trampa**: si alguien mantiene el pulsador apretado desde antes, el sistema debe darse cuenta.

Anexo B. Conexionado

El mismo de todas las prácticas. No hace falta cablear nada nuevo.

SEÑAL	PIN	PAPEL EN EL JUEGO
TM1638 · STB	PA10	Display y LEDs
TM1638 · CLK	PB4	
TM1638 · DIO	PB5	
Pulsador azul de usuario	PC13	El botón del juego, por interrupción externa

ANEXOS

Material de consulta

Anexo A. El juego de caracteres del TM1638

Los **38 caracteres** que la biblioteca sabe dibujar, con el patrón de segmentos que envía y qué segmentos enciende. Pedir cualquier otro hace que el display muestre un error.

> Esta tabla **no está copiada a mano**: se genera leyendo la propia biblioteca al construir el libro. Si algún día se añade un carácter, aparecerá aquí solo.

CAR.	PATRÓN	SEGMENTOS	CAR.	PATRÓN	SEGMENTOS
0	`0x3f`	a b c d e f	**l**	`0x06`	b c
1	`0x06`	b c	**J**	`0x0e`	b c d
2	`0x5b`	a b d e g	**L**	`0x38`	d e f
3	`0x4f`	a b c d g	**O**	`0x3f`	a b c d e f
4	`0x66`	b c f g	**P**	`0x73`	a b e f g
5	`0x6d`	a c d f g	**S**	`0x6d`	a c d f g
6	`0x7d`	a c d e f g	**U**	`0x3e`	b c d e f
7	`0x07`	a b c	**a**	`0x5f`	a b c d e g
8	`0x7f`	a b c d e f g	**b**	`0x7c`	c d e f g
9	`0x6f`	a b c d f g	**c**	`0x58`	d e g
espacio	`0x00`	(ninguno)	**d**	`0x5e`	b c d e g
_	`0x08`	d	**h**	`0x74`	c e f g
-	`0x40`	g	**i**	`0x04`	c

CAR.	PATRÓN	SEGMENTOS	CAR.	PATRÓN	SEGMENTOS
A	0x77	a b c e f g	**n**	0x54	c e g
B	0x7f	a b c d e f g	**o**	0x5c	c d e g
C	0x39	a d e f	**r**	0x50	e g
E	0x79	a d e f g	**t**	0x78	d e f g
F	0x71	a e f g	**u**	0x1c	c d e
H	0x76	b c e f g	**y**	0x6e	b c d f g

Lo que no se puede escribir

QUÉ FALTA	CUÁLES
Mayúsculas que no existen	D, G, K, M, N, Q, R, T, V, W, X, Y, Z
Minúsculas que no existen	todas salvo las de la tabla
Parejas indistinguibles	B se ve como 8 · O como 0 · I como 1 · S como 5

Para símbolos que no estén en la tabla se puede escribir el patrón de segmentos directamente, encendiendo los que hagan falta. El capítulo 10 explica cómo.

Anexo B. Conexionado completo

Todas las prácticas usan la misma asignación de pines. Respetándola, cualquier programa del bloque funciona sin cambiar nada.

PIN	ETIQUETA	QUÉ ES	PRÁCTICAS
PA5	`LD2`	LED verde de la placa. Ya cableado	P1, P4
PC13	`B1`	Pulsador azul. Ya cableado. **Activo a cero**	P1, P4, P5, P6, P7
PB4	`CLK`	Reloj del módulo TM1638	P2 a P7
PB5	`DIO`	Dato del TM1638. **Bidireccional**	P2 a P7
PA10	`STB`	Selección del TM1638	P2 a P7
PA1	`ADC1_IN1`	Cursor del potenciómetro	P5, P6
3V3	—	Alimentación del módulo y del potenciómetro	P2 a P7
GND	—	Masa común	P2 a P7

> El potenciómetro va como **divisor de tensión**: un extremo a 3V3, el otro a masa y el cursor a PA1. Conectado solo por dos patillas, la entrada queda flotando y el valor salta sin que nadie toque el mando.

Dos pines más aparecen configurados en todos los proyectos aunque ninguna práctica los use: **PA2** y **PA3**, la USART2, que la placa tiene cableada al puerto serie del programador. Se dejan porque los genera la herramienta al elegir la placa.

Alimentación

El módulo se alimenta desde la propia placa, a **3,3 V**. Los pines del microcontrolador trabajan a esa tensión: **no conectes a una entrada nada que entregue 5 V**, porque puede estropear el pin.

Anexo C. Glosario

Los términos que aparecen a lo largo del libro, con la definición que se usa aquí.

TÉRMINO	QUÉ ES
ADC	Conversor analógico-digital. Traduce una tensión a un número.
Antirrebote	Filtro que evita contar varias veces una sola pulsación.
APB1, APB2	Los dos buses de periféricos. APB1 va a la mitad de frecuencia.
Bit-banging	Generar un protocolo moviendo pines desde el programa, sin periférico que lo haga.
Cadencia no bloqueante	Hacer algo cada cierto tiempo sin detener el programa, comprobando el reloj en vez de esperar.
Deriva	Error de tiempo que se acumula y crece sin parar. Distinto del temblor.
DMA	Periférico que mueve datos entre memoria y periféricos sin ocupar al procesador.
EXTI	El bloque que permite que un pin genere una interrupción.
Flanco	El instante en que una señal cambia. Se opone a nivel.
Flash	La memoria donde vive el programa. No se borra al apagar.
HAL	La biblioteca de ST que da funciones en vez de obligar a escribir en registros.
HSI	El oscilador interno de 16 MHz. Es el que usan las prácticas.
Latencia de la Flash	Ciclos que el procesador espera al leer de la Flash. A 84 MHz son dos.

TÉRMINO	QUÉ ES
Nivel	El estado en que está una señal. Se opone a flanco.
NVIC	El controlador que decide qué interrupción se atiende y en qué orden.
PLL	El circuito que multiplica la frecuencia del oscilador para llegar a 84 MHz.
Prescalador	Divisor que baja la frecuencia que entra a un temporizador.
Pull-up, pull-down	Resistencias que fijan el nivel de una entrada cuando nada la gobierna.
SRAM	La memoria de datos. Se pierde al apagar.
SysTick	El temporizador de 24 bits que lleva el núcleo. La HAL lo usa para contar milisegundos.
Temblor	Variación de un periodo que no se acumula. Distinto de la deriva.
TM1638	El controlador del módulo LED&KEY: ocho dígitos, ocho LEDs y ocho pulsadores con tres hilos.
USER CODE	Las marcas entre las que el código sobrevive a una regeneración.
volatile	Palabra que obliga al compilador a releer una variable en cada acceso, en vez de suponer que no cambia.